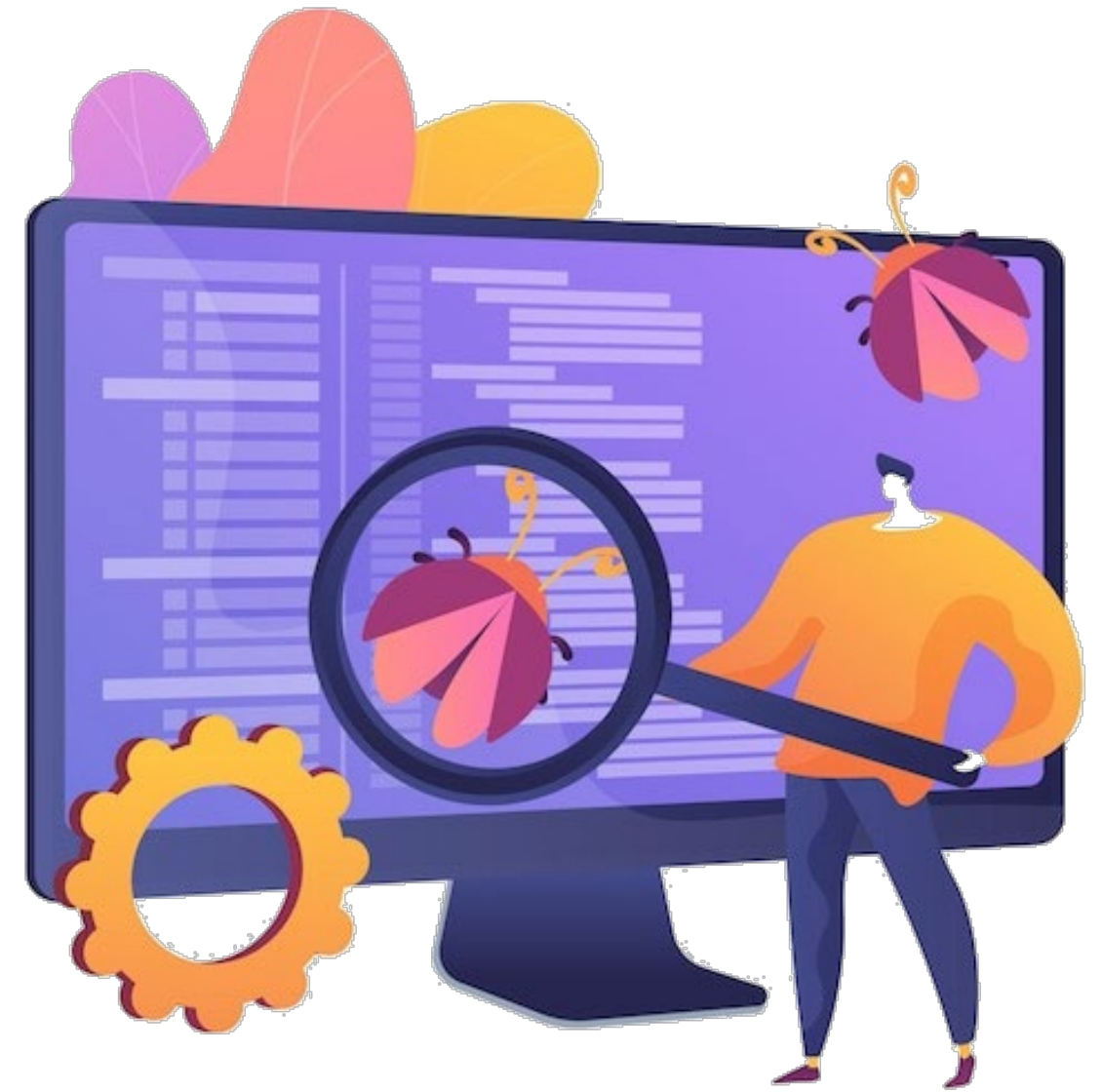


*the*knowledgeacademy

ISTQB Software Testing Foundation Training



ISTQB Software Testing Foundation

Course Time Table and Matrix

Course Time Table and Matrix

S.no.	Day	Start	End	Supporting Material – Delegate Pack	Key Learning Points	Detail	Syllabus References
1	1	09:00	09:30	Slide 1-4	Course Structure/Objective /Introductions	Explain the Course Structure and Rough Agenda/Timings	N/A
2	1	09:30	10:00	Slide 5-13	What is Testing?	Briefly go through about What is Testing	1.1
2	1	10:00	10:30	Slide 14-28	Why is Testing Necessary?	Learn the Features Of Testing	1.2
15-Minute Break							
2	1	10:45	11:00	Slide 29-34	Testing Principles	Implement Principles Of Testing	1.3
2	1	11:00	11:30	Slide 35-50	Test Activities, Testware and Test Roles	Briefly go through about Test Activities/ Testware/Test Role	1.4
2	1	11:30	12:30	Slide 51-57	Essential Skills and Good Practices in Testing	Develop an understanding on Skills Required for Testing	1.5
60-Minute Lunch Break							
3	1	01:30	02:30	Slide 59-83	Testing in the Context Of a Software Development Lifecycle	Learn the Lifecycles Of Software Development	2.1
3	1	02:30	03:15	Slide 84-109	Test Levels and Test Types	Briefly GO through the different levels and types Of test	2.2
3	1	03:15	03:45	Slide 110-115	Maintenance Testing	Gain Knowledge about the Maintenance Of Testing	2.3
3	1	03:45	04:00	N/A	Recap and Homework	Revision	N/A

Course Time Table and Matrix

S.no.	Day	Start	End	Supporting Material – Delegate Pack	Key Learning Points	Detail	Syllabus References
4	2	09:00	10:00	Slide 117-124	Static Testing Basics	Learn more about the Static Testing and its importance	3.1
4	2	10:00	10:45	Slide 125-135	Feedback and Review Process	Develop an understanding Of process Of Feedback and Review	3.2
15-Minute Break							
5	2	11:00	11:45	Slide 137-141	Test Techniques Overview	Briefly GO through the overview Of Testing Techniques	4.1
5	2	11:45	1:45	Slide 142-176	Black-box Test Techniques	Develop an understanding on Black-box Test Techniques	4.2
60-Minute Lunch Break							
5	2	02:45	04:15	Slide 177-199	White-box Test Techniques	Develop an understanding on White-box Test Techniques	4.3
5	2	04:15	05:30	Slide 200-211	Experience-based Test Techniques	Briefly GO through the Experience-based Test Techniques based on the Knowledge, Expertise, and Intuition Of the Testing Team to Design and Execute Tests	4.4
5	2	05:30	06:30	Slide 212-233	Collaboration-based Test Approaches	Learn the Collaboration-based Test Approaches	4.5
3	2	06:30	06:45	N/A	Recap and Homework	Revision	N/A
6	3	09:00	10:00	Slide 234-264	Test Planning	Learn the objectives and Resources Required for Testing	5.1

Course Time Table and Matrix

S.no.	Day	Start	End	Supporting Material – Delegate Pack	Key Learning Points	Detail	Syllabus References
6	3	10:00	11:15	Slide 265-286	Risk Management	Briefly go through systematic process of Identifying, Assessing, and Mitigating Risks in order to Minimise the objective	5.2
15-Minute Break							
6	3	11:30	01:00	Slide 287-303	Test Monitoring, Test Control and Test Completion	Gain Knowledge about Tracking, Making Adjustments, and Testing Activities of desired objective	5.3
60-Minute Lunch Break							
6	3	02:00	03:00	Slide 304-311	Configuration Management	Learn the discipline of Identifying, Documenting, Controlling, and Tracking the Components and Changes in a System or Software Throughout its Lifecycle	5.4
6	3	03:00	03:45	Slide 312-319	Defect Management	Learn the discipline of Identifying, Documenting, Controlling, and Tracking the Components and Changes in a System or Software Throughout its Lifecycle	5.5
7	3	03:45	04:15	Slide 321-328	Tool Support for Testing	Gain Knowledge about the various Tools of Testing	6.1
7	3	04:15	04:30	Slide 329-333	Benefits and Risks of Test Automation	Develop an understanding of various Benefits and Risks of Test Automation	6.2
7	3	04:45	05:00	N/A	Recap and Exam Discussion/Sample Exam	Develop an understanding of various Benefits and Risks of Test Automation	N/A

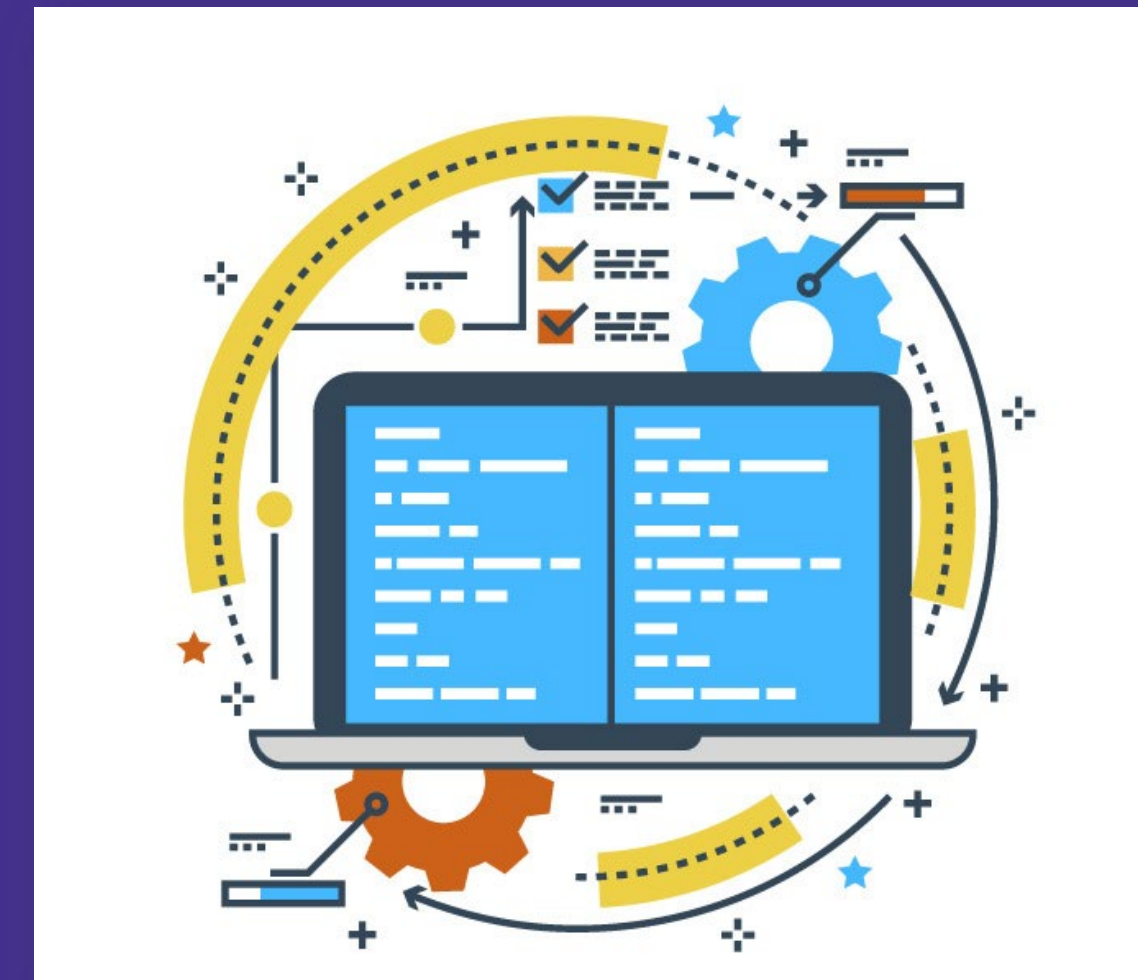
About The Knowledge Academy

The world's largest provider of classroom and online training courses

- ✓ World Class Training Solutions
- ✓ Subject Matter Experts
- ✓ Highest Quality Training Material
- ✓ Accelerated Learning Techniques
- ✓ Project, Programme, and Change Management, ITIL® Consultancy
- ✓ Bespoke Tailor Made Training Solutions
- ✓ PRINCE2®, MSP®, ITIL®, Soft Skills, and More

Course Syllabus

- **Domain 1:** Fundamentals of Testing
- **Domain 2:** Testing Throughout the Software Development Lifecycle
- **Domain 3:** Static Testing
- **Domain 4:** Test Analysis and Design
- **Domain 5:** Managing the Test Activities
- **Domain 6:** Test Tools



Domain 1: Fundamentals of Testing

Outline of Domain 1

- *Module 1: What is Testing?*
- *Module 2: Why is Testing Necessary?*
- *Module 3: Testing Principles*
- *Module 4: Test Activities, Testware and Test Roles*
- *Module 5: Essential Skills and Good Practices in Testing*

Module 1: What is Testing?



What is Testing?

- Testing refers to the process of evaluating a system, product, or application to identify any defects, errors, or discrepancies between its expected and actual behaviour. It may include static or dynamic testing.
- It is an essential part of the Software Development Life Cycle (SDLC) and is performed to ensure the quality, reliability, and correctness of the software.
- Testing involves executing a program or system with the intention of finding bugs or verifying that it meets specified requirements.
- Two common types of testing:
 1. Static testing involves reviewing and inspecting software documentation and code for defects without executing the program
 2. Dynamic testing is testing software by executing it, examining actual results against expected outcomes, and evaluating its behavior.

Here are five key characteristics of software testing:

1. **Systematic and Structured:** Software testing follows a systematic procedure, beginning with the identification of objectives, test planning, test case design, and execution, followed by result analysis and reporting. Proper documentation is maintained throughout the testing lifecycle.
2. **Objective-oriented:** The primary purpose of testing is to identify defects and ensure that the software functions according to the specified requirements. Testing is not about proving that a system works but rather about finding where it doesn't. Different testing types and levels aim at different objectives, such as unit testing for individual component correctness or usability testing for user experience.
3. **Iterative:** Software testing often involves multiple iterations. As bugs are identified and fixed, tests are rerun to ensure that fixes don't introduce new issues and that the original issues were resolved. This iterative process continues until the software reaches the desired quality level.
4. **Dynamic and Adaptable:** As software evolves, the testing process and test cases need to adapt to new features, changes in requirements, or identified vulnerabilities. The testing process must be agile enough to accommodate these changes without compromising on quality.
5. **Multifaceted:** There are different types of software testing, including but not limited to unit testing, integration testing, system testing, acceptance testing, performance testing, security testing, usability testing, and regression testing. Each of these has its methods, tools, and purposes. Software testing considers the software from various angles to ensure all aspects are evaluated.

What is Testing?

Test Objectives

- *The following are some objectives of testing:*

1. Ensuring Coverage

2. Evaluate Work Product

3. Verify Requirements

4. Validate Test Objects

5. Build Confidence

6. Reduce Risk

7. Share Information to Stakeholders

8. Find Failure and Defects

9. Verifying Test Objects

What is Testing?

1. **Ensuring Coverage:** Ensuring the test object's thorough examination to guarantee product quality and reliability.
2. **Evaluate Work Products:** Assessing key software development elements to align with project goals and ensure quality control.
3. **Verify Requirements:** Ensuring the software meets specified functionalities and behaviours.
4. **Validate Test Objects:** Checking if the test object meets stakeholder expectations and requirements.
5. **Build Confidence:** Validating the software's functionality and performance to establish trust in its quality.
6. **Reduce Risk:** Identifying and mitigating potential project risks through early software testing.

1. **Ensuring Coverage:** Ensuring coverage of a test object is a fundamental aspect of software testing that plays a crucial role in determining the quality and reliability of the product. The term "coverage" in this context refers to the degree to which the test object is examined and evaluated during the testing process.
2. **Evaluate Work Products:** Evaluating work products in software development encompasses the assessment of key elements such as requirements, user stories, designs, and code. This critical process ensures alignment with project goals and stakeholder expectations, providing quality control throughout the development lifecycle. Evaluating work products is an essential part of the software development lifecycle.
3. **Verify Requirements:** Verifying requirements involves ensuring that the software's functionalities and features align with the specified requirements. Testers check whether the application behaves as intended and meets the documented requirements, ensuring that it functions correctly and meets the expected criteria.
4. **Validate Test Objects:** Validating test objects is a crucial step in the software testing process that ensures that the item being tested meets the expectations and requirements laid down by the stakeholders.
5. **Build Confidence:** Building confidence in the quality of a test object hinges on rigorous software testing. Comprehensive test coverage ensures that various software aspects, from functionality to security, are evaluated thoroughly.
6. **Reduce Risk:** Software testing aims to minimise the likelihood of risks materialising. Each software project has uncertainties and potential risks that can impact its development and lifecycle. By incorporating risk management early in the development phase, testing helps identify and mitigate potential risks, ensuring a smoother and more controlled project execution.

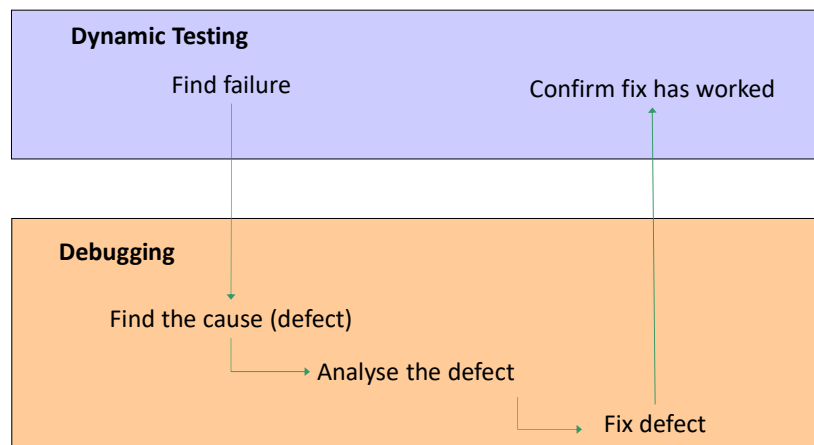
What is Testing?

7. **Share Information with Stakeholders:** Providing detailed testing insights to stakeholders for informed decision-making.
8. **Find Failures and Defects:** Detecting and addressing software issues early to meet user requirements.
9. **Verifying Test Objects:** Checking test objects for compliance with specific external standards and agreements.

7. **Share Information with Stakeholders:** Testing provides stakeholders with comprehensive information to make informed decisions regarding the quality of the software. This includes sharing details about test coverage, testing reports highlighting missing functionalities or identified issues and communicating technological or other constraints. The objective is to ensure stakeholders have a clear understanding of the quality-related challenges and considerations.
8. **Find Failures and Defects:** Another crucial objective of software testing is to discover failures and defects in the software. The primary goal is to uncover as many issues as possible while ensuring that the software meets user requirements. Detecting defects early in the testing cycle allows for prompt resolution and reduces the likelihood of these issues causing problems during later stages or after deployment.
9. **Verifying Test Objects:** Verifying test objects is a critical procedure in the software testing process, focusing on whether the test object meets specific standards, such as contractual agreements, legal mandates, and regulatory guidelines. This aspect of verification goes beyond functional correctness, delving into compliance with external rules and agreements.

What is Testing?

Testing and Debugging



© The Knowledge Academy Ltd

The relationship between **dynamic testing** and **debugging**:

- Dynamic testing often leads to the discovery of defects or issues.
- Once an issue is identified, debugging is initiated to understand and resolve the issue.
- After debugging, dynamic testing can be done again to ensure the defect has been addressed.

Testing and Debugging

Testing

- Testing refers to the identification of failures that result from flaws in the software (dynamic testing) or can outrightly uncover defects in the test object (static testing).
- It involves verifying the software's functionality, performance, and ability to handle various exceptional and boundary cases.



© The Knowledge Academy Ltd

Key components of dynamic testing include:

- **Test Cases:** These are specific conditions or sequences of events that testers use to determine if a software application functions correctly.
- **Test Data:** This is the data that is used in conjunction with test cases. It can be inputs to the system or data that the system processes.
- **Execution:** This is the process of running the software with the given test cases and test data.
- **Results:** After execution, testers get results which are then compared against expected outcomes.

Testing and Debugging

Debugging

- Debugging is the process within software development aimed at diagnosing and rectifying anomalies or defects. When a failure is triggered during dynamic testing, debugging focuses on understanding and resolving the underlying causes of this failure.
- The standard debugging procedure encompasses:
 - **Reproduction of a Failure:** Before addressing a problem, it's essential to consistently recreate the failure. This ensures that the issue is understood in its entirety and isn't sporadic.
 - **Diagnosis:** Once the failure is replicated, the next step is to identify its root cause. This diagnosis dives deep into the code or system to pinpoint precisely where and why the defect emerged.
 - **Fixing the Cause:** After diagnosing the root cause, appropriate remedies are implemented to rectify the defect, ensuring the software functions as intended.

© The Knowledge Academy Ltd

Key components of debugging include:

- **Issue Identification:** Recognising that there's a discrepancy between the expected and observed outcomes.
- **Issue Isolation:** Narrowing down the area of code or functionality where the issue resides.
- **Root Cause Analysis:** Understanding why the issue is happening. This might involve reviewing code, checking data flows, or analysing configurations.
- **Resolution:** Making necessary code or configuration changes to resolve the issue.
- **Verification:** After making changes, the affected area or the entire system might be re-tested to ensure the issue is truly resolved.

Testing and Debugging

Difference between Testing and Debugging

Testing	Debugging
Process of evaluating software to identify defects	Process of addressing issues that arise when dynamic testing reveals a failure
Determines if software meets specified requirements	Focuses on finding and resolving specific issues
Performed during various stages of software development	Performed after testing to address identified issues
Aims to ensure quality, reliability, and correctness	Aims to eliminate bugs and improve software behavior
Involves executing the software and analysing results	Involves analysing code, logs, and error messages
Goal is to validate functionality and uncover issues	Goal is to improve software functionality and performance.
Testing can be Static or Dynamic.	Utilises troubleshooting and code analysis techniques
Testing tools may be used to automate test execution	Debugging tools aims to identify the reasons behind this failure

Example:

Imagine you're developing a mobile application that helps users calculate their daily calorie intake.

Dynamic Testing:

Before releasing the app to the public, you decide to perform dynamic testing. You design a test case where a user logs meals totaling 2,000 calories. You input the meals and quantities into the app and expect it to display "2,000 calories consumed today." However, after execution, the app displays "1,500 calories consumed today."

Debugging:

Clearly, there's a discrepancy between the expected and actual outcomes. You initiate the debugging process. On inspecting the code, you find an error in the formula where the app miscalculates calories for certain food items. You make the necessary code adjustments to fix the issue.

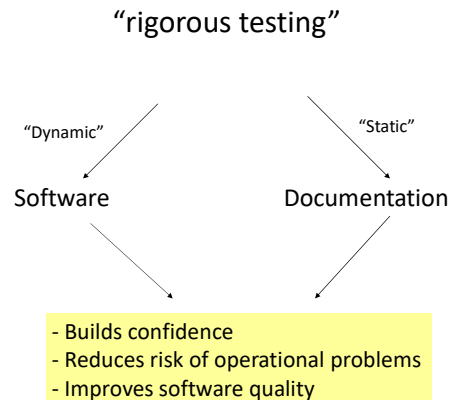


Module 2: Why is Testing Necessary?



Why is Testing Necessary?

The Role of Testing



© The Knowledge Academy Ltd

Role of Testing:

1.Ensuring Correctness & Completeness: Testing verifies that dynamic software operates according to its requirements while simultaneously ensuring that static documentation comprehensively and accurately represents the system's design and expected behaviour.

2.Enhancing Quality: By identifying defects in both software functionality and documentation clarity, testing aids in elevating the overall quality of the product and its accompanying resources.

3.Facilitating Communication: Testing static documentation, such as requirement specifications, ensures it effectively communicates the intended functionality and constraints. This, in turn, supports the accurate testing and development of dynamic software.

4.Mitigating Risks: For dynamic software, testing identifies potential vulnerabilities or malfunctioning areas. In terms of static documentation, it ensures there are no ambiguities or contradictions that might lead to flawed software implementation.

Supporting User Experience: Testing ensures that the dynamic software provides the desired user experience and that the static documentation (like user manuals) effectively guides users, enabling them to derive maximum value from the software.

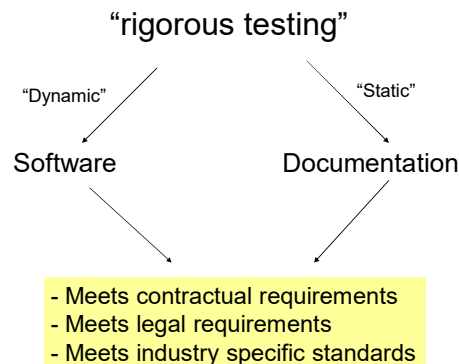
1.Promoting Consistency: Testing ensures that the dynamic software behaves consistently under various conditions and that the static documentation aligns with the software's actual behaviour, ensuring no discrepancies.

2.Driving Development and Maintenance: Testing static documentation can reveal gaps or areas for improvement, guiding subsequent software development. For dynamic software, testing feedback can inform necessary updates or fixes.

3.Validating Changes: Whenever updates or modifications are made, testing confirms that the dynamic software still functions as expected and that the static documentation remains relevant and aligned.

Why is Testing Necessary?

(Continued)



© The Knowledge Academy Ltd

Rigorous Testing:-

- **Comprehensiveness:** It covers all possible scenarios, including edge cases and boundary conditions.
- **Structure:** It often follows a systematic and methodical process, ensuring that no steps are overlooked.
- **Documentation:** Rigorous testing often requires detailed documentation, capturing both the testing process and results for future reference or audits.
- **Validation and Verification:** It ensures that the software or system not only meets the user's needs (validation) but also functions correctly (verification).

Dynamic Software:-

- **Nature:** Refers to software applications or components that execute or run to perform their functions. It's about behaviour during runtime.
- **Interaction:** Users or other systems can interact with dynamic software, triggering various operations or obtaining different outcomes based on inputs or conditions.
- **Flexibility:** Can respond to user inputs or environmental changes in real-time.
- **Examples:** Web applications, mobile apps, desktop software, and other executable programs.
- **Evaluation:** Dynamic software is evaluated using dynamic testing methods, which involve executing the software and checking its behaviour.

Static Documentation:-

- **Nature:** Refers to written documents, diagrams, or specifications that don't change unless manually updated. It's about information that remains constant until edited.
- **Interaction:** Users can read or refer to static documentation, but they cannot "execute" it. It serves as a reference or guide.
- **Flexibility:** Doesn't respond to user actions in real-time. Any change in the document requires manual editing and doesn't happen dynamically.
- **Examples:** Software requirement specification documents, user manuals, architectural diagrams, and design documents.
- **Evaluation:** Static documentation can be reviewed for accuracy, completeness, and clarity.

Why is Testing Necessary?

- *Software testing is of utmost importance for the following reasons:*

01 Defects Detection

02 Customer Satisfaction

03 Monetary Penalties

04 Ensuring Quality

05 Reduced Maintenance Costs

06 Performance and Reliability

07 Early Issue Detection

08 Business Continuity

Why is Testing Necessary?

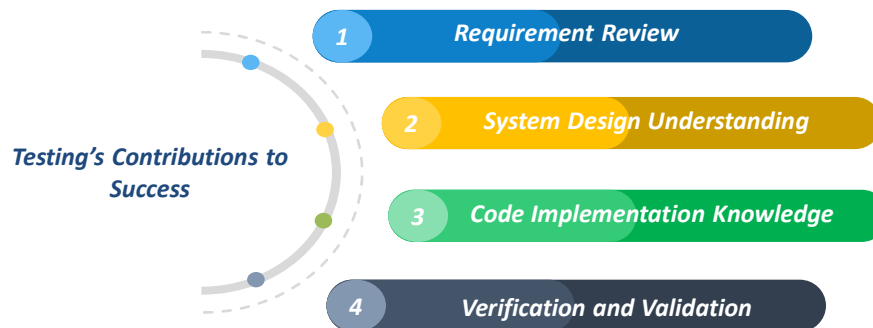
- 1. Defects Detection:** Defects Detection refers to the process of identifying issues or flaws in software during dynamic testing.
- 2. Customer Satisfaction:** Effective testing ensures that the customer finds the organisation reliable and remains satisfied with the application. If the testing organisation fails to deliver a quality product, it may result in customer dissatisfaction and potential loss of customers to competitors.
- 3. Monetary Penalties:** In some cases, contracts may include penalties based on timelines and product quality. Proper software testing can help prevent such monetary losses by delivering a high-quality product within the specified time frame.
- 4. Ensuring Quality:** Testing is crucial for ensuring the quality of the product. A high-quality product instils confidence in customers and builds their trust in the organisation.

Why is Testing Necessary?

5. **Reduced Maintenance Costs:** Delivering a quality product with fewer defects leads to lower maintenance costs. Proper testing helps detect and resolve issues early, resulting in reduced efforts and costs associated with maintenance.
6. **Performance and Reliability:** Testing ensures that the software application performs effectively and does not result in failures. This is important to avoid expensive consequences in the future or during later stages of development.
7. **Early Issue Detection:** Testing helps detect bugs and issues early in the product's life cycle. This allows for timely resolution, avoiding the need for major redesign, re-implementation, and retesting, which can be costly.
8. **Business Continuity:** A lack of software quality and stability can impact the success and adoption of the product. Users are less likely to use software that has bugs or stability issues. In the case of a product organisation or startup, poor software quality may lead to a lack of product adoption and potential business losses.

Testing's Contributions to Success

- Software Testing plays a crucial role in minimising the likelihood of problematic deliveries by employing suitable test techniques and leveraging expertise at various test levels.
- Below are a few instances illustrating how Testing can contribute to the success of a project:



© The Knowledge Academy Ltd

Using relevant test techniques can lessen the frequency of complex deliveries when those techniques are used with the suitable level of test expertise, at the appropriate points and at the appropriate test levels in the software development lifecycle.

Examples include:

- ✓ Having design defects and allow tests to be recognised at an early stage. testers involved in user story refinement or requirements reviews could detect defects in these work products. The risk of untestable or incorrect features being developed is reduced by identifying and removing requirements defects.
- ✓ The Party's understanding of the design and how to test it will be increased if the tester works closely with the system designer when the system is being designed. This increased understanding can lessen the risk of fundamental
- ✓ Each party's understanding of the code and how to test it can increase if the testers work closely with developers while the code is under development. This will reduce the risk of defects within the tests and the code.
- ✓ Having testers verify and validate the software before releasing it can detect failures that might have been missed, as well as support the process of removing the defects that result in failures (i.e., debugging). This increases the possibility that the software fulfils stakeholder requirements.

Testing and Quality Assurance (QA)

- Testing and Quality Assurance (QA) are integral parts of the software development lifecycle, yet they serve distinct functions and are oriented differently. Even though many professionals might use the terms "testing" and "quality assurance" interchangeably, they are separate domains within the broader umbrella of ensuring software quality.
- Testing falls under the category of Quality Control (QC), which is a product-centric approach. The core essence of QC is to identify and rectify defects in the final product. Among the various techniques encompassed by QC, testing is one of the most prominent. However, QC isn't limited to just testing; it encompasses other methodologies too.
- Formal methods, such as model checking and proof of correctness, and practical techniques like simulation and prototyping, also fall within the purview of QC. The primary goal of these methods is to ensure that the final software product aligns with the desired quality benchmarks.

Testing and Quality Assurance (QA)

- On the other hand, Quality Assurance (QA) is inherently process-oriented. It is rooted in the philosophy that adhering to a robust and optimised process will lead to the production of high-quality software. Instead of focusing solely on the final product, QA emphasises ensuring that every step of the software development process, from initial design to final testing, is executed flawlessly.
- This preventive approach aims to preemptively eliminate defects by optimising the processes. Moreover, QA is not the sole responsibility of a designated team; it's a collective effort, implying that every individual involved in the project, regardless of their specific role, bears the responsibility of upholding the standards set by QA.
- The symbiotic relationship between testing results, QA, and QC is also noteworthy. In the realm of QC, test results directly inform the necessary corrective actions, guiding the resolution of identified defects. For QA, these results act as a feedback mechanism. They provide invaluable insights into the efficiency and effectiveness of both the development and testing processes, highlighting areas of potential improvement.

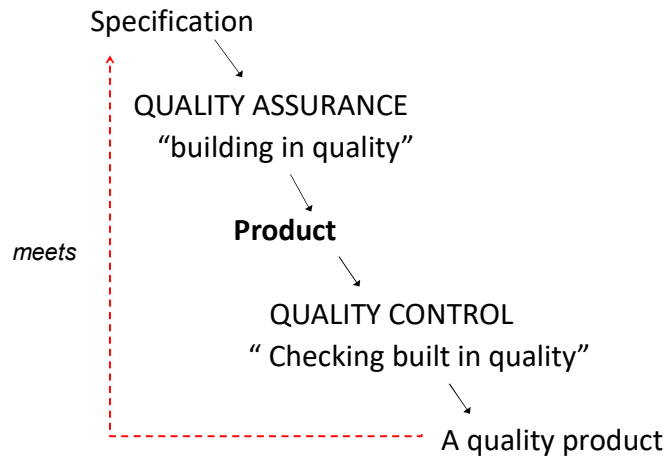
Testing and Quality Assurance (QA)

(Continued)

- Quality Assurance includes activities such as defining process standards, conducting project audits, and implementing development procedures. It can be viewed as a set of activities designed to evaluate the manufacturing process of products.
- While people often use the terms "testing" and "Quality Assurance" (QA) interchangeably, they are not synonymous. Testing is a type of Quality Control (QC), which takes a corrective approach focusing on activities that help achieve desired levels of quality. Major QC methods include testing, as well as formal methods like model checking and proof of correctness, as well as simulation and prototyping.
- The outcomes of testing are utilised in both QA and QC. In QC, these results are utilised to rectify defects, while in QA, they offer insights into the performance of development and testing processes.

Testing and Quality Assurance (QA)

Testing and Quality Management



© The Knowledge Academy Ltd

In Quality Management two key principles are used in an attempt to assure product quality. Quality Assurance is concerned with mechanisms that are put in place that increase the likelihood that the ‘manufactured’ product will be of requisite quality. Quality Control is concerned with checking that the manufactured product meets its quality specification. Testing should be integrated as one of the quality assurance activities (e.g. alongside development standards, training and defect analysis).

Quality Assurance (QA): Quality Assurance is a proactive, process-focused approach that aims to ensure the procedures, techniques, and processes involved in producing a product are suitable to prevent defects. It's about creating a system and establishing standards that inherently yield quality. In the context of software, QA might involve setting coding standards, process documentation, and method improvements. The primary goal of QA is to instill confidence that quality will be built into the products.

Building in Quality: This refers to the philosophy of integrating quality measures and standards from the very onset of a project, rather than as an afterthought or a subsequent step. By incorporating quality checks and standards at every stage of production or development, defects are minimised, and quality becomes an inherent part of the output. In software development, this might manifest as practices like Test-Driven Development (TDD) or early integration testing.

Product Quality Control: Quality Control (QC) is a reactive approach that focuses on identifying and rectifying defects in finished products. Unlike QA, which looks at processes, QC examines the actual output—be it a physical product or a piece of software—to ensure it meets the established standards. Techniques like testing, inspections, and reviews are commonly used in QC. In software, this would involve various testing methods, from unit testing to acceptance testing, to ensure the software meets specified requirements.

Quality Product: A quality product is the end result of meticulous quality assurance and stringent quality control. It meets or exceeds the expectations and requirements of its users or customers. In essence, a quality product delivers its promised value, functions reliably, and stands the test of time, with minimal defects or issues. It's a product that customers can trust and are satisfied with.

Errors, Defects, Failures, and Root Causes

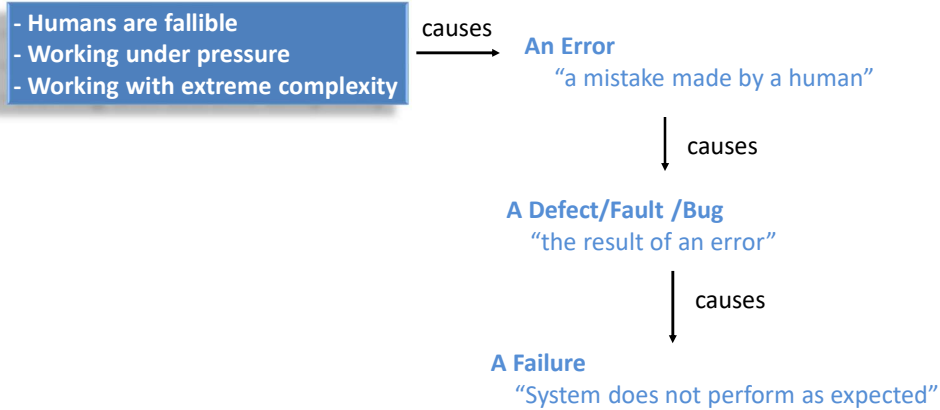
- **Errors:** Errors refer to mistakes made by individuals during the development process. These mistakes can occur due to various reasons such as lack of experience, incorrect understanding of requirements, programming errors, or oversights. Errors are unintentional and can lead to the introduction of defects in the software.
- **Defects:** Defects, also known as bugs or software issues, are flaws or malfunctions in the software that cause it to deviate from its intended behaviour or fail to meet specified requirements. Defects are typically introduced due to errors made during the development process. They can range from minor issues to critical flaws that affect the functionality, performance, or security of the software.
- **Failures:** Failures occur when the software does not perform its intended functions correctly or produces incorrect results. Failures are observable deviations from the expected behaviour of the software. They can occur due to defects present in the software or due to external factors such as incompatible hardware, incorrect usage, or environmental conditions.

Errors, Defects, Failures, and Root Causes

- **Root Cause:** The root cause refers to the underlying reason or source of a defect or failure. It is the fundamental factor that, when addressed or resolved, can prevent the recurrence of similar issues. Identifying the root cause is important in order to effectively fix defects and prevent them from reoccurring in the future. Root cause analysis involves investigating the factors that contributed to the defect or failure, understanding the underlying issues, and implementing appropriate corrective actions.

Errors, Defects, Failures, and Root Causes

Causes Of Software Defects



© The Knowledge Academy Ltd

Software defects can arise from a multitude of sources, and understanding these root causes is crucial for prevention and quality assurance. Here's a deeper look into the mentioned causes:

Humans are Fallible Examples:

- Misinterpreting a requirement and implementing a feature incorrectly.
- Overlooking edge cases when coding a particular function.
- Introducing errors while trying to fix another issue.

Working Under Pressure Examples:

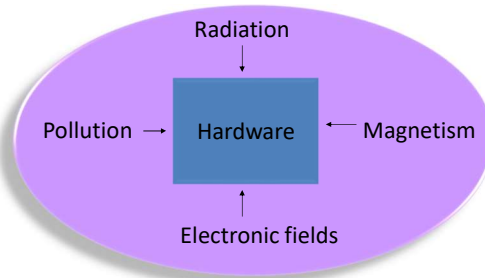
- Omitting unit tests or other testing phases to expedite delivery.
- Failing to conduct comprehensive code reviews due to time constraints.
- Implementing "quick fixes" that may not address underlying issues.

Working with Extreme Complexity Examples:

- Introducing a new feature that inadvertently disrupts an existing one due to unforeseen interactions.
- Failing to consider how various modules or components might interact under certain conditions.
- Overlooking potential security vulnerabilities when integrating third-party services or libraries.

Errors, Defects, Failures, and Root Causes

Other Causes Of Failure



Examples of "environmental conditions"

© The Knowledge Academy Ltd

In the realm of software and hardware systems, there are several external factors that can contribute to failures. The ones you mentioned primarily relate to environmental and physical factors that can impact hardware, which in turn can cause software to malfunction or behave unpredictably. Here's a detailed look into these causes:

Radiation: Cosmic rays or other sources of radiation can strike the semiconductor material in a computer chip, altering the state of its transistors. This phenomenon is particularly concerning for devices used in high-altitude applications, like aviation and space. A single event upset (SEU) can change the state of a memory bit, leading to potential data corruption.

Pollution: Dust, corrosive gases, and other pollutants can interfere with electronic components, especially if these components are exposed or if the protective mechanisms fail. Over time, contaminants can lead to physical degradation of hardware, resulting in reduced performance or outright failure.

Magnetism: Magnetic fields can interfere with electronic devices, especially those that rely on magnetic storage (like traditional hard drives). A strong magnetic field can corrupt data or even damage the storage medium.

Electronic Fields (Electromagnetic Interference, EMI): All electronic devices emit some level of electromagnetic radiation. When the emitted radiation from one device interferes with the operation of another, it's called electromagnetic interference. This interference can cause devices to malfunction, reduce their performance, or corrupt data.

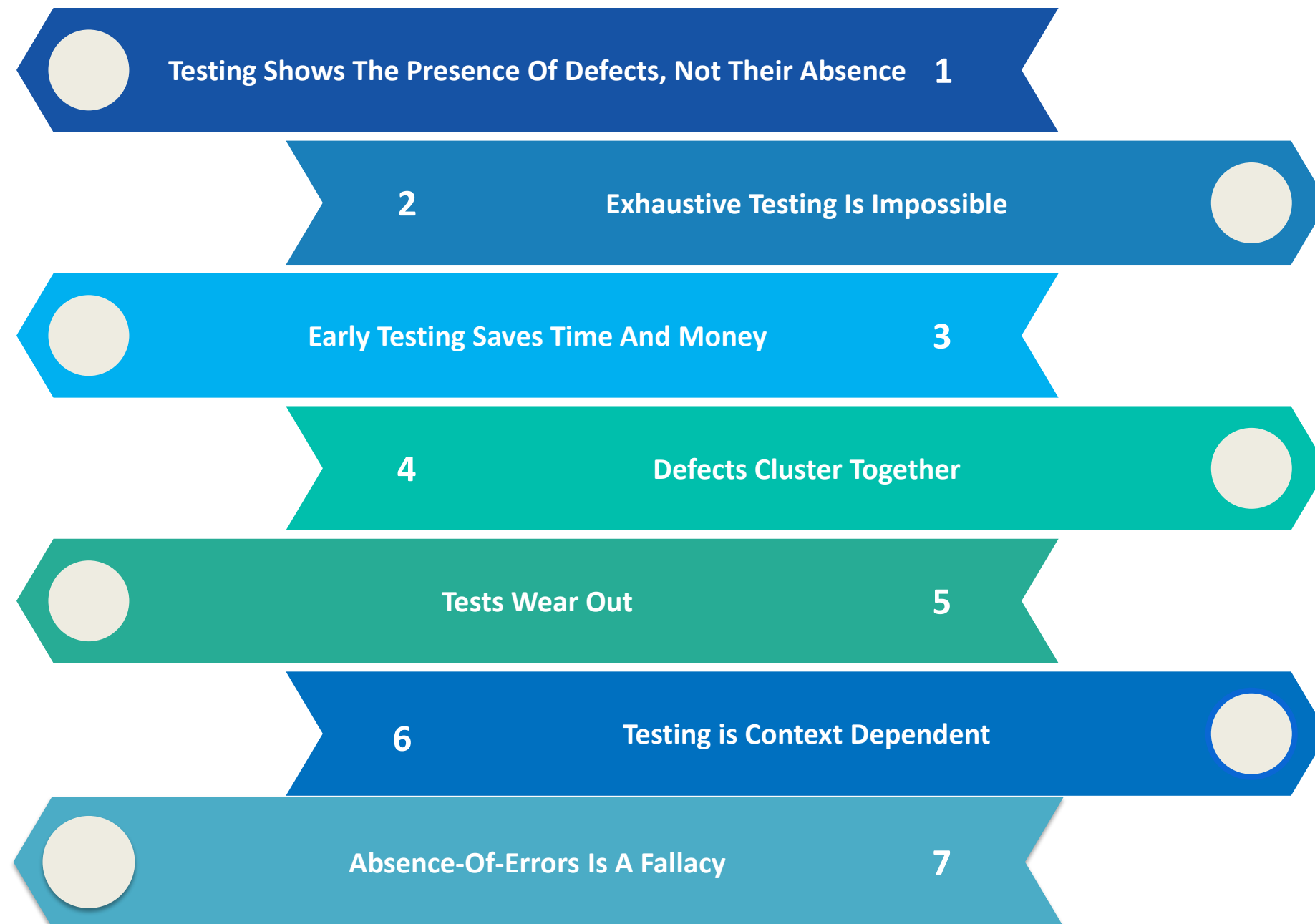
Module 3: Testing Principles





Testing Principles

The following are Testing Principles:



Testing Principles

1. Testing Shows the Presence, not the Absence of Defects

- Testing can uncover existing defects, but it can't guarantee that the software is defect-free.
- The absence of detected defects doesn't imply the software is defect-free; it may still have undetected issues.
- Testing provides information on the quality of the software and helps improve it, but it cannot certify perfection or complete absence of defects.

2. Exhaustive Testing is Impossible

- It is an undeniable fact that conducting exhaustive testing would require an impractical and unlimited amount of effort.
- Such an approach would be highly ineffective and exceed the project timeline. Instead, testing techniques are carefully chosen and prioritised to focus on specific combinations.

Testing shows the presence, not the absence of defects:

Example: Imagine a popular word processing software. After a new feature is added for spell check, tests show that the feature can identify certain spelling errors. However, it might miss other complex errors or make incorrect suggestions. Just because the test didn't identify these issues doesn't mean they don't exist.

Exhaustive testing is impossible:

Example: Consider a flight reservation system. Testing every possible combination of flight dates, routes, seating classes, payment methods, and passenger details would take lifetimes. Instead, testing teams prioritise by using risk-based testing, focusing on high-traffic routes or commonly used payment methods.

Testing Principles

(Continued)

- This ensures that testing efforts are optimised based on a risk assessment of the areas being tested.
- One of the crucial skills possessed by testing experts is the ability to identify and prioritise the most critical functions to be tested.

3. Early Testing Saves Time and Money

- Initiating the testing process at the earliest stage is crucial to promptly identify any defects or errors.
- By starting early, it becomes possible to minimise the number of issues discovered during later stages of testing, resulting in cost savings.
- This principle of testing will help in reducing the cost of quality and result in detecting fewer failures in SDLC.

Early testing saves time and money:

Example: A company designing a new e-commerce platform identifies a flaw in the product's architecture during the design phase. Addressing it at this stage is much cheaper and quicker than if it had been discovered after the platform was built and operational.

Testing Principles

4. Defects Cluster Together

- In the testing phase, it is common to find that a significant portion of the defects are concentrated within a small number of modules.
- These modules may be complex or contain legacy code, leading to a higher likelihood of issues. This phenomenon aligns with the Pareto Principle, where approximately 80% of the defects are discovered within 20% of the tested modules.
- Being aware of these specific modules can greatly benefit testers, as they can prioritise their efforts and focus on these critical areas. By concentrating on these high-risk modules, testers can effectively allocate their resources and improve the overall testing process.

5. Tests Wear Out

- When tests are repeatedly performed, their ability to detect new defects diminishes over time. To mitigate this impact, it may be necessary to modify existing tests and test data, as well as develop new tests.

© The Knowledge Academy Ltd

Defects cluster together:

Example: In a music streaming app, users report multiple issues around the playlist feature—like inability to add songs, delete them, or change their order. While other features work fine, the playlist section may have most of the app's defects, adhering to the Pareto principle.

Tests wear out:

Example: An online banking portal has a test for verifying login functionality. After a few successful rounds of testing, the same test stops detecting a new defect introduced during a later development phase because it wasn't designed to catch that particular issue. This shows the diminishing returns of repeatedly using the same test.

Testing Principles

(Continue)

- However, it is worth noting that repeating the same tests can yield beneficial results in certain scenarios, such as in automated regression testing.

6. Testing is Context Dependent

- Testing is highly contextual and relies on the specific characteristics and requirements of the test subject. For instance, the approach taken for testing an application intended for the cruise industry will differ significantly from that for an application in the insurance industry.

7. Absence-of-Defects Fallacy

- Even if software has undergone extensive testing and has been determined to be 99% free of bugs, it can still be rendered unusable if it does not align with the correct requirements.
- As highlighted in the initial principle, testing goes beyond identifying errors or defects and also involves evaluating if the software meets the business needs. Rectifying these errors becomes essentially futile if the system fails to fulfill the requirements of the end user.

© The Knowledge Academy Ltd

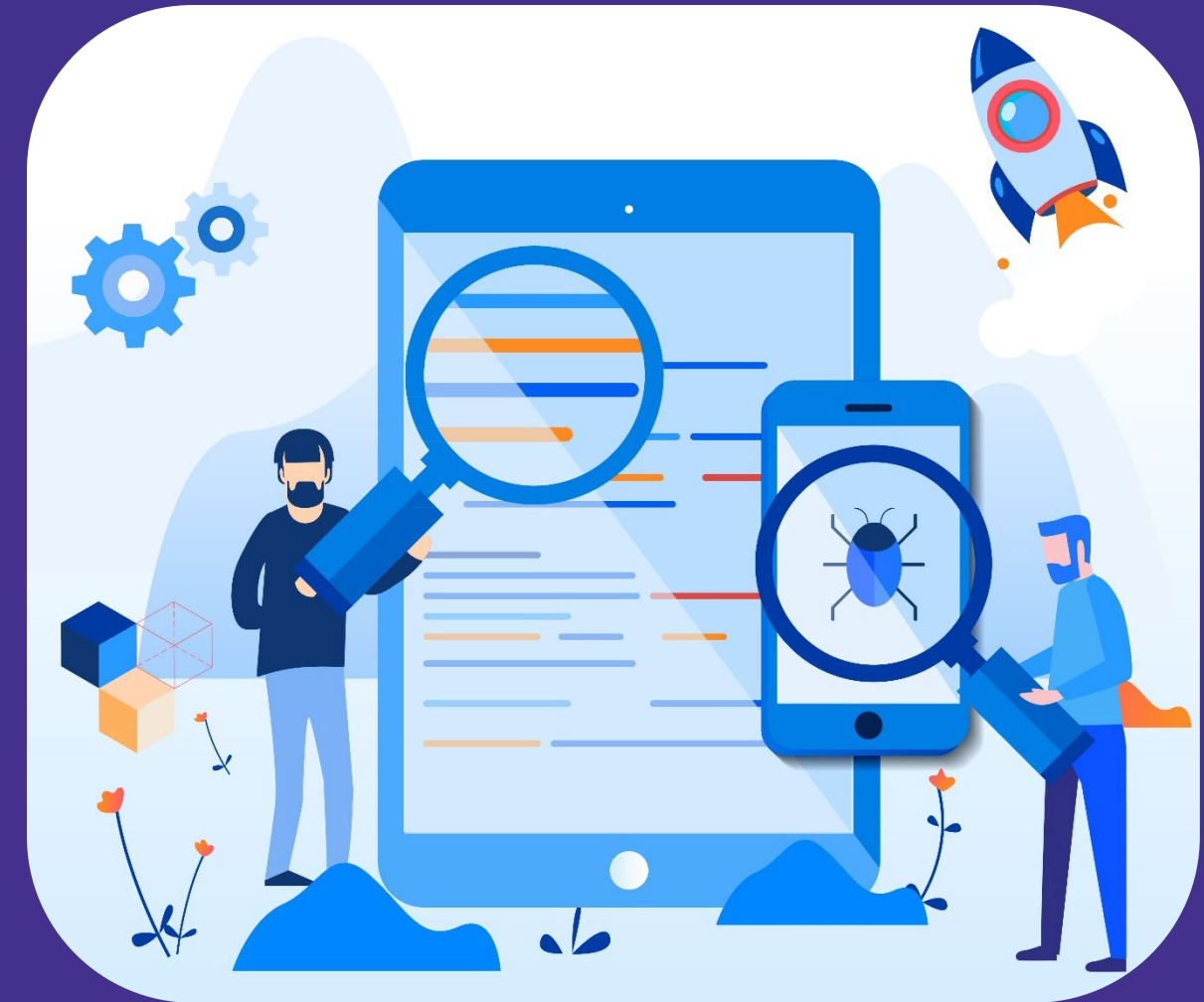
Testing is context dependent:

Example: Testing a real-time stock trading application, where split-second data accuracy is paramount, will differ from testing a blog website. While both need rigorous testing, the emphasis, tools, and techniques used for each will differ.

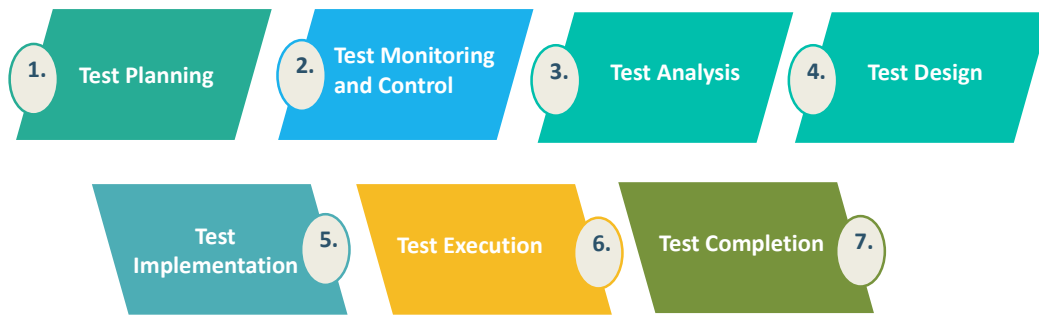
Absence-of-defects fallacy:

Example: A new restaurant management software has been tested thoroughly against requirements and all defects were addressed. However, upon release, restaurant owners find that it doesn't integrate well with popular payment systems. Even though the software met its specified requirements, it fails to meet user needs in the real-world context.

Module 4: Test Activities, Testware and Test Roles



Test Activities and Tasks



- Test activities and tasks define the structured approach taken during the software testing process.
- These activities encompass everything from planning to completion. Initially, a test strategy is defined, outlining the broader objectives and methods.

Test Activities and Tasks

1. Test Planning:

Test planning involves creating a blueprint for the entire testing process. It provides a clear direction, scope, and framework for the tests to be carried out, ensuring all stakeholders are aligned with the objectives of testing. Test planning identifies resources, timelines, and responsibilities, setting the stage for effective communication and coordination among the test team. This stage also involves the establishment of test criteria, including the definition of success for the test phases.

Test Planning Main Tasks:

- Determine the scope and objectives of testing
- Identify deliverables
- Allocate resources
- Schedule test activities
- Define exit criteria

Test Planning Main Tasks:

- **Determine the scope and objectives of testing:** This involves defining what will be tested and what the test aims to achieve, such as ensuring the software meets its requirements and is defect-free.
- **Identify deliverables:** This refers to specifying the documents, tools, and reports that will be produced during the testing process.
- **Allocate resources:** This task involves assigning personnel, budget, and tools necessary for the testing process.
- **Schedule test activities:** This entails planning when and in what order the testing activities will occur.
- **Define exit criteria:** This involves setting the conditions under which testing can be concluded, such as a certain level of test coverage or number of resolved defects.

Test Activities and Tasks

2. Test Monitoring and Control:

This phase ensures that the testing process stays on track. It involves continuous tracking, oversight, and, if necessary, adjustment of the testing activities. As the project progresses, test monitoring and control include the comparison of actual progress against the plan, allowing for proactive identification of variances. This phase also involves risk management, where potential issues are identified and mitigated to keep the testing phase within the agreed-upon schedule and budget.

Test Monitoring and Control Main Tasks:

- Track and assess test progress
- Compare actual outcomes with planned outcomes
- Adjust test schedule based on real-time data
- Review corrective actions

Test Monitoring and Control Main Tasks:

- **Track and assess test progress:** This task involves monitoring the status of testing activities and ensuring they are on track to meet the objectives.
- **Compare actual outcomes with planned outcomes:** This involves evaluating if the testing is proceeding as planned and meeting the expected results.
- **Adjust test schedule based on real-time data:** This task requires modifying the testing schedule and strategy in response to actual progress and any issues encountered.
- **Review corrective actions:** This involves assessing the effectiveness of corrective actions taken to address any deviations from the plan.

Test Activities and Tasks

3. Test Analysis:

At this stage, the test basis is examined in detail to derive testable features, determining "what" needs to be tested. During test analysis, the team also assesses the testability of the requirements and system architecture, often resulting in a refined set of requirements.

Test Analysis Main Tasks:

- Review documentation to identify test requirements
- Break down requirements into test conditions
- Evaluate potential risks
- Prioritise test conditions



© The Knowledge Academy Ltd

Test Analysis Main Tasks:

- **Review documentation to identify test requirements:** This involves going through all relevant documentation to understand what needs to be tested, including specifications, design documents, and user stories.
- **Break down requirements into test conditions:** This task is about translating the test requirements into specific conditions that can be tested.
- **Evaluate potential risks:** It's important to assess what risks are associated with the software or its features to prioritise testing efforts.
- **Prioritise test conditions:** Based on the importance and risk, test conditions are ranked to determine the order of test execution.

Test Activities and Tasks

4. Test Design:

Test design is about detailing "how" to test. It translates analysis into actionable test cases and determines the expected outcomes for each scenario. It often incorporates peer reviews to validate the coverage and effectiveness of test cases before they are finalised.

Test Design Main Tasks:

- Convert test conditions into detailed test cases
- Identify input data and expected outcomes
- Determine test environment setup
- Organise test cases into logical sequences

Test Design Main Tasks:

- **Convert test conditions into detailed test cases:** This involves creating step-by-step instructions that will be executed during testing.
- **Identify input data and expected outcomes:** For each test case, define the necessary input data and the expected results after running the test.
- **Determine test environment setup:** Decide on the hardware, software, network configurations, and other environment setups needed to execute the test cases.
- **Organise test cases into logical sequences:** Arrange the test cases in an order that reflects the user's journey or logical feature sets for efficient testing.

Test Activities and Tasks

5. Test Implementation:

This phase involves setting up the necessary resources to execute tests, including the test environment, test data, and any automation scripts. It ensures that all test cases are reviewed, approved, and integrated into the test management system, ready for execution.

Test Implementation Main Tasks:

- Create or acquire the required test data
- Document detailed test procedures
- Develop automation scripts if needed
- Validate the test environment

Test Implementation Main Tasks:

- **Create or acquire the required test data:** This involves either generating new data that simulates real-world use cases or acquiring existing datasets that can be used for testing. The goal is to have a comprehensive set of data that can cover various test scenarios, including edge cases.
- **Document detailed test procedures:** Detailed test procedures are essential for consistency and repeatability. This documentation should include the setup required, execution steps, and the criteria for passing or failing each test case.
- **Develop automation scripts if needed:** For tests that are run repeatedly or that require complex setups, automation scripts can save time and reduce human error. These scripts should be developed according to best practices and thoroughly tested to ensure they work as expected.
- **Validate the test environment:** Before execution, the test environment, which includes hardware, software, network configurations, and any other necessary tools, must be verified to ensure it is a faithful replication of the production environment or the specified requirements for testing.

Test Activities and Tasks

6. Test Execution:

As the name suggests, this phase is where actual testing occurs. Test cases are executed, results are documented, and discrepancies are logged. Testers not only run the tests but also retest as necessary, perform regression testing to check that new changes haven't adversely affected existing functionalities, and conduct sanity checks to ensure system stability.

Test Execution Main Tasks:

- Execute tests per the defined procedures
- Capture and log test results
- Document discrepancies or anomalies
- Analyse test failures to determine causes

Test Execution Main Tasks:

- **Execute tests per the defined procedures:** This is the process of carrying out the tests as documented. It requires attention to detail to ensure that each step is performed exactly as intended.
- **Capture and log test results:** As tests are executed, results must be recorded. This includes not only whether the test passed or failed but also any relevant outputs or behaviors observed during the test.
- **Document discrepancies or anomalies:** Any deviations from expected outcomes should be documented as discrepancies or anomalies. This is critical for later analysis to understand the nature of any defects or issues.
- **Analyse test failures to determine causes:** When a test fails, determining the root cause is essential. This could involve debugging the code, reviewing logs, and checking test configurations to identify what went wrong.

Test Activities and Tasks

7. Test Completion:

Upon reaching the end of the testing cycle, the test completion phase involves wrapping up, documenting outcomes, and ensuring learnings are captured for future projects. It also involves a final assessment of the test coverage and defect analysis to ensure all critical issues have been addressed.

Test Completion Main Tasks:

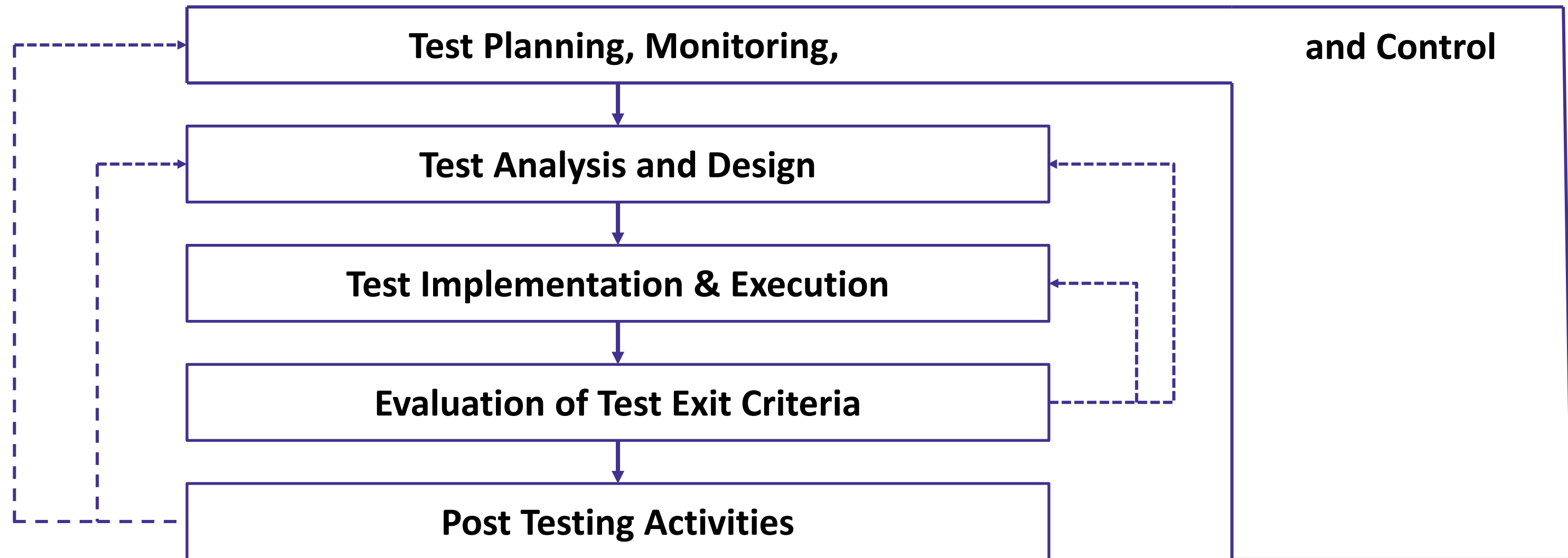
- Evaluate the exit criteria
- Document remaining known defects
- Archive test artifacts
- Disassemble the test environment
- Conduct a retrospective to identify lessons learned

Test Completion Main Tasks:

- **Evaluate the exit criteria:** The exit criteria, defined during the test planning phase, should be reviewed to determine if all conditions for completion have been met. This might include a certain level of test coverage, number of critical defects closed, or other project-specific criteria.
- **Document remaining known defects:** Not all defects may be fixed by the end of the testing cycle. Those that remain should be documented, including their severity, impact, and any planned future action.
- **Archive test artifacts:** This includes storing all relevant test cases, data, scripts, logs, and results. Archiving these artifacts is crucial for accountability, future reference, and compliance with audit requirements.
- **Disassemble the test environment:** Resources should be cleaned up, which includes decommissioning servers, freeing up licenses, and restoring systems to their pre-testing state.
- **Conduct a retrospective to identify lessons learned:** A retrospective meeting with the test team and possibly other stakeholders should be held to discuss what was learned during the testing process. This is an opportunity to identify best practices, process improvements, and other insights that can benefit future projects.

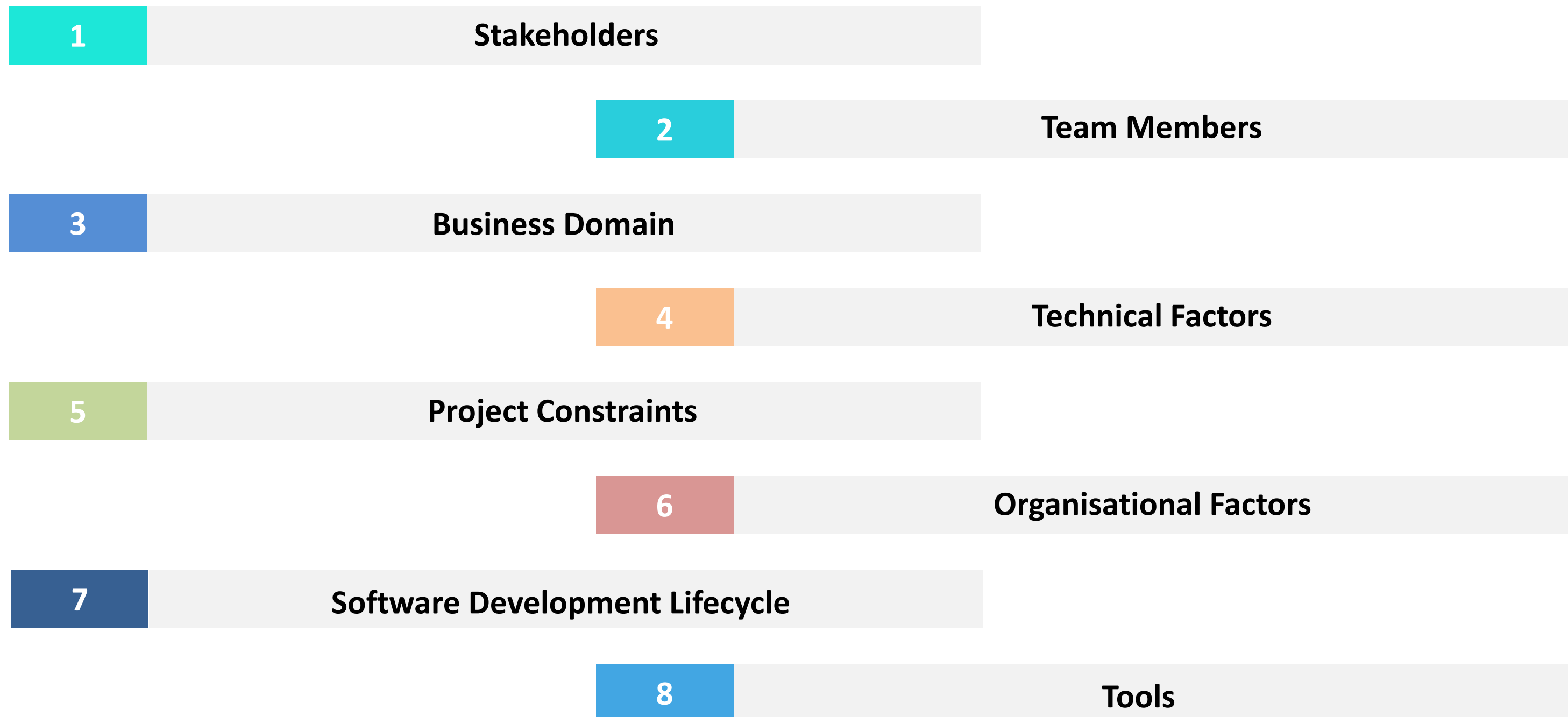
Test Activities and Tasks

Possible Iterations in Test Process



Test Process in Context

- The test process for an organisation can be influenced by the following factors:*



Test Process in Context

- **Stakeholders:** Understanding stakeholders' needs, expectations, and requirements is paramount. Their willingness to cooperate and engage in the testing process plays a crucial role in achieving comprehensive test coverage.
- **Team Members:** The skills, knowledge, experience, and availability of team members impact testing outcomes. Adequate training and support are essential to ensure a proficient testing team.
- **Business Domain:** The criticality of the test object, identified risks, market demands, and specific legal regulations within the business domain shape testing priorities. Addressing domain-specific challenges enhances the testing process.
- **Technical Factors:** The nature of the software, product architecture, and technologies employed dictate the testing methods and tools to be utilised.

© The Knowledge Academy Ltd

Stakeholders: Beyond just understanding their needs, it's essential to keep stakeholders informed throughout the testing process. Their feedback can highlight blind spots, and their continual involvement often results in a smoother acceptance phase, as they are kept in the loop about potential defects or trade-offs.

Team Members: Collaboration among team members is as vital as their individual skills. Promoting a culture of knowledge sharing can uncover hidden issues and foster innovative testing strategies. Moreover, recognising and addressing any skill gaps in real-time will enhance the overall effectiveness of the testing team.

Business Domain: Familiarity with the business domain allows testers to simulate real-world scenarios more effectively. Recognising the nuances and subtleties of the industry can lead to more accurate, context-driven testing, ultimately producing a product better aligned with its intended audience.

Technical Factors: While the inherent software elements drive the test strategy, it's also pivotal to consider any future technological evolutions or potential integrations. This foresight ensures that the software remains robust and adaptable in the face of change.

Test Process in Context

- **Project Constraints:** The project's scope, timeline, budget, and resource availability influence testing strategies and priorities. Efficient resource allocation is essential for successful testing outcomes.
- **Organisational Factors:** The existing organisational structure, policies, and practices shape the testing framework. Adhering to established practices ensures consistency and quality.
- **Software Development Lifecycle:** The chosen development methods and engineering practices impact the testing process. Aligning testing activities with the development lifecycle enhances synergy and cooperation.
- **Tools:** The availability, usability, and compliance of testing tools influence the degree of automation and effectiveness of testing efforts.

Project Constraints: It's not enough to merely recognise constraints; testers must be agile, adapting their strategies to work within these limitations. For instance, in a time-crunch, risk-based testing can prioritise critical functions over less essential features.

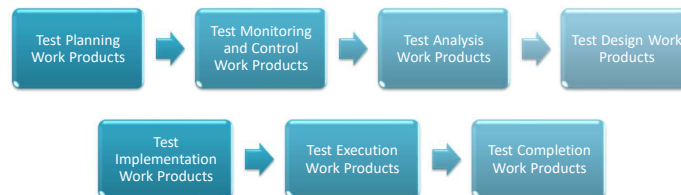
Organisational Factors: Every organisation has its cultural nuances and unwritten rules. These elements can impact the testing process. For instance, a company with a strong feedback culture might have more review stages, while a hierarchy-driven entity might limit decision-making to senior members. Understanding and navigating these nuances is crucial for a streamlined testing process.

Software Development Lifecycle (SDLC): The test process must also be adaptable to various SDLC models, whether agile, waterfall, or hybrid. For instance, in an agile setup, testing might be more iterative and integrated, whereas in waterfall, it could be a distinct phase.

Tools: Besides the immediate functionality of testing tools, testers should consider their scalability, support community, integration capabilities, and long-term viability. Choosing the right tools can not only improve the current testing process but also ensure sustainability and ease in future projects.

Testware

- Testware refers to the artifacts and documentation produced during the testing process, including test plans, test cases, test scripts, test data, test reports, and any supporting tools.
- It facilitates test execution, analysis, and collaboration among testing teams and stakeholders, ensuring comprehensive testing and efficient software evaluation.
- Test work products are built as part of the test process. There is significant variation in the kinds of work products created during the process, in the manner those work products are organised and handled, and in the names utilised for those work products.
- ***The following are the different types of test work products:***



© The Knowledge Academy Ltd

Test planning work products: This usually involves one or more test plans. The test plan incorporates information regarding the test basis, to which the other test work products will be related through traceability information and exit criteria which will be utilised during test monitoring and control.

Test monitoring and control work products: Test monitoring and control work products usually involve several kinds of test reports, involving test progress reports produced on an ongoing or regular basis and test summary reports produced at several completion milestones.

Test analysis work products: Test analysis work products involve defined and prioritised test conditions that are ideally bi-directionally traceable to the specific element(s) of the test basis it covers.

Test design work products: Test design results in test cases as well as sets of test cases to exercise the test conditions described in test analysis. It is usually a good practice to design high-level test cases without concrete values for input data as well as expected results.

Test implementation work products: Test implementation work products involve test procedures as well as the sequencing of those test procedures, test suites and a test execution schedule.

Test execution work products: Test execution work products incorporate documentation of individual test cases or test procedures status, defect and documentation regarding which test object(s), test item(s), test tools, and testware were involved in the testing.

Test completion work products: Test completion work products involve test summary reports, action items for enhancement of subsequent projects or iterations, change requests or product backlog items, and finalised testware.

Traceability Between the Test Basis and Test Work Products

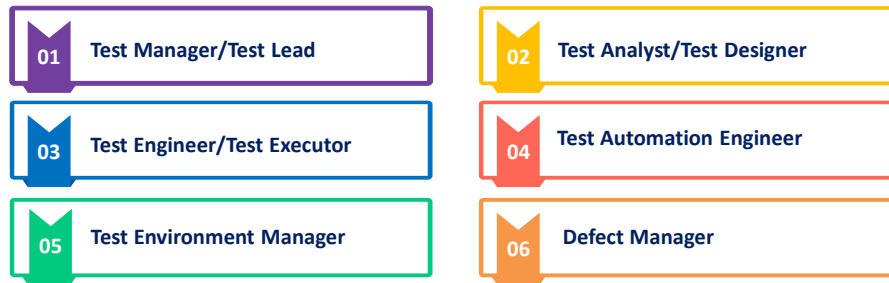
- For implementing effective test monitoring and control, it is essential to establish and maintain traceability during the test process between every element of the test basis and the several test work products related to that element.
- ***Along with the evaluation of test coverage, good traceability supports:***
 - ✓ Analysing change impact
 - ✓ Making testing auditable
 - ✓ Meeting IT governance criteria
 - ✓ Enhancing the understandability of test progress reports as well as test summary reports to incorporate the status of elements of the test basis
 - ✓ Relating the testing's technical aspects to stakeholders in a way that they can understand
 - ✓ Providing information to evaluate product quality, project progress and process capability against business goals

Test Basis: Test basis refers to the documentation and information on which test cases are designed and test procedures are determined. Essentially, it's the source material from which test scenarios and cases are derived. Common **examples** of test basis include: requirements documents, design specifications, product code, user stories, and contracts or agreements.

Test Work Products: Test work products refer to the artifacts or documents produced before, during, or after the testing process. These products provide a structured and organised record of the testing activities and can be utilised for future reference, audit, or iteration. **Examples** include: Test plans, test cases and test scripts, test data, defect logs, test results/reports, and traceability matrices

Testing Roles

Roles in Testing



© The Knowledge Academy Ltd

Test Manager/Test Lead: The test manager or test lead is responsible for overall test planning, coordination, and management of the testing activities. They oversee the test team, allocate resources, define test strategies, and monitor progress.

Test Analyst/Test Designer: Test analysts or test designers are responsible for analysing the requirements, identifying test conditions, and designing test cases and scenarios. They collaborate with stakeholders to ensure test coverage and effectiveness.

Test Engineer/Test Executor: Test engineers or test executors are responsible for executing test cases, recording test results, and reporting defects. They follow the test design and execute tests based on the defined test procedures.

Test Automation Engineer: Test automation engineers focus on automating test cases and developing test scripts using appropriate test automation tools. They collaborate with the test team to identify suitable scenarios for automation and maintain the test automation framework.

Test Environment Manager: The test environment manager is responsible for setting up and maintaining the test environments required for testing. They ensure that the necessary hardware, software, and configurations are in place for executing tests effectively.

Defect Manager: The defect manager is responsible for managing the defect lifecycle. They track, prioritise, assign, and monitor the resolution of defects reported during testing. They also analyse defect patterns and provide insights for process improvements.

Module 5: Essential Skills and Good Practices in Testing



Generic Skills Required for Testing

- Generic skills required for testing include a combination of technical and soft skills. Technical skills encompass knowledge of testing methodologies, test design, test automation, defect tracking tools, and programming languages. Analytical and problem-solving abilities enable testers to identify, isolate, and rectify issues effectively.
- Attention to detail ensures thorough test coverage, while communication skills facilitate effective collaboration with team members and stakeholders.
- Time management skills help in meeting testing deadlines and priorities. Testers should possess adaptability to handle diverse projects and continuous learning to keep up with evolving technologies. A tester should possess good documentation skills to write clear and concise test cases and reports, enabling seamless collaboration with the development team.
- Effective communication skills facilitate clear articulation of findings and seamless collaboration with team members and stakeholders. Domain knowledge is a crucial generic skill in testing, as understanding the business domain and its processes helps testers create relevant test scenarios and identify potential risks and requirements.

© The Knowledge Academy Ltd

A Complex E-commerce System Testing:

Scenario: A tester named Alex is assigned to test an intricate e-commerce platform that integrates with multiple third-party payment gateways and inventory systems.

- **Testing knowledge:** Using his deep testing knowledge, Alex applied boundary value analysis and equivalence partitioning to efficiently test the vast range of possible input values in payment details.
- **Thoroughness and Attention to Details:** While testing, Alex's thorough nature led him to spot a discrepancy in the currency conversion rates between the platform and one of the third-party systems—something that could have led to significant revenue loss.
- **Communication skills:** Alex then had to inform the developer team about this critical issue. Instead of outrightly declaring it as a 'major defect', he explained the context, demonstrated the discrepancy, and actively listened to the developers' perspective. This approach ensured that the issue was accepted and addressed without causing any defensive reactions.
- **Analytical thinking:** To ensure he had covered all ground, Alex analysed the potential ripple effect of this discrepancy across other integrated systems.
- **Technical Knowledge:** Using his technical expertise, Alex also developed an automated script to simulate high-volume transactions and ensure system performance.
- **Domain Knowledge:** Since Alex had experience in e-commerce domain testing, he could relate to real-world scenarios and user expectations better, making his testing approach more user-centric.

Whole Team Approach

- The Whole Team Approach is a collaborative and inclusive approach to software development and testing that emphasises the involvement of all team members throughout the project lifecycle.
- It promotes the idea that testing is not just the responsibility of dedicated testers but involves the entire team, including developers, business analysts, product owners, designers, and stakeholders.



© The Knowledge Academy Ltd

Here are key aspects of the Whole Team Approach:

- **Shared Responsibility:** In the Whole Team Approach, testing and quality assurance are seen as everyone's responsibility, not just the testers. All team members actively contribute to the testing effort by participating in test planning, test design, and test execution activities.
- **Early and Continuous Testing:** Testing is integrated into the development process from the beginning. The team focuses on early and continuous testing to identify defects and validate the software against requirements as early as possible. This helps in catching issues early, reducing rework, and ensuring quality throughout the development cycle.
- **Collaborative Planning:** The team collaboratively plans testing activities, including identifying test objectives, defining test strategies, and prioritising testing tasks. Testers work closely with developers, business analysts, and other team members to align testing efforts with the project goals and ensure that test coverage is comprehensive.
- **Test Automation:** Test automation is embraced as a key aspect of the Whole Team Approach. Developers and testers collaborate to automate tests at different levels, such as unit testing, integration testing, and acceptance testing. Automation helps in faster and more frequent testing, improves efficiency, and enables regression testing.
- **Continuous Integration and Continuous Delivery:** The Whole Team Approach promotes the use of continuous integration and continuous delivery practices. Regularly integrating code changes and delivering working software in short iterations allows for ongoing testing and quick feedback, enabling faster identification and resolution of issues.
- **Cross-Functional Collaboration:** The Whole Team Approach encourages collaboration and communication among team members from different disciplines. Developers, testers, and other stakeholders work together closely, share knowledge, and provide feedback to ensure that the software meets both functional and non-functional requirements.
- **Knowledge Sharing and Learning:** The team emphasises knowledge sharing and learning to enhance the overall testing capabilities. Testers share their expertise with developers on testing techniques, while developers contribute insights on technical aspects that can impact testing. This cross-pollination of knowledge helps in improving the quality of testing.

Advantages of the Whole Team Approach

- Enhanced Team Dynamics
- Improved Communication and Collaboration
- Synergy through Diverse Skill Sets
- Ensures High-Quality Output
- Efficient Use of Resources
- Knowledge Transfer and Skill Upgradation
- Better Alignment with Business Goals
- Adaptable Test Strategies
- Shared Ownership
- Maintains Independence When Needed



© The Knowledge Academy Ltd

- **Enhanced Team Dynamics:** When everyone in the team feels responsible for quality and contributes to various tasks, it fosters a sense of unity and shared purpose. This collective responsibility can lead to improved morale and cohesion within the team, creating an environment where each member feels valued and included.
- **Improved Communication and Collaboration:** Co-location, whether physical or virtual, promotes more direct and frequent communication among team members. With boundaries blurred between roles, there's a continuous flow of information, leading to quicker issue resolution and clearer understanding of shared goals.
- **Synergy through Diverse Skill Sets:** By allowing any team member with the necessary skills to perform any task, the team can leverage a variety of expertise. This pooling of diverse skills can lead to innovative solutions and a broader perspective on problem-solving.
- **Ensures High-Quality Output:** With everyone being responsible for quality, there's a collective effort to ensure the output meets the desired standards. Testers, developers, and other stakeholders continuously collaborate to achieve the best results, ensuring a quality-centric approach.
- **Efficient Use of Resources:** Flexibility in task allocation means that work can be distributed based on current team availability and skill sets, leading to optimised resource utilisation and potentially shorter delivery times.
- **Knowledge Transfer and Skill Upgradation:** When testers transfer testing knowledge to other team members, it not only enhances the skills of the entire team but also promotes a culture of continuous learning.
- **Better Alignment with Business Goals:** The close collaboration with business representatives means that the development and testing are in line with business needs.
- **Adaptable Test Strategies:** Testers and developers working together can quickly adapt their testing strategies based on immediate feedback and changing requirements. This flexibility can lead to more relevant and efficient testing.
- **Shared Ownership:** With everyone responsible for the final product's quality, there's a shared sense of ownership. This collective ownership can lead to higher accountability and motivation to deliver the best possible product.
- **Maintains Independence When Needed:** While the whole team approach encourages collaboration, it also recognises the need for independence, especially in safety-critical situations.

Independence of Testing

- Independence of testing refers to the level of separation and autonomy that testers have from the development team and other project stakeholders.
- It is a key principle in software testing to ensure objectivity, impartiality, and thoroughness in the testing process. Here are some aspects of independence in testing:



© The Knowledge Academy Ltd

- **Independence from Development:** Testers should be independent from the development team to minimise bias and conflicts of interest. This allows them to provide an objective assessment of the software's quality and uncover defects without undue influence from the development process.
- **Separation of Roles:** Testers should have a distinct role and responsibility separate from developers. This separation helps maintain a fresh perspective and avoids potential conflicts that may arise when developers test their own work.
- **Test Planning and Strategy:** Testers should have the authority and autonomy to define test plans, strategies, and approaches based on their expertise and understanding of the system. They should be able to make decisions about test coverage, test techniques, and test priorities without undue influence or pressure.
- **Test Execution and Reporting:** Testers should have the freedom to execute tests and report their findings objectively, without any interference or manipulation. They should be able to report defects and issues based on their observations and analysis, without bias or external influence.
- **Test Environment and Tools:** Testers should have control over the test environment, test data, and testing tools. They should be able to set up and configure the test environment independently to ensure accurate and reliable testing.
- **Test Documentation and Reporting:** Testers should have the authority to document and report test results, including defects, test coverage, and test metrics, in an unbiased and transparent manner. They should be able to communicate the testing status and provide recommendations for improvement without undue pressure or censorship.
- **Test Process Improvement:** Testers should have the freedom to identify process gaps, propose improvements, and implement changes in the testing process. They should be able to advocate for best practices, introduce new techniques, and continuously enhance the effectiveness and efficiency of testing.

Levels Of Independence

Benefits of Independence of Testing:

- Objectivity
- Unbiased Assessment
- Fresh Perspective
- Improved Quality
- Risk Mitigation
- Validation of Requirements
- Challenging Assumptions

Drawbacks of Independence of Testing:

- Communication Overhead
- Reduced Collaboration
- Limited Knowledge Sharing
- Time and Resource Constraints
- Potential Misalignment
- Increased Cost
- Bottleneck Perception

© The Knowledge Academy Ltd

Benefits of Independence of Testing:

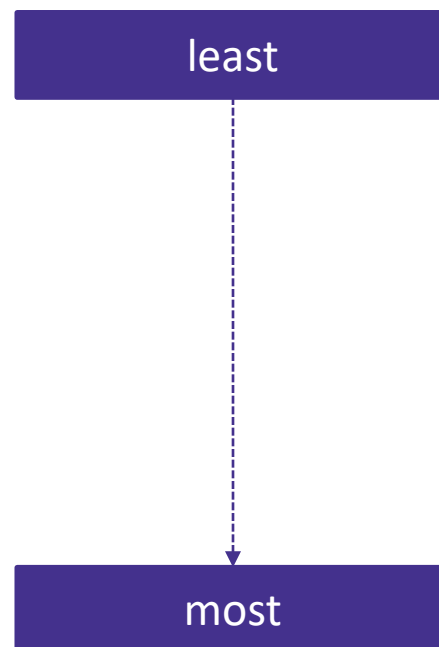
- **Objectivity:** Independent testers are unbiased and can evaluate the software impartially. They are more likely to identify defects and potential issues without being influenced by the development team's perspective.
- **Unbiased Assessment:** Testers who are not emotionally attached to the project are more likely to provide an unbiased assessment of the software's quality, leading to a more accurate representation of its actual state.
- **Fresh Perspective:** Independent testers bring fresh ideas and viewpoints to the testing process. Their different experiences and backgrounds can lead to unique testing approaches and uncover hidden defects.
- **Improved Quality:** By having a dedicated focus on quality assurance, independent testers can conduct rigorous testing, thereby improving the overall quality of the software.
- **Risk Mitigation:** Independent testing can help identify high-risk areas and potential pitfalls early in the development cycle, reducing the likelihood of severe issues later in the project.
- **Validation of Requirements:** Independent testers can validate whether the software meets the specified requirements and if it aligns with the stakeholders' expectations.
- **Challenging Assumptions:** They play a crucial role in challenging and validating the assumptions made by the development team and stakeholders, ensuring that the system meets the true needs of its users.

Drawbacks of Independence of Testing:

- **Communication Overhead:** Separation between testers and developers can lead to communication challenges, as feedback and clarification may take longer to reach the development team.
- **Reduced Collaboration:** Isolation of testers from the development team might result in decreased collaboration, which is essential for quick resolution of issues and understanding the system context.
- **Limited Knowledge Sharing:** Independent testers may lack the in-depth knowledge of the application's internals possessed by the development team, potentially missing some context during testing.
- **Time and Resource Constraints:** Maintaining a separate testing team requires additional resources and can increase the time needed for testing activities.
- **Potential Misalignment:** Independent testers may not fully understand the business objectives, leading to test scenarios that do not reflect real-world usage.
- **Increased Cost:** Having an independent testing team can lead to higher project costs due to the need for additional personnel and resources.
- **Bottleneck Perception:** Independent testing teams can be mistakenly viewed as bottlenecks, especially if there are delays in testing due to dependency on external factors, leading to potential conflicts and blame for release delays.

Levels Of Independence

Tests **DESIGNED** by :-



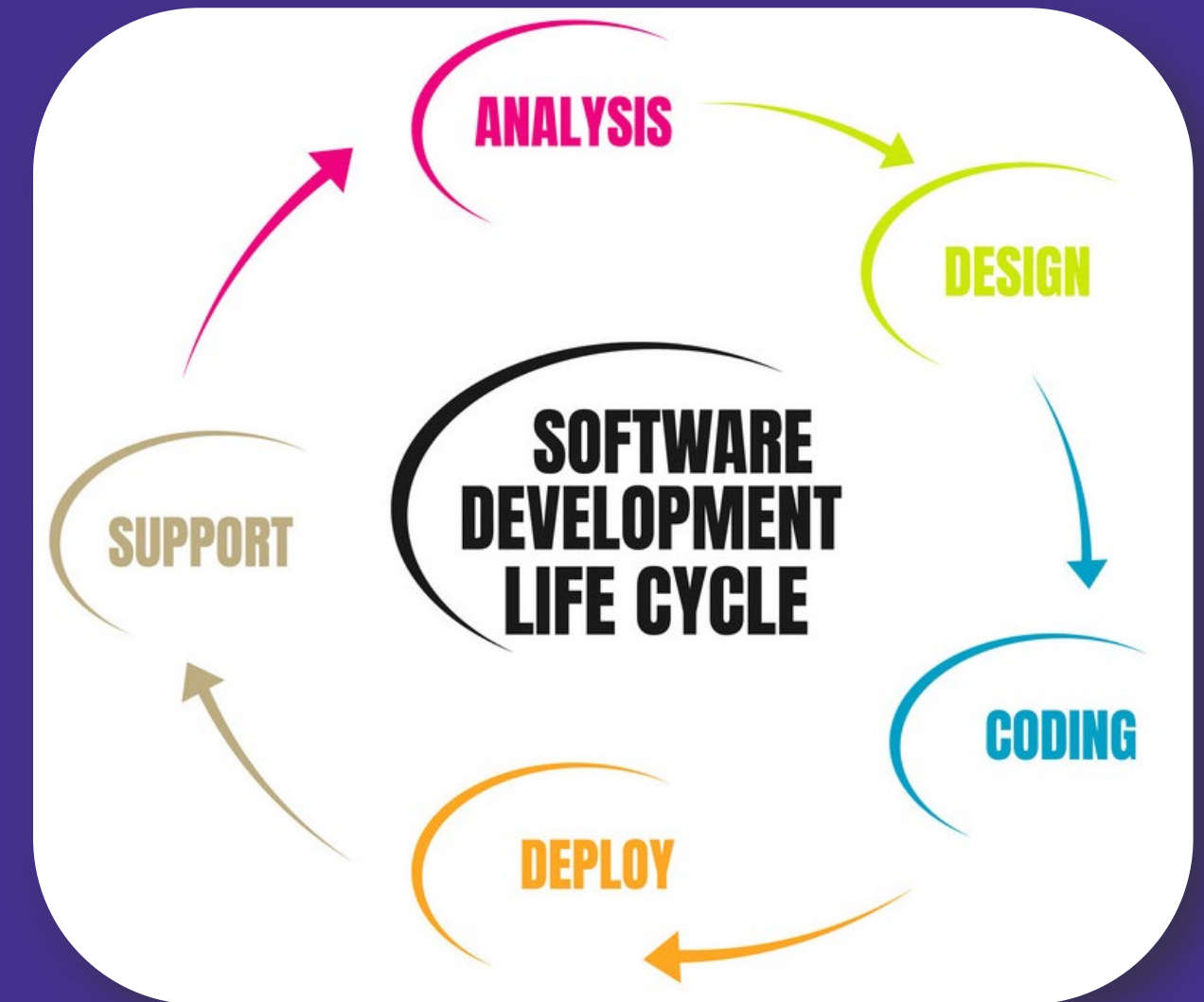
1. The person(s) who wrote the software
2. Another person(s) from the development team
3. A person(s) from a different group
 - independent test team
 - test specialists
4. A person(s) from a different organisation or company

Domain 2: Testing Throughout the Software Development Lifecycle

Outline of Domain 2

- *Module 1: Testing in the Context of a Software Development Lifecycle*
- *Module 2: Test Levels and Test Types*
- *Module 3: Maintenance Testing*

Module 1: Testing in the Context of a Software Development Lifecycle



Software Development and Software Testing

- It is critical for a tester to be familiar with common software development lifecycle models in order to conduct appropriate test activities.
- There are several characteristics of good testing that can be found in any software development lifecycle model:
 - ✓ There is a test activity for every development activity.
 - ✓ Each test level has its own set of test objectives.
 - ✓ During the corresponding development activity, test analysis and design for a given test level begin.
 - ✓ Testers take part in discussions to define and refine requirements and design, and they review work products as soon as drafts are available.

There is a test activity for every development activity:

In a well-structured software development lifecycle (SDLC), every developmental phase should be accompanied by corresponding test activities. For instance, when a feature is being designed, its test scenarios should simultaneously be planned. This ensures that testing is not an afterthought but an integral and continuous part of the development process. It helps in early identification of potential issues, leading to more efficient and cost-effective resolution.

Each test level has its own set of test objectives:

Different test levels (e.g., unit testing, integration testing, system testing, acceptance testing) have distinct purposes. Unit testing might focus on individual software components, while system testing evaluates the software as a whole. By clearly defining objectives for each level, the testing process becomes more systematic and thorough, ensuring that all aspects of the software are rigorously evaluated at the appropriate stage.

During the corresponding development activity, test analysis and design for a given test level begin:

This characteristic emphasises the need to begin test preparation concurrently with development activities. For instance, as developers start coding a module, testers should already be analysing potential test scenarios and designing relevant test cases for that module. This concurrent approach ensures that testing is ready to commence immediately after the development of a feature, speeding up the overall project timeline and increasing efficiency.

Testers take part in discussions to define and refine requirements and design, and they review work products as soon as drafts are available:

Inviting testers to early stages of the project ensures a quality-oriented approach from the outset. When testers are involved in requirement discussions, they can provide feedback from a quality perspective, highlighting potential challenges or ambiguities that might become issues later on. Their early involvement also ensures that they have a clear understanding of the product, enabling them to create more effective and relevant test cases. Furthermore, reviewing work products (like design documents or early code drafts) allows testers to spot inconsistencies or potential issues before they become deeply embedded in the system.

Software Development and Software Testing

(Continued)

- Regardless of the software development lifecycle model used, test activities should begin early in the lifecycle, adhering to the testing principle of early testing.
- The following are the most common software development lifecycle models:
 - ✓ Models of sequential development
 - ✓ Models of iterative and incremental development
- The software development process is described as a linear, sequential flow of activities in a sequential development model. This means that any phase of the development process should start after the previous phase is finished.

Models of Sequential Development:

Sequential development models follow a linear and phased approach, where each phase of the software development process must be completed before moving on to the next one. The most iconic example of this is the Waterfall model. In Waterfall, the project flows downwards through stages like Requirement Gathering, Design, Implementation, Verification, and Maintenance, much like a cascading waterfall. The primary advantage of sequential models is their clarity; each phase has a defined start and end point, making the process predictable. However, their rigidity can be a drawback, as they often don't handle changes in requirements or scope well after the project has commenced.

Models of Iterative and Incremental Development:

Iterative and incremental development models do not see the software development process as a linear flow. Instead, they involve repeated cycles (iterations and increments). One of the most popular examples is the Agile model. In Agile, the software is developed incrementally, with iterations often lasting a few weeks. After each iteration, a usable product increment is delivered, allowing stakeholders to provide feedback early and regularly. This approach is adaptive and is especially beneficial when requirements are expected to change or aren't fully known at the project's outset. It emphasises collaboration, customer feedback, and flexibility, making it more accommodating to changes than sequential models.

Software Development and Software Testing

Sequential Models

- Sequential models are traditional approach for software development and testing, which is often characterised by a linear, phase-by-phase progression.
- The development activities in the Waterfall model are completed one after the other. Test activities are only performed in this model after all other development activities have been completed.
- In contrast to the Waterfall model, the V-model integrates testing throughout the development process, thereby implementing the principle of early testing.
- Furthermore, the V-model includes test levels associated with each development phase, which aids in early testing. In this model, the execution of tests associated with each test level occurs sequentially, but overlapping occurs in some cases.

Software Development Life Cycle Models

Iterative SDLC Models

- Iterative SDLC (Software Development Life Cycle) models are development approaches where software is built in incremental iterations. Each iteration involves a mini-development cycle encompassing requirements gathering, design, development, testing, and deployment. After each iteration, a functional increment is delivered, allowing for continuous feedback and improvement.
- This model emphasises adaptability, flexibility, and customer involvement. Popular iterative models include Agile, Scrum, and Spiral, prioritising customer feedback, collaboration, and delivering incremental value throughout the development process.



Iterative SDLC models are widely adopted in the software industry due to their numerous advantages. By breaking the development process into smaller cycles, it allows for better management of complex projects and mitigates risks. The iterative approach promotes early and continuous customer feedback, enabling teams to make timely adjustments and deliver features that align with user needs.

Software Development Life Cycle Models

Incremental SDLC Models

- The incremental SDLC model is an approach to software development where a project is divided into smaller, manageable parts or increments. Each increment represents a portion of the complete software system and includes a set of features or functionalities.
- In this model, development occurs in multiple iterations, with each iteration building upon the work done in previous ones. Each increment is planned, designed, developed, and tested separately. Once an increment is completed and tested, it is integrated with the existing system. This process is repeated for each subsequent increment until the entire system is fully developed and integrated.

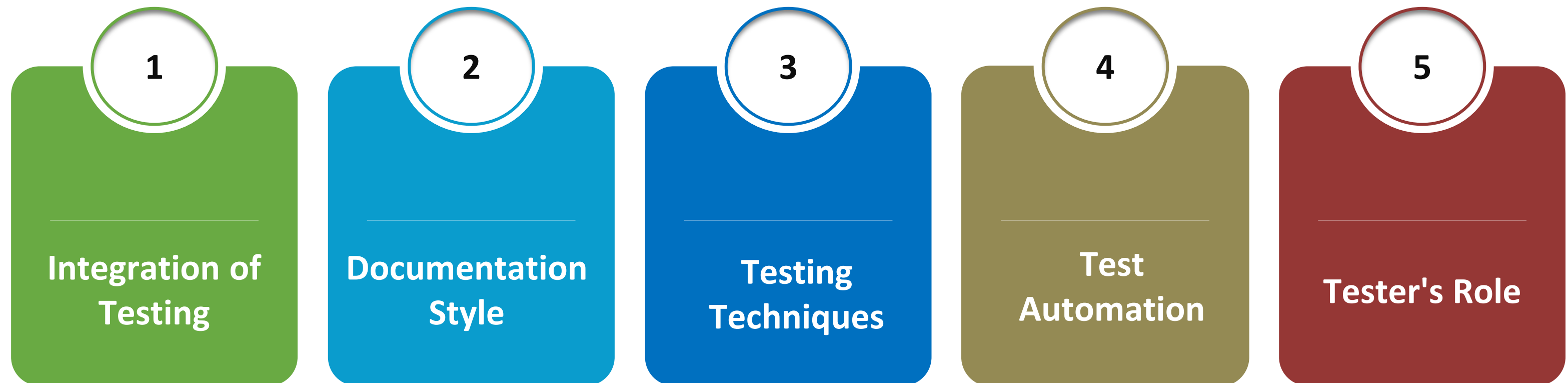


© The Knowledge Academy Ltd

The incremental model offers several benefits, including the ability to deliver a functional product sooner, the opportunity to gather feedback and make improvements throughout the development process, and the flexibility to adapt to changing requirements. It also allows for easier management of large projects by breaking them down into smaller, more manageable pieces.

Impact of the Software Development Lifecycle on Testing

- Testing must be adapted to the Software Development Life Cycle (SDLC) in order to be successful. The choice of SDLC methodology has a significant impact on various aspects of testing.
- *Here's a breakdown of how the choice of SDLC impacts the different areas:*



Impact of the Software Development Lifecycle on Testing

1. Integration of Testing

- ✓ SDLC determines when testing activities are integrated into the development process. In some methodologies like Waterfall, testing occurs as a separate phase after development, while in Agile, it is continuous throughout iterations.

2. Documentation Style

- ✓ SDLC guides the style of test documentation. Traditional SDLCs, such as Waterfall, require detailed and comprehensive test documentation, while Agile favours concise and flexible documents.



Impact of the Software Development Lifecycle on Testing

3. Testing Techniques

- ✓ Different SDLCs may require different testing techniques. For instance, sequential models like Waterfall emphasise requirements-based testing, while Agile approaches may focus on exploratory testing.

4. Test Automation

- ✓ The extent of test automation varies based on the chosen SDLC. Agile methodologies often encourage extensive automation for faster cycles, while traditional models may rely more on manual testing with selective automation.

5. Tester's Role

- ✓ The SDLC methodology influences the role of testers within the development team. In Agile, testers are often integrated into cross-functional teams, collaborating across roles, whereas traditional SDLCs may have more segregated roles for testers.

The **Sequential Model**, often referred to as the **V-Model** (or Validation and Verification model), represents an evolved approach to the traditional Waterfall model in software development. While both are linear in nature, the V-Model's distinct characteristic is how it visually illustrates the parallel relationship between each development stage and its corresponding testing phase.

Impact of the Software Development Lifecycle on Testing

Key characteristics of the V-Model include:

- **Requirements Analysis:** Gathering and documenting user needs and system specifications.
- **System Design:** Defining the system architecture based on requirements.
- **High-Level Testing Planning:** Creating the test plan aligning with design and requirements.
- **Unit Testing:** Testing individual units/modules for functionality.

The V-Model's distinctive feature is the downward left-to-right slope, signifying the progression from planning to testing, and emphasising the importance of early testing in the software development process.



© The Knowledge Academy Ltd

Key characteristics of the V-Model include:

- **Requirements Analysis:** At this initial phase, the primary focus is on understanding and documenting the user's needs and the system's specifications. Stakeholder interviews, workshops, and document reviews are typical methods to capture detailed and accurate requirements.
- **System Design:** Based on the gathered requirements, the overall system architecture is defined. This design phase is crucial as it lays out how the software components will interact with each other and with external systems, considering both functional and non-functional requirements.
- **High-Level Testing Planning:** Before delving deep into coding, the V-Model emphasises the importance of laying out a strategic test plan. This phase involves identifying testing objectives, defining test criteria, and aligning them with the system design and requirements. It sets the stage for subsequent testing activities by determining what to test, how to test, and when to test.
- **Unit Testing:** This is where individual units or components of the software are tested. Each unit is tested in isolation to ensure its functionality aligns with the design and requirements. Tools like xUnit frameworks (e.g., JUnit for Java, NUnit for .NET) are commonly used for this purpose. Defects identified here are typically easier and cheaper to rectify than those found in later stages.

Impact of the Software Development Lifecycle on Testing

The Sequential Model, comprises four distinct levels, each representing a stage of the software development process. These levels are:

1. **Requirements Gathering and Analysis:** In this initial stage, project stakeholders and software engineers gather and document user requirements. The focus is on understanding the scope of the project, defining functionalities, and setting project objectives.
2. **System Design:** Once requirements are established, the system design phase commences. This stage involves creating a detailed blueprint of the software architecture, specifying the system's structure and components.
3. **Implementation/Coding:** After the design is complete, the development team begins coding based on the design specifications. The focus is on transforming the design into functional code.
4. **Testing and Deployment:** The final level entails testing the developed software against the previously defined requirements. Once the software passes testing, it is deployed to the end-users.

© The Knowledge Academy Ltd

Requirements Gathering and Analysis:

The project stakeholders for our online bookstore may include book vendors, a marketing team, end-users (book readers), and the IT team. Together, they list out their requirements: a user-friendly interface, secure payment gateways, an efficient search function, a recommendation engine, and so on. The IT team collaborates with all parties to ensure that they have a comprehensive understanding of what the online bookstore should offer in terms of functionality and user experience.

System Design: With the requirements in hand, the software architects design the system. They decide to use a microservices architecture for flexibility and scalability. The design will include an authentication service for user accounts, a database design for storing book information and user data, a payment service integrating third-party gateways, and an interface design that's both mobile and desktop-friendly.

Implementation/Coding: Development teams start translating the design into actual code. The frontend team might use React for the user interface, while the backend team could use Python with Django for server-side operations. As they code, they refer back to the design specifications to ensure alignment.

Testing and Deployment: A Quality Assurance (QA) team tests the online bookstore. They check if the search function retrieves books accurately, if the payment process is smooth and secure, if the recommendation engine provides relevant suggestions, and more. After rigorous rounds of testing and necessary fixes, the platform is deemed ready. It's then deployed to a live environment, and users can now explore, select, and purchase their favorite books online.

This example illustrates how an idea transforms into a tangible product in a structured and sequential manner through the Sequential Model. Each level builds upon the previous, ensuring a systematic approach to software development.

Software Development Lifecycle and Good Testing Practices

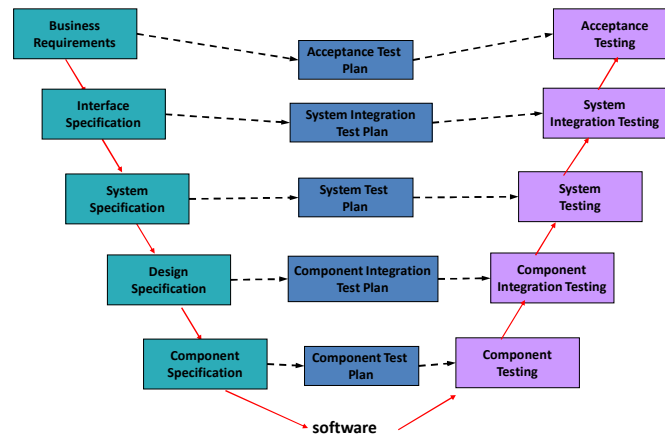
- Implementing good testing practices is essential for ensuring the quality and reliability of software. Here are some recommended testing practices:
- 1. **Integration of Testing Activities:** In the realm of software development, each activity has a corresponding test activity, ensuring that quality control is applied throughout the process.
- 2. **Targeted Test Objectives for Different Test Levels:** Different test levels possess unique objectives, preventing redundant testing while maintaining comprehensive coverage. This approach ensures that testing efforts are optimised by focusing on specific aspects of the software at each level.
- 3. **Early Test Analysis and Design:** To adhere to the principle of early testing, test analysis and design commence during the corresponding development phase of the SDLC.
- 4. **Involvement of Testers in Documentation Reviews:** Testers actively participate in reviewing work products as soon as drafts of documentation become available. This approach supports the shift-left strategy, where testing is initiated early in the SDLC.

© The Knowledge Academy Ltd

1. **Integration of Testing Activities:** By aligning testing with development, potential defects are identified early, minimising the impact on later stages. This approach establishes a continuous feedback loop, promoting consistent quality assurance from start to finish.
2. **Targeted Test Objectives for Different Test Levels:** Consequently, the software undergoes a well-rounded examination without unnecessary duplication of testing efforts.
3. **Early Test Analysis and Design:** This proactive strategy identifies potential issues early in the development process, leading to quicker mitigation of defects. This synergy between testing and development reduces rework and enhances overall efficiency.
4. **Involvement of Testers in Documentation Reviews:** By engaging testers during the documentation phase, defects and inconsistencies are caught sooner, reducing the likelihood of defects permeating subsequent stages.

Software Development Models

Test Level Plans



© The Knowledge Academy Ltd

The above are examples of typical software work products

Software work products (such as business scenarios or use cases, requirements specifications, design documents and code) produced during development are often the basis of testing in one or more test levels. Verification and validation (and early test design) can be carried out during the development of the software work products.

For each test level a test level plan can be produced. This should ideally be done as soon as the appropriate work products has been produced. This plan will then be used to control the testing of the level in question when the time comes.

Testing as a Driver for Software Development

- **Test-Driven Development (TDD):** TDD is a software development methodology that directs the coding through test cases, rather than focusing on extensive software design. In this approach, tests are written first, followed by the code to satisfy those tests.
- **Acceptance Test-Driven Development (ATDD):** ATDD, takes TDD a step further by deriving tests from acceptance criteria as part of the system design process. These criteria define what the system is supposed to do, and the tests are written before the relevant part of the application is developed.
- **Behaviour-Driven Development (BDD):** BDD is a refinement of TDD that focuses on the desired behaviour of an application. Unlike the more technical focus of TDD and ATDD, BDD allows test cases to be written in a simple form of natural language, making it easy for various stakeholders to understand.



© The Knowledge Academy Ltd

Test-Driven Development (TDD): This ensures that the code precisely meets the requirements laid out by the tests. Once the code passes the tests, both the tests and code may be refactored to improve their quality. TDD emphasises early testing and a shift-left approach, meaning that testing begins early in the development process to catch defects as soon as possible.

Acceptance Test-Driven Development (ATDD): The advantage of ATDD is that it ensures that the developed feature or system component meets the acceptance criteria and, consequently, the customer's expectations. Like TDD, ATDD also adheres to the principles of early testing and the shift-left approach.

Behaviour-Driven Development (BDD): Typically using the Given/When/Then format, BDD expresses the requirements as user stories or scenarios. These test cases are then automatically translated into executable tests, bridging the communication gap between development, testing, and business teams. BDD ensures that the software is aligned with business objectives and that it behaves as intended.

In all three approaches, testing plays a central role in guiding development. The tests act as living documentation, ensuring that the code behaves as expected and meets stakeholder needs. By using testing as a driver for development, these methodologies promote a more structured and focused development process, resulting in higher-quality software that aligns closely with user expectations and business requirements.

DevOps and Testing

- It is essential to understand the relationship between DevOps and testing.
- DevOps is a software development approach that combines development (Dev) and operations (Ops) to foster collaboration, streamline processes, and enable faster delivery of high-quality software.
- Testing plays a significant role in the DevOps context, ensuring that the software meets the desired quality standards before deployment.
- DevOps promotes a shift-left approach, where testing activities are integrated throughout the entire software development lifecycle, rather than being limited to a separate phase.
- This means that testers are involved right from the requirements gathering and design stages, collaborating closely with developers, operations, and other stakeholders.
- Testers work alongside the development team to ensure that quality is built into the software from the beginning, aiming for early detection and resolution of defects.

© The Knowledge Academy Ltd

How DevOps and Testing Work Together?

- **Shared Responsibility:** Developers, testers, and operations share the responsibility for delivering high-quality software.
- **Early Involvement:** Testers are involved from the start, even from the requirements and design stages, to ensure quality is built into the product.
- **Feedback Loop:** Testing provides constant feedback to all involved, making it easier to identify and address issues before they escalate.
- **Test-Driven Development (TDD):** A practice often encouraged in DevOps, where tests are written before the code that needs to be tested.
- **Automated Pipelines:** Both DevOps and testing heavily rely on automation to quickly move from code commit to deployment, while ensuring the software's reliability and quality.

DevOps and Testing

Benefits of DevOps

- 1. Fast Feedback:** DevOps encourages continuous integration, leading to immediate feedback on the code. This allows developers to promptly detect and correct any issues, such as code defects or negative impacts on existing functionality. It ensures that code alterations align with project requirements and do not inadvertently break other parts of the system.
- 2. Shift-Left Approach:** By shifting testing to earlier phases in the development cycle, DevOps fosters a culture of quality. Component tests and static analysis are utilised to verify code quality even before integration, thus leading to robust, high-quality code.
- 3. Stable Test Environments:** Automation in the DevOps cycle, particularly with CI/CD, ensures consistent and reliable test environments. Stable test environments are vital for achieving accurate test results and a smoother development lifecycle.

DevOps and Testing

4. **View on Non-functional Characteristics:** DevOps emphasises not just functional testing but also focuses on non-functional attributes like performance and reliability. Continuous monitoring and testing of these aspects ensure that the system performs optimally.
5. **Reduces Manual Testing:** By implementing a delivery pipeline that automates various stages, DevOps significantly reduces the necessity for repetitive manual testing. This leads to a faster development cycle and frees up valuable time and resources.
6. **Minimised Regression Risk:** In the DevOps approach, automated regression testing plays a crucial role in minimising risks. Automated regression tests ensure that new changes do not negatively impact existing functionalities, mitigating the risk of unexpected failures.
7. **DevOps Delivery Pipeline:** The DevOps delivery pipeline must be precisely defined and set up. Misalignment or misconfiguration can lead to inefficiencies, delays, or even failures in the development process.

DevOps and Testing

8. **CI/CD Tool Maintenance:** Continuous Integration and Continuous Deployment (CI/CD) tools must be carefully selected, integrated, and maintained. This can be time-consuming and requires specialised skills.
9. **Test Automation Challenges:** Establishing and maintaining automated testing can be complex and labor-intensive. It requires additional resources and, if not managed correctly, can become a bottleneck in the development process.



Shift Left Approach

- The Shift Left approach in software testing refers to the practice of conducting testing activities earlier in the software development lifecycle. By shifting testing left, testing activities are initiated as soon as possible, often starting during the requirements and design phases.
- This approach aims to identify defects and issues at their early stages, minimising their impact and reducing overall testing costs. It emphasises collaboration between developers, testers, and stakeholders to ensure better communication and understanding of requirements.
- Adopting Shift Left also promotes faster feedback loops, accelerating the development process, and enhancing the software's quality by addressing issues proactively before they propagate to later stages.
- This approach enables faster delivery of high-quality software products, as issues are identified and resolved in real-time, ensuring a smoother and more efficient software development process.

Importance of Shift Left Approach?

- **Improved Quality:** Early Detection of Issues: Identifying problems in the initial stages of development allows for less expensive and quicker fixes. Built-in Quality: Testers working alongside developers from the get-go ensures that quality is built into the product, rather than being "tested in" later.
- **Enhanced Collaboration:** Team Synergy: Involving everyone—developers, testers, operations, and business analysts—from the beginning fosters better understanding and teamwork. Clearer Communication: Regular interactions between team members ensure that everyone understands the requirements and expectations.
- **Faster Time-to-Market: Streamlined Processes:** Continuous testing and integration enable faster development cycles. Reduced Lead Time: With early testing, the time from concept to market is reduced, offering a competitive advantage.
- **Cost-Effectiveness:** Reduced Rework: Catching bugs early means they are cheaper and quicker to fix, which saves both time and money in the long run. Optimised Resources: Developers and testers can work more efficiently, which can lead to cost savings.
- **Better Risk Management:** Comprehensive Coverage: Early and continuous testing provides more opportunities to identify and mitigate risks. Enhanced Reliability: Consistent, early testing makes the final product more reliable, thus boosting customer confidence.

Shift Left Approach

Good practices to achieve a “shift-left” in testing

- **Reviewing Specifications:** By reviewing the specifications from the perspective of testing, teams often identify potential defects such as ambiguities, incompleteness, and inconsistencies. This proactive approach helps prevent issues from becoming embedded in the code, leading to more accurate and efficient development.
- **Writing Test Cases Early:** Writing test cases before the code is written allows developers to think from a testing mindset from the onset. Having the code run in a test harness during implementation ensures that code is testable and aligns with expected outcomes, leading to more robust and reliable code.
- **Using CI/CD for Fast Feedback:** Continuous Integration (CI) and Continuous Deployment (CD) come with fast feedback and automated component tests. When code is submitted to the repository, these practices ensure immediate evaluation, allowing for rapid adjustments. This promotes a more flexible and responsive development process.

Shift Left Approach

(Continued)

- **Completing Static Analysis:** Performing static analysis of source code prior to dynamic testing or as part of an automated process ensures that code adheres to quality standards and identifies potential errors at an early stage. This means fewer issues in later stages, saving time and resources.
- **Performing Non-Functional Testing Early:** By conducting non-functional testing starting at the component test level, it's possible to evaluate aspects like performance and reliability sooner. Traditionally, these tests are performed later in the SDLC (Software Development Life Cycle) when a complete system is available. Shifting these tests to earlier stages enhances the understanding of how components will behave in the final system.

Retrospectives and Process Improvement

- Retrospectives refer to the practice of reflecting on past experiences and identifying opportunities for improvement in the testing process. They provide a structured approach for teams to assess their performance, learn from their successes and failures, and make adjustments to enhance future testing efforts.
- Retrospectives involve gathering feedback, analysing what worked well and what didn't, and implementing changes to enhance future projects. It encourages open and honest communication among team members, fostering a culture of continuous learning and growth.
- During a retrospective, the team identifies strengths, weaknesses, challenges, and areas of improvement in the testing process. Testers share their observations, insights, and suggestions to enhance the overall testing process and deliver higher-quality software.

Retrospective Examples:

Example: Automating Repetitive Testing Tasks

- **Problem:** Team members express frustration over the time consumed by repetitive manual testing.
- **Discussion:** During the retrospective, the team realises that automating these tests could save time.
- **Action:** Implement test automation for those repetitive tasks before the next sprint.

Process Improvement Examples:

Example: Transition to Test-Driven Development (TDD)

- **Retrospective Finding:** High bug rates in code.
- **Improvement:** Adopt TDD, writing tests before code.
- **Effect:** Lower defect rates in subsequent sprints.

Retrospectives and Process Improvement

Retrospectives benefits for testing include

- **Increased Test Effectiveness / Efficiency:** Through retrospectives, teams can evaluate their testing processes, identify areas for improvement, and implement suggestions that enhance both effectiveness and efficiency. By constantly refining their approach, teams ensure that testing is aligned with goals, targeted, and resource-efficient.
- **Increased Quality of Testware:** By jointly reviewing test processes during retrospectives, team members gain insights into what is working well and what isn't. This collective analysis helps to improve the quality of testware, leading to more reliable testing outcomes and ultimately a higher-quality product.
- **Team Bonding and Learning:** Retrospectives create an environment where team members can openly raise issues and propose improvement points. This collaborative approach fosters a sense of unity and shared purpose within the team. It also promotes learning, as team members can share knowledge and insights, helping the team grow and adapt together.

Retrospectives and Process Improvement

- **Improved Quality of the Test Basis:** Addressing and solving deficiencies in the extent and quality of requirements is vital to effective testing. Retrospectives provide a forum where these deficiencies can be identified and addressed, ensuring that the test basis is solid and reflects the true needs and expectations of the project.
- **Better Cooperation Between Development and Testing:** Regular review and optimisation of collaboration between development and testing teams are essential for a smooth development process. Retrospectives enable an open dialogue about how these two crucial functions can work together more effectively. This improves communication and understanding, leading to a more harmonious and productive relationship

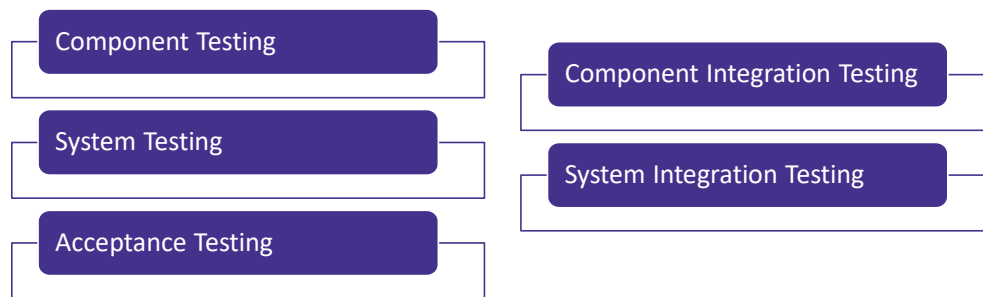


Module 2: Test Levels and Test Types



Test Levels

- Test levels refer to a group of test activities that are organised and managed together to achieve specific testing objectives. Understanding test levels is crucial for testers to effectively plan, design, and execute testing activities.
- *Here are the commonly recognised test levels within the certification:*



© The Knowledge Academy Ltd

Component Testing: Also known as unit testing, this level focuses on testing individual components or modules of the software in isolation. Testers verify the functionality of each component and ensure they perform as intended. Component testing often involves working closely with developers and using techniques such as stubs and drivers to simulate interfaces and dependencies.

Integration Testing: Integration testing aims to test the interaction between multiple components or modules after they have been integrated into a larger system. Testers ensure that the integrated components work harmoniously, detect any interface issues, and verify that data flows correctly between different modules. This level of testing identifies defects that may arise due to the interaction between components.

System Testing: System testing verifies the behavior of the entire system as a whole, including all integrated components, modules, and external interfaces. Testers focus on validating functional and non-functional requirements, conducting end-to-end tests, and assessing system performance, security, usability, and reliability. System testing aims to ensure that the software meets the specified business and user requirements.

System Integration Testing:

System integration testing emphasises the interfaces of the system under test with other systems and external services. This test level requires suitable environments that preferably mirror the operational setting. By concentrating on the interactions between the system and other external components, system integration testing ensures that different systems cooperate smoothly.

Acceptance Testing: Acceptance testing is performed to determine whether the system satisfies the user's requirements and is ready for deployment. It involves executing tests based on real-life scenarios or user stories. Testers collaborate with stakeholders, including business analysts and end-users, to validate that the system meets the desired objectives and is fit for its intended purpose.

Component Testing

- Also known as UNIT, MODULE, or PROGRAM testing.

Searches for defects in and verifies the functioning of software modules,

- programs, objects, classes and so on. **that are separately testable.**

Test Basis:-

Component specifications
Detailed design
Code
Data Model

Typical test objects:-

Components, units or modules
Classes
Code and data structures
Database modules

Component testing searches for defects in, and verifies the functioning of, components (e.g. modules, programs, objects, classes, etc.) that are separately testable.

It may be done in isolation from the rest of the system, depending on the context of the development life cycle and the system.

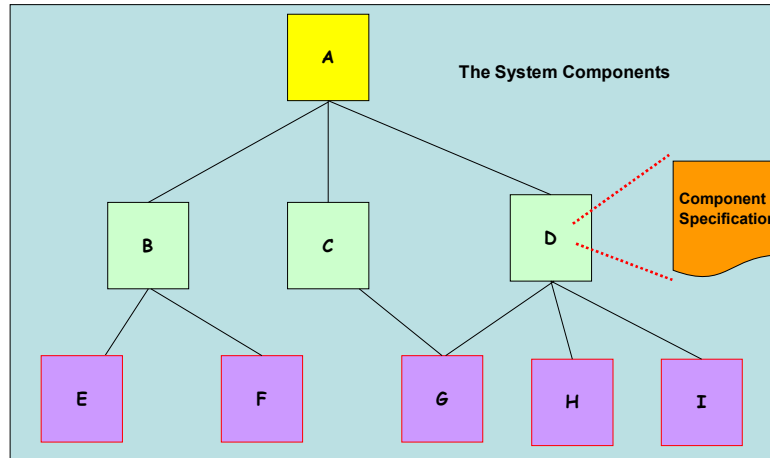
Component testing is often also referred to as unit, module or program testing .

Objectives of component testing include:

- Reducing risk
- Verifying whether the functional and non-functional behaviours of the component are as designed and specified
- Building confidence in the component's quality
- Finding defects in the component
- Preventing defects from escaping to higher test levels

Component Testing

The System



© The Knowledge Academy Ltd

A software system is usually designed to comprise of a set of components each of which will result in a small subset of the full code requirement. For each component a specification is produced which details what is required of that piece of code. In other words the system is decomposed into its constituent parts, each part referred to as a component.

ISTQB definition of a system is therefore:-

system: A collection of components organised to accomplish a specific function or set of functions. Although not shown in the diagram above, the necessary communication between components to form a complete working system also has to be specified.

Examples of typical defects and failures for component testing include:

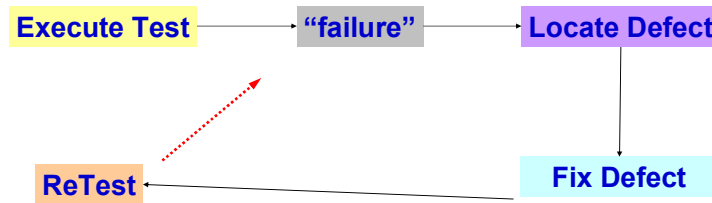
- Incorrect functionality (e.g., not as described in design specifications)
- Data flow problems
- Incorrect code and logic

Component Testing

(Continued)

Usually done by the programmer who wrote the code

Typical practice:-



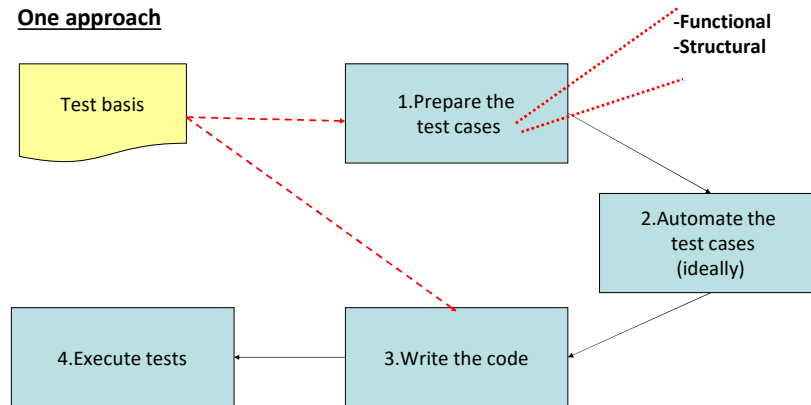
No formal defect management!

In practice, component testing usually involves the programmer who wrote the code. Defects are typically fixed as soon as they are found, without formally managing these defects.

Component Testing

(Continued)

One approach



'the test first approach' or 'test driven development'

© The Knowledge Academy Ltd

One approach to component testing is to prepare and, if possible, automate test cases before coding. This is called a test-first approach or test-driven development. This approach is highly iterative and is based on cycles of developing test cases, then building and integrating small pieces of code, and executing the component tests correcting any issues and iterating until they pass.

Re-iterate that both functional and structural tests should be prepared. This approach is preferable to 'test last', where the tests are often prepared as an afterthought once coding is complete. With 'test first' the requirement to pass the tests that have been prepared tends to focus the mind of the developer which greatly improves the quality of the code.

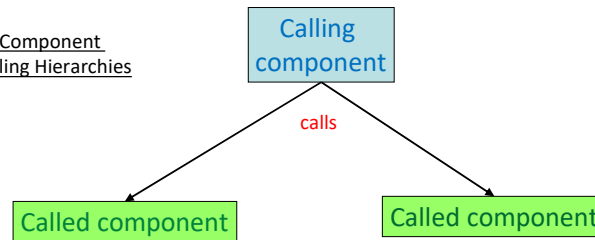
ISTQB definition:-

test driven development: A way of developing software where the test cases are developed, and often automated, before the software is developed to run those test cases.

Component Testing

(Continued)

Component
Calling Hierarchies



Drivers replace missing **Calling** components.

Stubs replace missing **Called** components.

Both are special purpose skeletal modules

Some necessary terminology. In the relationship between components one is designated as 'the caller' and the other as 'the called'. The communication between them is via 'calls'. Hence the caller calls the called. This presents problems when testing components because the absence of a calling component or called component means that it is not possible to properly test a component that communicates with them.

This problem is overcome by the use of stubs and drivers

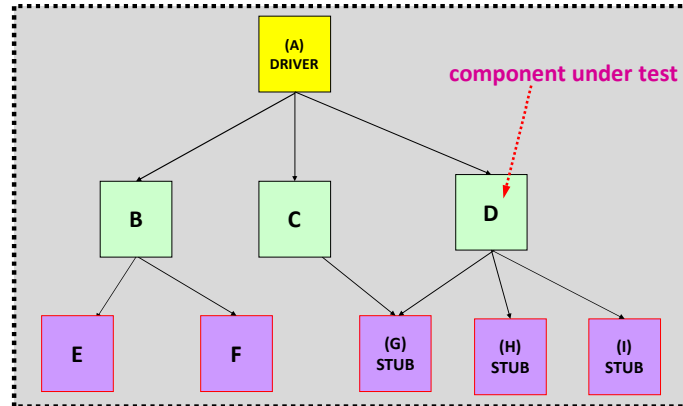
stub: A skeletal or special-purpose implementation of a software component, used to develop or test a component that calls it or is otherwise dependent on it.

driver: A software component or test tool that replaces a component that takes care of the control and/or the calling of a component or system.

Component Testing

(Continued)

- **Unit Test Framework/Debugging Tool**



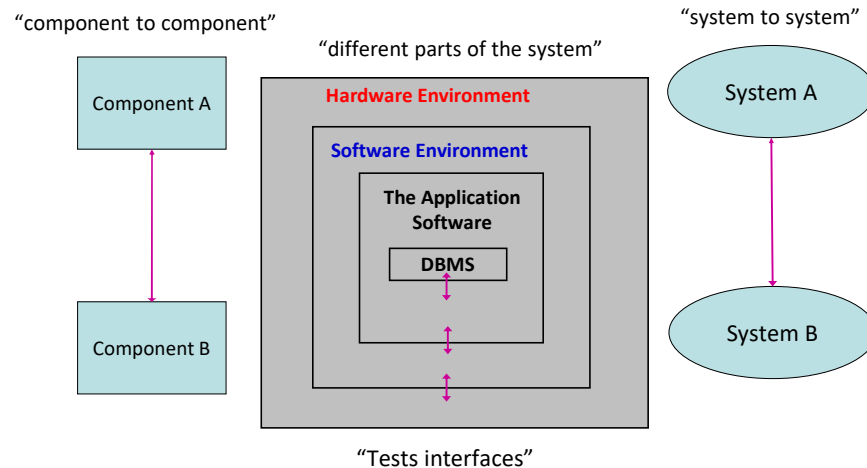
© The Knowledge Academy Ltd

A Unit Test Framework is a software tool that provides an environment for developers to write, run, and evaluate unit tests. Unit tests are automated tests that verify the behavior of individual components or units of a software application

A Debugging Tool is a software application used for testing and debugging other programs. The primary function of a debugging tool is to help the programmer identify errors or anomalies in the code.

Both Unit Test Frameworks and Debugging Tools are essential for software development, helping developers to write robust and error-free code. They are often used in conjunction to get the most comprehensive testing and debugging coverage.

Integration Testing



© The Knowledge Academy Ltd

Integration testing tests interfaces between components, interactions with different parts of a system, such as the operating system, file system and hardware, and interfaces between systems.

There may be more than one level of integration testing and it may be carried out on test objects of varying size as follows:

1. Component integration testing tests the interactions between software components and is done after component testing
2. System integration testing tests the interactions between different systems or between hardware and software and may be done after system testing. In this case, the developing organisation may control only one side of the interface.

This might be considered as a risk, Business processes implemented as workflows may involve a series of systems. Cross platform issues may be significant.

The greater the scope of integration, the more difficult it becomes to isolate failures to a specific component or system, which may lead to increased risk and additional time for troubleshooting.

Objectives of integration testing include:

- Reducing risk
- Verifying whether the functional and non-functional behaviours of the interfaces are as designed and specified
- Building confidence in the quality of the interfaces
- Finding defects (which may be in the interfaces themselves or within the components or systems)
- Preventing defects from escaping to higher test levels

Integration Testing

(Continued)

Test Basis:-

- Software and System Design
- Architecture
- Workflows
- Use Cases
- Sequence diagrams

Typical test objects:-

- Sub-systems & databases
- Infrastructure
- Interfaces
- API's

CIT & SIT

Testing of specific non-functional characteristics (e.g., performance) may be included in integration testing as well as functional testing.

At each stage of integration, testers concentrate solely on the integration itself. For example, if they are integrating module A with module B they are interested in testing the communication between the modules, not the functionality of the individual module as that was done during component testing.

Both functional and structural approaches may be used.

Ideally, testers should understand the architecture and influence integration planning.

If integration tests are planned before components or systems are built, those components can be built in the order required for most efficient testing.

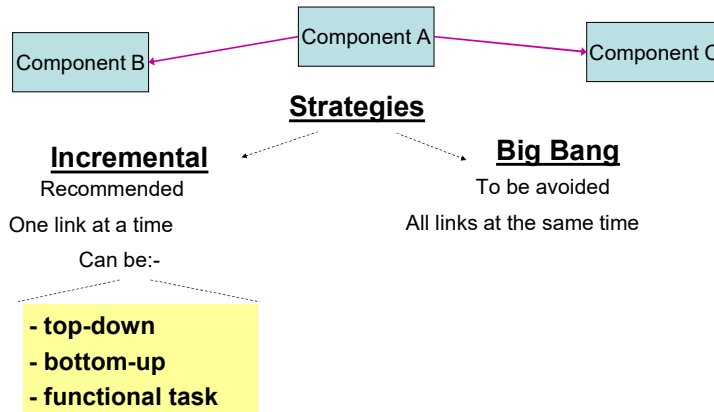
Examples of typical defects and failures for system integration testing include:

- Inconsistent message structures between systems
- Incorrect data, missing data, or incorrect data encoding
- Interface mismatch ☒ Failures in communication between systems
- Unhandled or improperly handled communication failures between systems
- Incorrect assumptions about the meaning, units, or boundaries of the data being passed between systems
- Failure to comply with mandatory security regulations

Integration Testing

Component Integration Testing

"Tests interactions between software components"



© The Knowledge Academy Ltd

Testing of specific non-functional characteristics (e.g., performance) may be included in integration testing as well as functional testing.

At each stage of integration, testers concentrate solely on the integration itself. For example, if they are integrating module A with module B they are interested in testing the communication between the modules, not the functionality of the individual module as that was done during component testing.

Both functional and structural approaches may be used.

Ideally, testers should understand the architecture and influence integration planning.

If integration tests are planned before components or systems are built, those components can be built in the order required for most efficient testing.

Examples of typical defects and failures for system integration testing include: ☐ Inconsistent message structures between systems ☐ Incorrect data, missing data, or incorrect data encoding ☐ Interface mismatch ☐ Failures in communication between systems ☐ Unhandled or improperly handled communication failures between systems ☐ Incorrect assumptions about the meaning, units, or boundaries of the data being passed between systems ☐ Failure to comply with mandatory security regulations

System Testing

- 'The process of testing a system to verify that it meets specified requirements'.

Test Basis:-

system and software requirements specification
uses cases
models, state diagrams
risk analysis reports
Epics, user stories
system and user manuals

Typical test objects:-

Applications
operating systems
system configuration
system under test (SUT)

Concerned with the end to end behaviour of the whole system/product

System testing is concerned with the behaviour of a whole system/product.

The testing scope shall be clearly addressed in the Master and/or Level Test Plan for that test level.

System testing should investigate functional and non-functional requirements of the system, and data quality characteristics.

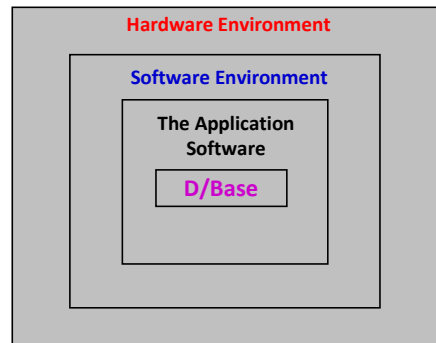
Testers also need to deal with incomplete or undocumented requirements.

Typical defects and failures Examples of typical defects and failures for system testing include:

- Incorrect calculations
- Incorrect or unexpected system functional or non-functional behaviour
- Incorrect control and/or data flows within the system
- Failure to properly and completely carry out end-to-end functional tasks
- Failure of the system to work properly in the production environment(s)
- Failure of the system to work as described in system and user manuals.

System Testing

The Test Environment



'should correspond to the final target environment'

© The Knowledge Academy Ltd

In system testing, the test environment should correspond to the final target or production environment as much as possible in order to minimise the risk of environment-specific failures not being found in testing.

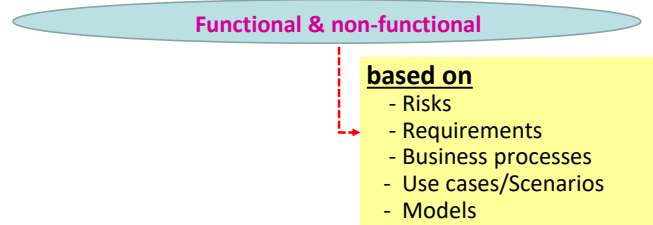
Whilst this is the ideal it is often prohibitively expensive in practice, which means risks are taken.

Objectives of system testing include:

- Reducing risk
- Verifying whether the functional and non-functional behaviors of the system are as designed and specified
- Validating that the system is complete and will work as expected
- Building confidence in the quality of the system as a whole
- Finding defects
- Preventing defects from escaping to higher test levels or production

System Testing

(Continued)



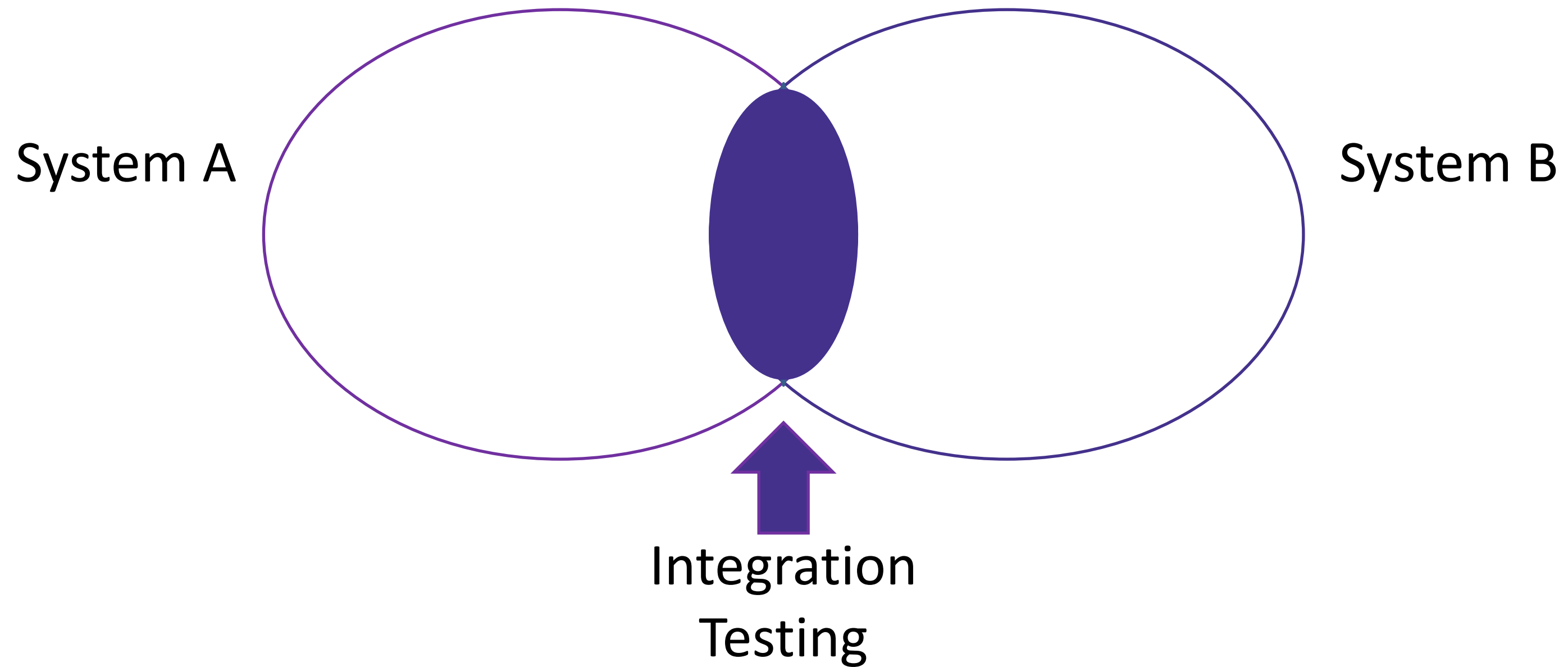
Often done by an independent test team

System testing is concerned with the behaviour of a whole system/product.
The testing scope shall be clearly addressed in the Master and/or Level Test Plan for that test level.
System testing should investigate functional and non-functional requirements of the system, and data quality characteristics.
Testers also need to deal with incomplete or undocumented requirements.

Typical defects and failures Examples of typical defects and failures for system testing include:

- Incorrect calculations
- Incorrect or unexpected system functional or non-functional behaviour
- Incorrect control and/or data flows within the system
- Failure to properly and completely carry out end-to-end functional tasks
- Failure of the system to work properly in the production environment(s)
- Failure of the system to work as described in system and user manuals.

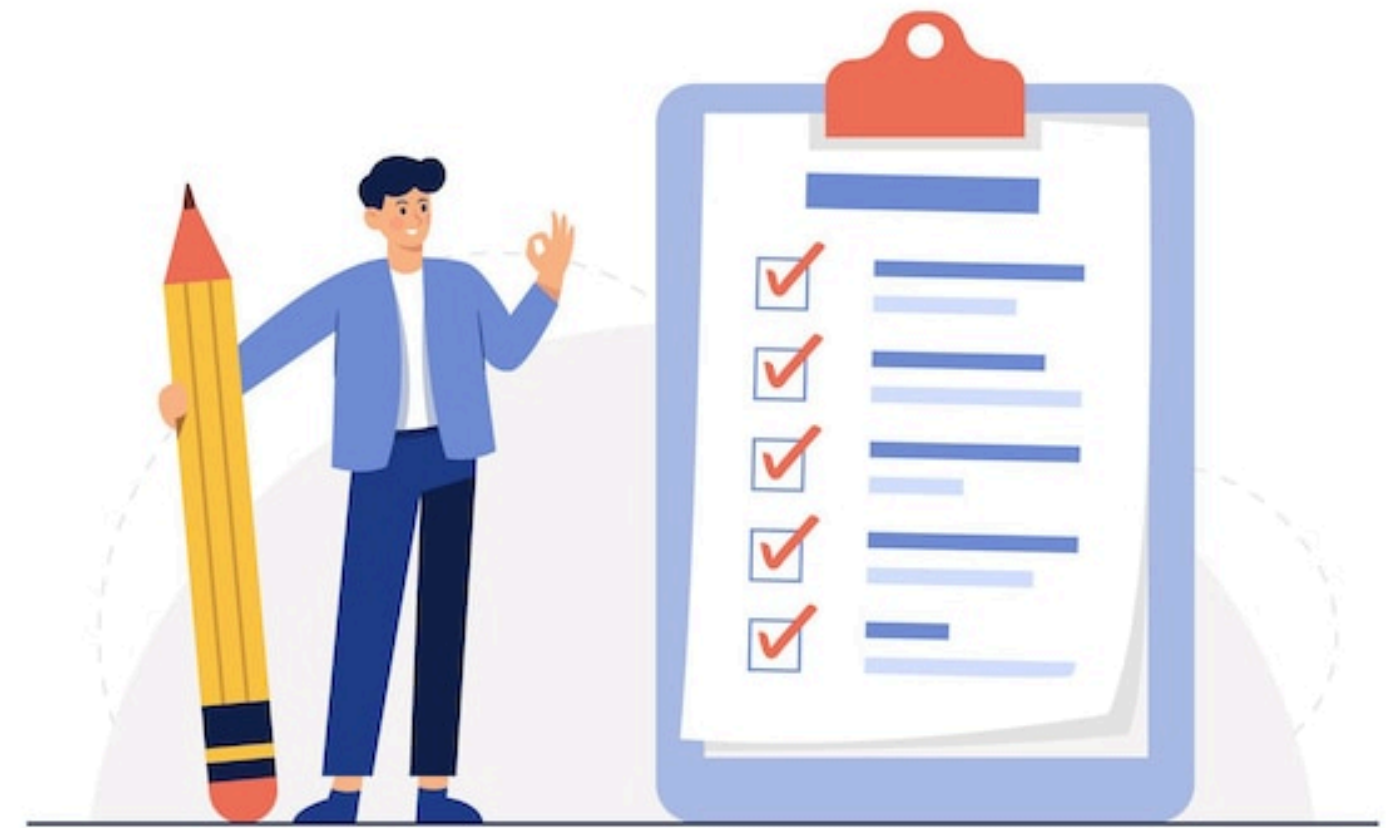
System Integration Testing (SIT)



System Integration Testing (SIT)

Importance of Testing Interfaces

Interfaces are the points where two different systems or components connect and communicate with each other. Errors at these connection points can have significant implications, leading to data inconsistencies, functionality breakdowns, or even complete system failure. System Integration Testing aims to uncover issues in these areas, making sure that integrated components are working together seamlessly.



Acceptance Testing

Establishes **confidence** in the system

Test Basis:-

- User requirements
- System requirements
- Use cases
- Business processes
- Risk analysis reports
- Installation procedures
- Regulations

Typical test objects:-

- Business processes
- Operational processes
- Maintenance processes
- User procedures
- Forms, Reports
- Recovery systems and hot sites

Includes functional and non-functional characteristics

© The Knowledge Academy Ltd

Acceptance testing is often the responsibility of the customers or users of a system; other stakeholders may be involved as well.

The goal in acceptance testing is to establish confidence in the system, parts of the system or specific non-functional characteristics of the system.

Finding defects is not the main focus in acceptance testing.

Acceptance testing may assess the system's readiness for deployment and use, although it is not necessarily the final level of testing. For example, a large-scale system integration test may come after the acceptance test for a system.

Acceptance testing may occur at various times in the life cycle, for example:

- A COTS software product may be acceptance tested when it is installed or integrated
- Acceptance testing of the usability of a component may be done during component testing
- Acceptance testing of a new functional enhancement may come before system testing

Confidence really means that you would be happy to release the product into the live environment with out worry.

Typical defects and failures Examples of typical defects for any form of acceptance testing include:

- System workflows do not meet business or user requirements
- Business rules are not implemented correctly
- System does not satisfy contractual or regulatory requirements
- Non-functional failures such as security vulnerabilities, inadequate performance efficiency under high loads, or improper operation on a supported platform.

Acceptance Testing

(Continued)

- **User Acceptance (UAT)**

- ✓ Verifies fitness for use of the system by business users.
- ✓ Typically the responsibility of business users.

- ✓ **Operational Acceptance (OAT)**

- ✓ Verifies fitness for use of the system by system administrators.
- ✓ Done by system administrators.

includes

- testing of back-up/restore
- disaster recovery
- user management
- maintenance tasks
- data load and migration tasks
- periodic checks of security vulnerabilities

© The Knowledge Academy Ltd

User acceptance testing

Typically verifies the fitness for use of the system by business users.

Operational (acceptance) testing

The acceptance of the system by the system administrators, including:

Backup and restore procedures

- Disaster recovery procedures
- Non-functional requirements
- Operations documentation
- Deployment and installation instructions
- Performance targets
- Database packages
- Security standards or regulations

User acceptance should be done people who will use the live system on a day to day basis

Operational acceptance by people who will look after the live system on a day to day basis

Acceptance Testing

Contract and Regulation Acceptance

- Performed against contract acceptance criteria.
- Performed against regulations:
 - ✓ governmental
 - ✓ legal
 - ✓ Safety

Alpha and Beta

- Applies to Commercial Off-The-Shelf software (COTS).
- Done by potential or existing customers.

Alpha

- done at **vendors site**
- aka factory testing
- (may be done by independent test team)

Beta

- done at **customers site**
- aka field or site testing

Contract and regulation acceptance testing

Contract acceptance testing is performed against a contract's acceptance criteria for producing custom-developed software. Acceptance criteria should be defined when the parties agree to the contract. Regulation acceptance testing is performed against any regulations that must be adhered to, such as government, legal or safety regulations.

Alpha and beta (or field) testing

Developers of market, or COTS, software often want to get feedback from potential or existing customers in their market before the software product is put up for sale commercially. Alpha testing is performed at the developing organisation's site but not by the developing team. Beta testing, or field-testing, is performed by customers or potential customers at their own locations.

Organisations may use other terms as well, such as factory acceptance testing and site acceptance testing for systems that are tested before and after being moved to a customer's site.

Test Types

1. **Functional Testing:** This type of testing verifies that the software functions as intended, based on its specified requirements. Testers validate the expected behaviour of the software and ensure that it performs the desired functions accurately.



Takes an external (black-box) view of the software

- A unit/system has a functional specification.
- Inputs & test data generated from functional specification and/or models of the system.
- Outputs check functionality.

aka. specification-based testing

The functions that a system, subsystem or component are to perform may be described in work products such as a requirements specification, use cases, or a functional specification, or they may be undocumented.

The functions are "what" the system does.

Functional tests are based on functions and features (described in documents or understood by the testers) and their interoperability with specific systems, and may be performed at all test levels (e.g., tests for components may be based on a component specification).

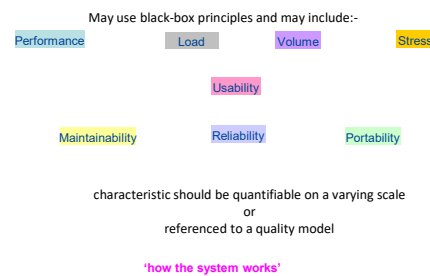
Specification-based techniques may be used to derive test conditions and test cases from the functionality of the software or system. Functional testing considers the external behaviour of the software (black-box testing).

functional, non-functional, white-box, and change-related tests will be given across all test levels, for a banking application, starting with **functional tests**:

- For component testing, tests are designed based on how a component should calculate compound interest.
- For component integration testing, tests are designed based on how account information captured at the user interface is passed to the business logic.
- For system testing, tests are designed based on how account holders can apply for a line of credit on their checking accounts.
- For system integration testing, tests are designed based on how the system uses an external microservice to check an account holder's credit score.
- For acceptance testing, tests are designed based on how the banker handles approving or declining a credit application.

Test Types

- 2. Non-Functional Testing:** Non-functional testing focuses on evaluating the quality attributes of the software, such as performance, usability, security, reliability, and maintainability. Testers assess how the software performs under different conditions and measure its compliance with non-functional requirements.
- ISO 25010 is a standard that defines a model for software product quality evaluation, focusing on characteristics like functionality, reliability, usability, and performance.



Non-functional testing may be performed at all test levels.

The term non-functional testing describes the tests required to measure characteristics of systems and software that can be quantified on a varying scale, such as response times for performance testing, or security.

Non-functional testing considers the external behaviour of the software and in most cases uses black-box test design techniques to accomplish that.

The following are examples of **non-functional tests**:

- For component testing, performance tests are designed to evaluate the number of CPU cycles required to perform a complex total interest calculation.
- For component integration testing, security tests are designed for buffer overflow vulnerabilities due to data passed from the user interface to the business logic.
- For system testing, portability tests are designed to check whether the presentation layer works on all supported browsers and mobile devices.
- For system integration testing, reliability tests are designed to evaluate system robustness if the credit score microservice fails to respond.
- For acceptance testing, usability tests are designed to evaluate the accessibility of the banker's credit processing interface for people with disabilities.

Test Types

The ISO/IEC 25010 standard provides a classification for these non-functional software quality characteristics, and here's an exploration of each one:

- **Performance Efficiency:** Performance efficiency is about how well the software uses resources to carry out its tasks within a defined context and constraints. It examines aspects such as response time, resource utilisation, and scalability.
- **Compatibility:** Compatibility refers to the ability of the software to operate with other products, systems, or components without modification. Compatibility testing ensures that the software can function across different environments, devices, browsers, or alongside other software applications.
- **Usability:** Usability testing focuses on how easily end-users can understand and use the software. It aims to ensure that the application is user-friendly and aligns with user expectations and needs.
- **Reliability:** Reliability measures the ability of the system to function without failure over a specified period under defined conditions. It includes factors like availability, fault tolerance, recoverability, and stability.

© The Knowledge Academy Ltd

ISO/IEC 25010 is an international standard that focuses on software and system quality models. It is part of the ISO/IEC 25000 series, also known as SQuaRE (Software Quality Requirements and Evaluation). The standard aims to define and categorise software quality attributes to aid organisations in assessing and improving their software products. ISO/IEC 25010 replaces the earlier ISO/IEC 9126 standard and offers a more comprehensive and detailed approach to software quality.

Test Types

- **Security:** Security testing is concerned with the protection of data and information within the system. It involves identifying potential weaknesses that might make the system vulnerable to attacks such as unauthorised access, data breaches, or other malicious activities.
- **Maintainability:** Maintainability testing looks at how easy it is to modify and update the software. This characteristic includes aspects like modularity, reusability, and readability of code.
- **Portability:** Portability involves the ease with which the software can be transferred from one environment to another. Portability testing ensures that the application can run in various environments without the need for significant changes.

Test Types

- 3. White-box Testing:** White-box testing is a software testing approach that examines the internal code and structure of an application. Testers create test cases based on the code logic to validate its correctness and ensure all paths are tested for robustness.
- Measures the extent to which a structure has been exercised by tests:
 - expressed as a %age.
 - 100% = full 'coverage'.
 - less indicates that more tests may be necessary.
 - usually measured by tools.

Takes an internal (white-box) view of the software

- 4. Black-box Testing:** Black-box testing is a software testing technique in which the internal workings and structure of the software application are not known to the tester. The testing is conducted based on the software's external specifications, requirements, and functionality, treating the software as a "black box" where only the inputs and outputs are visible.

White-Box testing may be performed at all test levels. Structural techniques are best used after specification-based techniques, in order to help measure the thoroughness of testing through assessment of coverage of a type of structure. Coverage is the extent that a structure has been exercised by a test suite, expressed as a percentage of the items being covered. If coverage is not 100%, then more tests may be designed to test those items that were missed to increase coverage.

At all test levels, but especially in component testing and component integration testing, tools can be used to measure the code coverage of elements, such as statements or decisions.

Structural testing may be based on the architecture of the system.

Such structural testing approaches can also be applied at system, system integration or acceptance testing levels (e.g., to business models or menu structures).

The following are examples of **white-box tests**:

- For component testing, tests are designed to achieve complete statement and decision coverage for all components that perform financial calculations.
- For component integration testing, tests are designed to exercise how each screen in the browser interface passes data to the next screen and to the business logic.
- For system testing, tests are designed to cover sequences of web pages that can occur during a credit line application.
- For system integration testing, tests are designed to exercise all possible inquiry types sent to the credit score microservice.
- For acceptance testing, tests are designed to cover all supported financial data file structures and value ranges for bank-to-bank transfers.

Confirmation Testing and Regression Testing

Confirmation Testing and Regression Testing

1. Confirmation Testing

- Confirmation testing is a specialised phase of software testing performed to confirm that an original defect that was identified has been successfully fixed. It's essential for ensuring that the changes made to correct the defect have not adversely impacted other parts of the system.
- The primary objective of confirmation testing is to verify that the defect has been properly resolved and that the corrected code is working as expected. It's a follow-up testing phase once the development team has modified the code to fix the identified defect.



Example:

Suppose you have an e-commerce website and users have reported an issue where the checkout button is not functioning properly. After the development team fixes this issue, a test is designed to specifically target this functionality.

Confirmation Testing and Regression Testing



2. Regression Testing

- Regression Testing is performed to ensure that changes or modifications in the software do not introduce new defects or negatively impact existing functionalities.
- Testers retest previously tested features, modules, or components to verify their continued correctness and stability after changes have been made.
- The primary goal of regression testing is to uncover any regression defects that may have occurred due to the modifications and to ensure that the overall system still functions as intended.

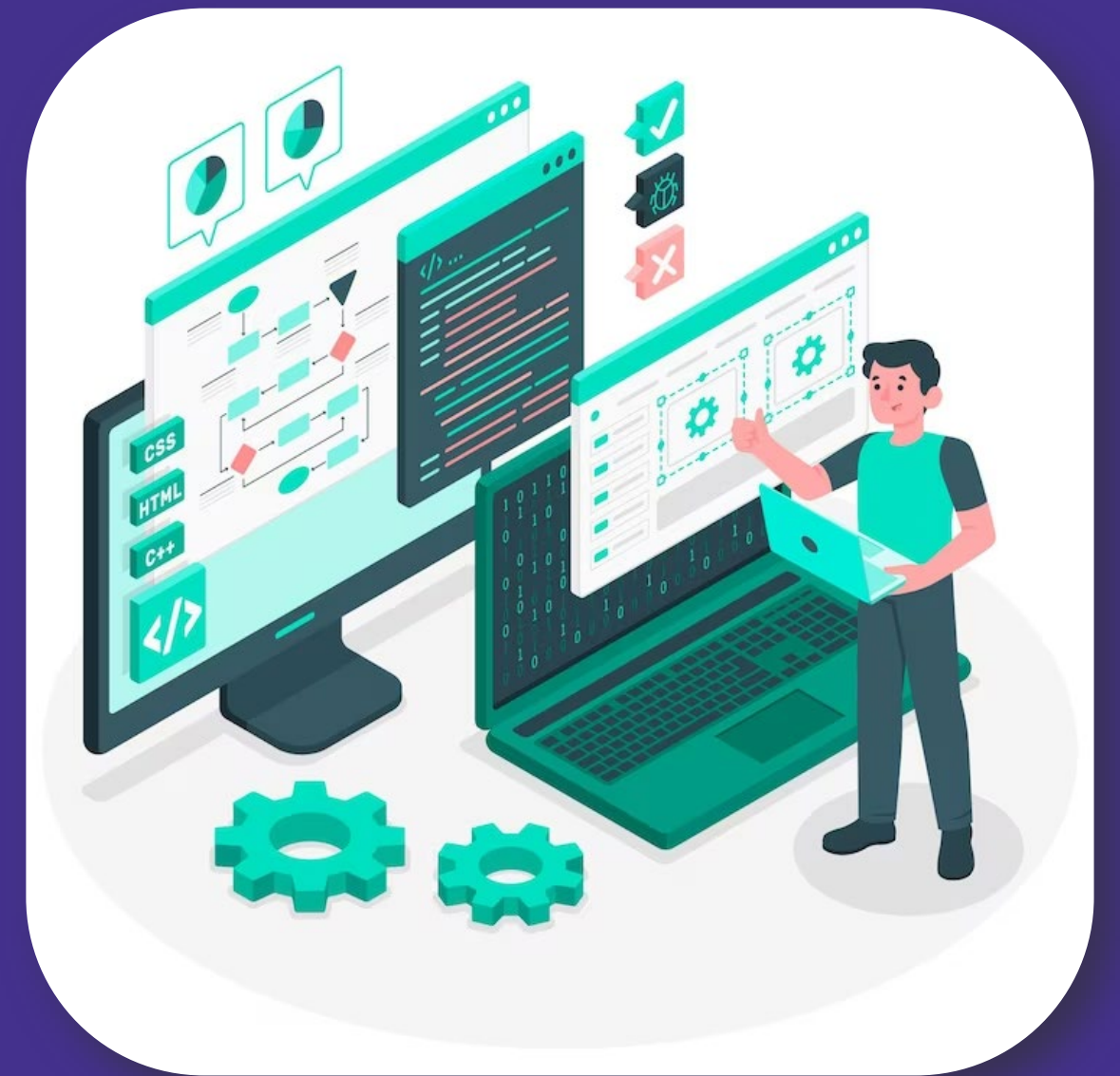
© The Knowledge Academy Ltd

Example:

Using the same e-commerce website example, suppose a new feature has been added to provide discount coupons to users. While this feature has been added, we need to make sure it hasn't broken any existing functionalities.

Both Confirmation Testing and Regression Testing are important aspects of the software testing lifecycle. While they are used in different contexts, they complement each other in ensuring software quality. Below is an explanation of each, along with examples.

Module 3: Maintenance Testing



Maintenance Testing

- Maintenance Testing refers to the testing activities that are carried out during the maintenance phase of the software development lifecycle.
- This phase involves making changes or enhancements to an existing software system to address defects, add new features, improve performance, or ensure compatibility with new environments or platforms.
- Maintenance testing involves not only verifying that the modified parts work correctly but also testing the changes themselves. It ensures that updates, fixes, or enhancements are thoroughly tested to identify any issues introduced and maintain the overall quality and reliability of the software.
- It involves retesting the affected areas, performing regression testing to ensure that existing functionalities are not affected by the changes, and conducting impact analysis to identify potential risks and dependencies associated with the modifications.

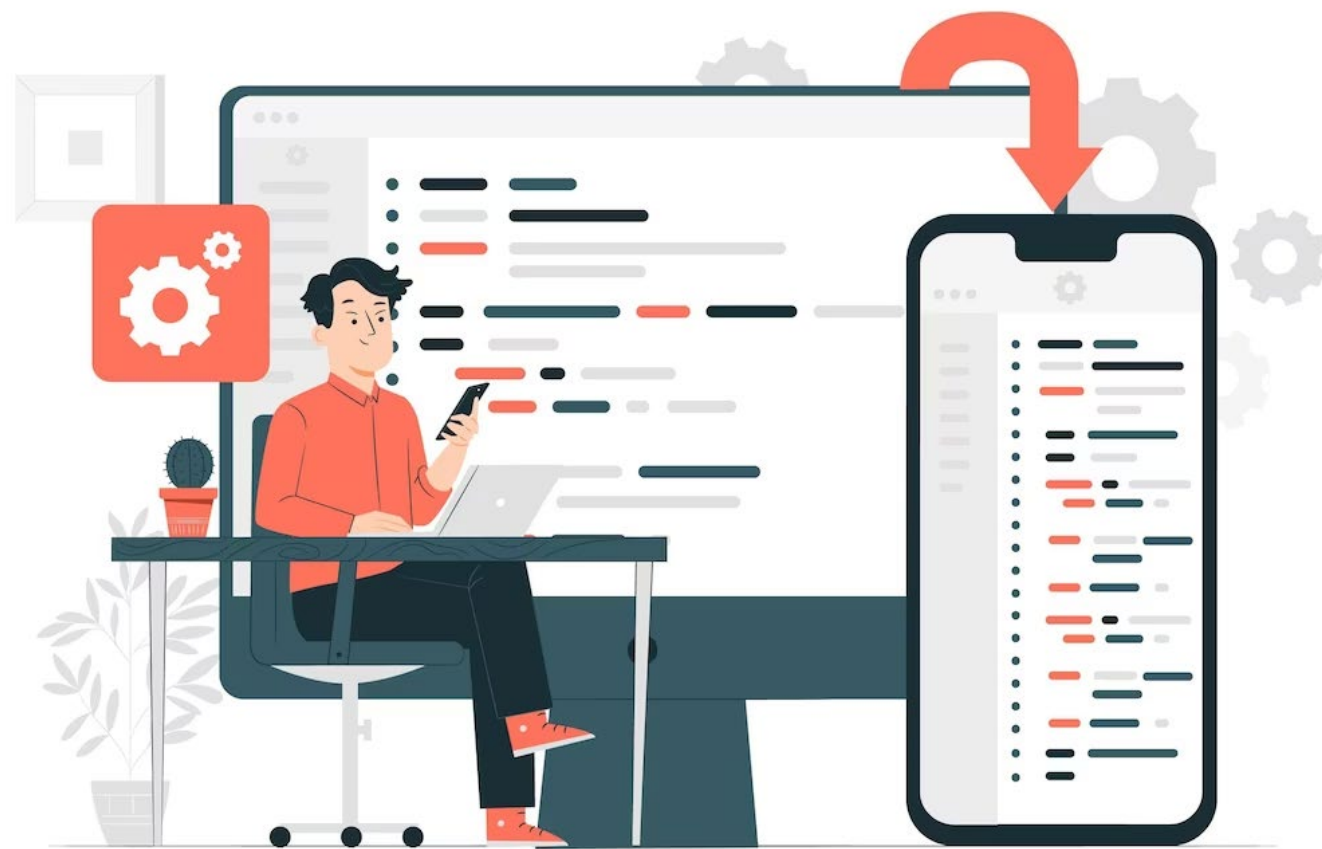
Benefits of Maintenance Testing

1. **Ensures Reliability:** Maintenance testing helps ensure that the software remains reliable after modifications. It detects issues that might have been introduced during the maintenance process, thereby increasing the overall reliability of the software.
2. **Preserves Software Integrity:** Over time, multiple changes can degrade the software's integrity. Maintenance testing ensures that the core functions of the software are not compromised, maintaining the software's structural and functional integrity.
3. **Validates Bug Fixes:** Maintenance Testing is critical for validating that bug fixes have been effectively implemented. It also ensures that the fixes work as intended without introducing new issues.
4. **Improves User Experience:** User experience is likely to degrade if new updates or patches introduce bugs or break existing functionalities. Maintenance Testing helps in preserving the quality of user experience by ensuring that all elements function as intended.
5. **Reduces Costs:** Catching issues early in the maintenance phase is generally less costly than dealing with them after they have affected the end-users. Maintenance testing can, therefore, be a cost-effective way to manage software quality.

Maintenance Testing

(Continued)

- Testers are equipped with the knowledge and skills to assess the impact of changes, identify test cases for retesting, and verify that the modified software meets the specified requirements and quality standards.
- For maintenance testing, it is important to ensure that testers have an understanding of testing activities in the maintenance phase and are capable of effectively validating and verifying software changes to ensure the ongoing reliability and usability of the software system.



Maintenance Testing

Background

triggered by:-

- **Modification to application**

- planned enhancements
- corrective changes
- emergency changes
- environment changes

- **Modifications to environment**

- operating system
- database
- hardware
- COTS package

- **Migration**

- from one platform to another
- from one database to another

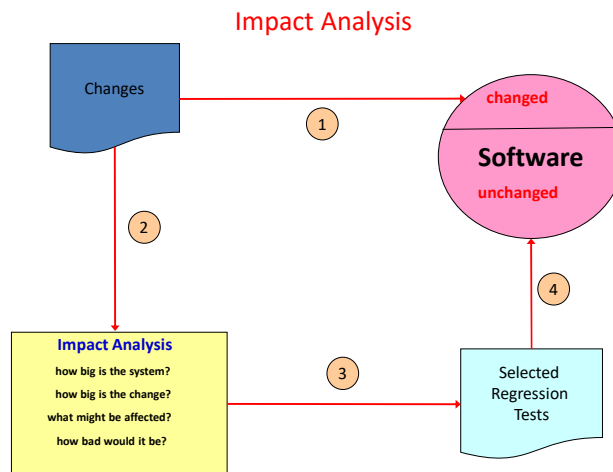
- **Retirement**

- archiving

Once deployed, a software system is often in service for years or decades. During this time the system, its configuration data, or its environment are often corrected, changed or extended. The planning of releases in advance is crucial for successful maintenance testing. A distinction has to be made between planned releases and hot fixes. Maintenance testing is done on an existing operational system, and is triggered by modifications, migration, or retirement of the software or system.

- Modifications include planned enhancement changes (e.g., release-based), corrective and emergency changes, and changes of environment, such as planned operating system or database upgrades, planned upgrade of Commercial-Off-The-Shelf software, or patches to correct newly exposed or discovered vulnerabilities of the operating system.
- Maintenance testing for migration (e.g., from one platform to another) should include operational tests of the new environment as well as of the changed software. Migration testing (conversion testing) is also needed when data from another application will be migrated into the system being maintained.
- Maintenance testing for the retirement of a system may include the testing of data migration or archiving if long data-retention periods are required.

Maintenance Testing



© The Knowledge Academy Ltd

In addition to testing what has been changed, maintenance testing includes extensive regression testing to parts of the system that have not been changed.

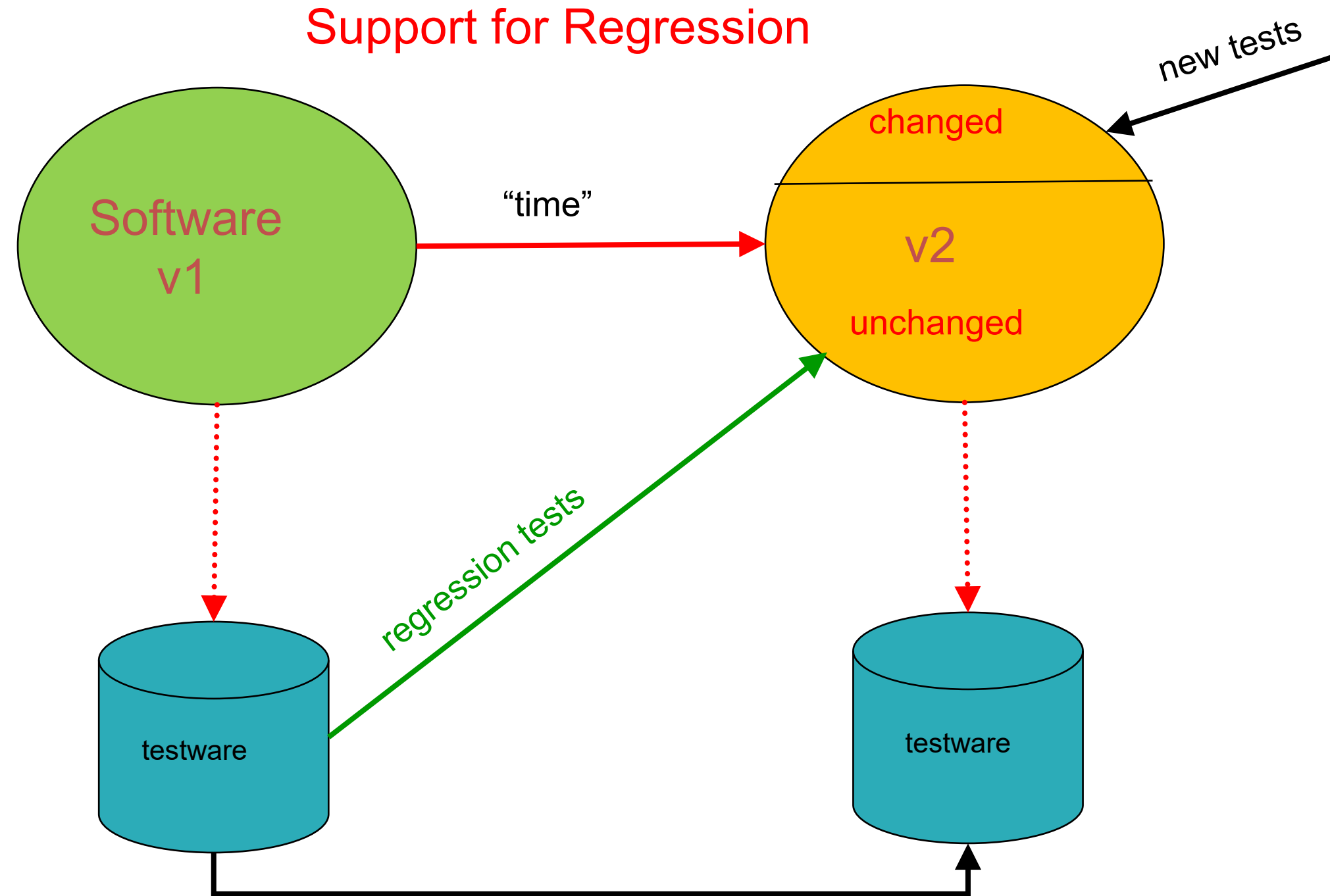
The scope of maintenance testing is related to the risk of the change, the size of the existing system and to the size of the change.

Depending on the changes, maintenance testing may be done at any or all test levels and for any or all test types. Determining how the existing system may be affected by changes is called impact analysis, and is used to help decide how much regression testing to do. The impact analysis may be used to determine the regression test suite.

IA may be used to decide if the fix is worthwhile.

Maintenance testing can be difficult if specifications are out of date or missing, or testers with domain knowledge are not available.

Maintenance Testing



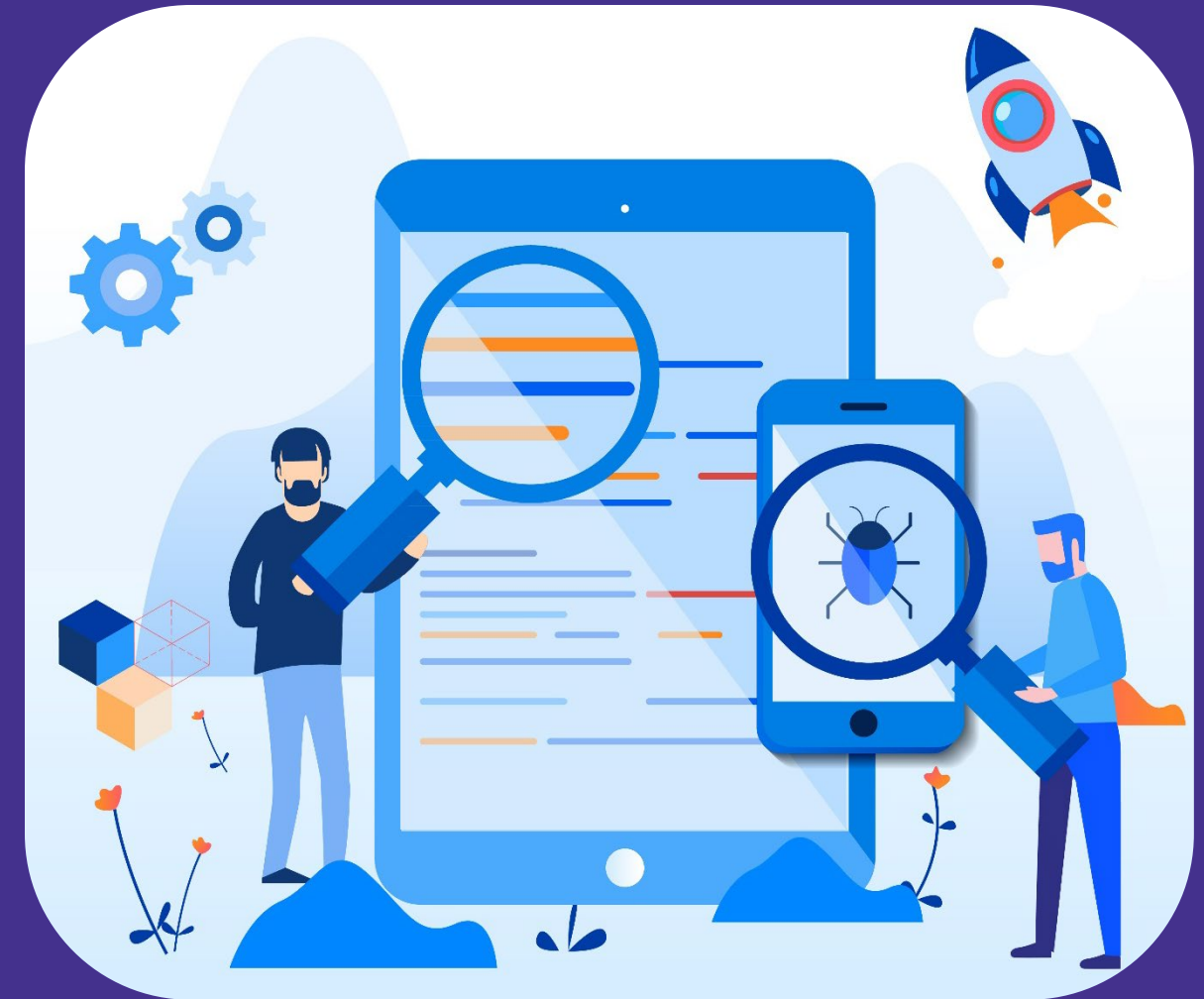
Domain 3:

Static Testing

Outline of Domain 3

- *Module 1: Static Testing Basics*
- *Module 2: Feedback and Review Process*

Module 1: Static Testing Basics



Static Testing Basics

Static

- Finds causes of failure
- Finds them early
- Defect cheap to fix

Dynamic

- Finds the failures themselves
- Finds them late
- Defect expensive to fix

The techniques are complementary

Static testing, as the name suggests, evaluates the software without actually executing the code. Instead, it involves reviewing the software artefacts like requirement documents, design documents, code, and more.

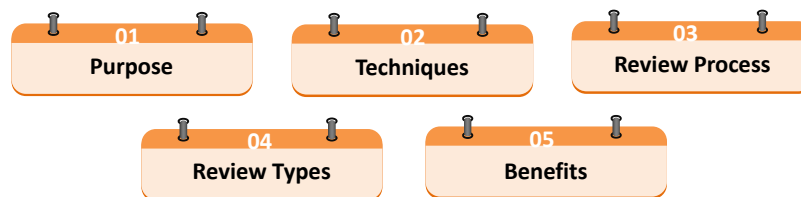
- **Finds causes of failure:** By reviewing the software artefacts, static testing can identify issues that might lead to failures before the code is run. For instance, during a code review, an engineer might spot a potential null pointer exception which, if unaddressed, would cause a crash during execution.
- **Finds them early:** Since static testing is done during the early phases of the software development lifecycle, such as during the requirement gathering and design stages, it allows teams to identify and rectify defects at the outset.
- **Defect cheap to fix:** Addressing defects at this stage is more cost-effective. It's cheaper to rectify an error in the requirement or design phase than after the software has been deployed to end-users.

Dynamic testing, on the other hand, involves the actual execution of the software. This could be either manually, by testers clicking through an application, or automatically, using testing scripts and tools.

- **Finds the failures themselves:** Dynamic testing uncovers the actual defects when the software is run. For instance, if there's a bug in the code that causes a system crash when a particular action is taken, dynamic testing will expose this by replicating the action.
- **Finds them late:** Typically, dynamic testing is conducted after the software is coded and integrated, making it a later phase activity. This might be during unit testing, system testing, or even acceptance testing stages.
- **Defect expensive to fix:** Discovering defects at this stage can be costly. Once the software is fully developed, making changes can be complex, requiring more time, resources, and potentially leading to further issues. Moreover, if defects are found after release, it can lead to reputation damage and customer dissatisfaction, further escalating the costs.

Static Testing Basics

- Static testing basics refer to the fundamental concepts and techniques used in static testing.
- Static testing is a software testing technique that involves examining the software documentation and source code without actually executing the program.
- Its purpose is to find defects early in the software development life cycle, thereby improving the quality of the final product.
- *Here are some key points regarding static testing basics:*



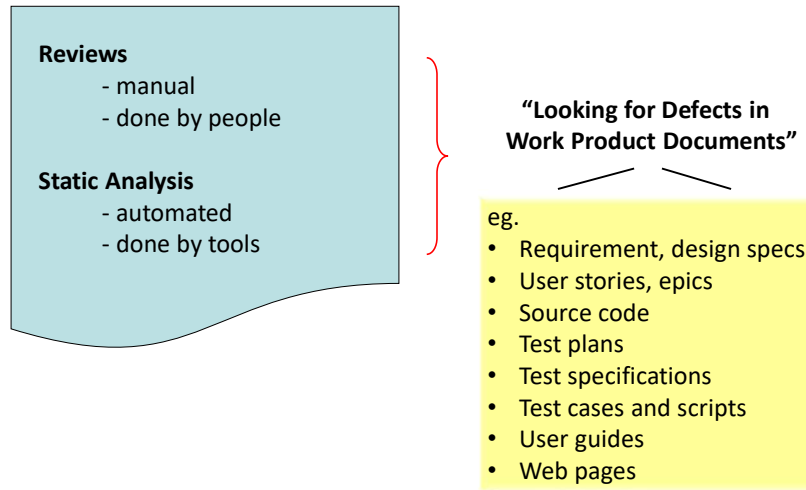
© The Knowledge Academy Ltd

1. **Purpose:** Static testing aims to identify defects, such as errors, omissions, ambiguities, and inconsistencies, in software artifacts before the software is executed.
2. **Techniques:** Static testing techniques include manual and automated reviews, inspections, walkthroughs, and code analysis. These techniques involve a systematic examination of software documentation, requirements, designs, and source code.
3. **Review Process:** Reviews involve a group of individuals, such as developers, testers, and stakeholders, analysing the software artifacts to find defects. The review process typically follows a defined set of steps, including planning, preparation, meeting, and follow-up.
4. **Review Types:** Different types of reviews are conducted during static testing, such as informal reviews, walkthroughs, technical reviews, and inspections. Each review type has its own objectives and level of formality.
5. **Benefits:** Static testing offers several benefits, including early defect detection, improved software quality, cost-effectiveness, knowledge sharing among team members, and reduced rework and maintenance efforts.

Work Products Examinable by Static Testing

- Static testing involves examining various work products during the software development life cycle.
- These work products include requirements documents, design specifications, test plans and test cases, code and scripts, user manuals and documentation, configuration files, interface specifications, database schema, and project documentation.
- By applying static testing techniques such as reviews, inspections, and automated analysis to these work products, software development teams can identify defects, inconsistencies, and potential risks at an early stage.
- This helps ensure the completeness, correctness, and quality of the software, leading to improved overall product quality and reduced rework efforts. Static testing plays a crucial role in enhancing the effectiveness and efficiency of the software development process.

Work Products Examinable by Static Testing



Lecturer Notes

Almost any work product can be examined using static testing (reviews and/or static analysis), for example:

- Specifications, including business requirements, functional requirements, and security requirements
- Epics, user stories, and acceptance criteria
- Architecture and design specifications
- Code
- Testware, including test plans, test cases, test procedures, and automated test scripts
- User guides
- Web pages
- Contracts, project plans, schedules, and budgets
- Models, such as activity diagrams, which may be used for Model-Based testing

Value of Static Testing

- Static testing provides significant value to the software development process. By examining software artifacts without actually executing the program, static testing helps identify defects, errors, and inconsistencies early on, reducing the cost and effort associated with finding and fixing issues in later stages.
- It promotes early defect detection, allowing for timely corrections and preventing the propagation of defects to subsequent phases of development. Static testing techniques, such as reviews and inspections, foster collaboration and knowledge sharing among team members, improving communication and understanding of the software.
- Additionally, static analysis tools automate the process, enabling comprehensive code analysis and identification of potential vulnerabilities, performance bottlenecks, and adherence to coding standards.
- The value of static testing lies in its ability to improve software quality, minimise rework, enhance productivity, and ultimately deliver a higher-quality product to end-users.

Differences between Static Testing and Dynamic Testing

Static Testing	Dynamic Testing
Examines software artefacts without executing the program.	Involves executing the software to observe its behaviour.
Identifies defects, errors, and inconsistencies early in the development life cycle.	Focuses on validating the functional and non-functional aspects of the software.
Examples of techniques include reviews, inspections, walkthroughs, and static code analysis.	Examples of techniques include functional testing, performance testing, security testing, and usability testing.
Typically performed by analysing documents, source code, and other work products.	Requires the software to be executed using appropriate test data and test cases.

Differences between Static Testing and Dynamic Testing

- ***In comparison to dynamic testing, typical defects that are cheaper and easier to find and fix through static testing involve:***
 - ✓ Requirement defects (e.g., inconsistencies, contradictions, ambiguities, omissions, inaccuracies, and redundancies).
 - ✓ Design defects (e.g., database structures or inefficient algorithms, high coupling, low cohesion).
 - ✓ Coding defects (e.g., variables with undefined values, unreachable code, variables that are declared but never used, duplicate code).
 - ✓ Deviations from standards (e.g., lack of compliance with coding standards).
 - ✓ Incorrect interface specifications (e.g., different units of measurement utilised by the calling system than by the called system).
 - ✓ Security vulnerabilities (e.g., susceptibility to buffer overflows).
 - ✓ Gaps or inaccuracies in test basis coverage or traceability (e.g., missing tests for an acceptance criterion).

Module 2: Feedback and Review Process



Feedback and Review Process

Benefits of Early and Frequent Stakeholder Feedback

- Early and frequent stakeholder feedback plays a crucial role in the software development process.
- By seeking feedback from stakeholders at the early stages of the project, such as during requirements gathering and design phases, testers can ensure that the software aligns with their expectations and requirements.
- This feedback helps in identifying potential misunderstandings, gaps, or inconsistencies in the initial project deliverables. Incorporating stakeholder feedback early on minimises the risk of rework and prevents costly changes later in the development cycle.
- Moreover, frequent stakeholder feedback throughout the development process enables testers to continuously validate the software against stakeholder expectations.

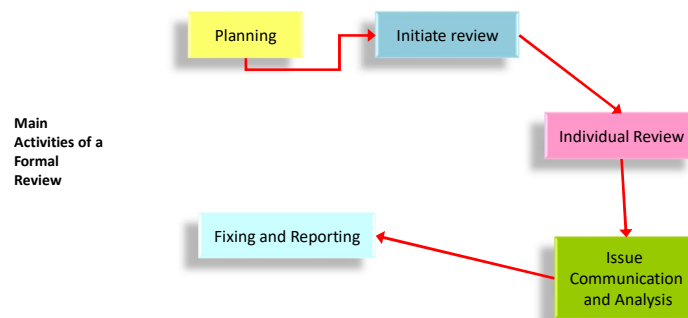
Feedback and Review Process

(Continued)

- Regular feedback loops provide opportunities to address concerns, make necessary adjustments, and align the software with evolving stakeholder needs.
- Additionally, early and frequent stakeholder feedback helps in identifying potential usability issues, functional gaps, or performance concerns before the software is deployed or released to end-users. This feedback allows for timely course corrections, ensuring that the software meets the desired quality standards and user satisfaction.
- By valuing early and frequent stakeholder feedback, testers can enhance customer satisfaction, improve the quality of the software, and increase the overall success of the project.

Review Process Activities

- The review process encompasses several activities that are conducted to effectively perform reviews of software artifacts. These activities ensure that the review process is well-planned, organised, and executed.



© The Knowledge Academy Ltd

Planning: Defining the scope, which includes the purpose of the review, what documents or parts of documents to review, and the quality characteristics to be evaluated ☐ Estimating effort and timeframe ☐ Identifying review characteristics such as the review type with roles, activities, and checklists ☐ Selecting the people to participate in the review and allocating roles ☐ Defining the entry and exit criteria for more formal review types (e.g., inspections) ☐ Checking that entry criteria are met (for more formal review types)

Initiate review: Distributing the work product (physically or by electronic means) and other material, such as issue log forms, checklists, and related work products ☐ Explaining the scope, objectives, process, roles, and work products to the participants ☐ Answering any questions that participants may have about the review

Individual review: (i.e., individual preparation) ☐ Reviewing all or part of the work product ☐ Noting potential defects, recommendations, and questions

Issue communication and analysis: Communicating identified potential defects (e.g., in a review meeting) ☐ Analysing potential defects, assigning ownership and status to them ☐ Evaluating and documenting quality characteristics ☐ Evaluating the review findings against the exit criteria to make a review decision (reject; major changes needed; accept, possibly with minor changes)

Fixing and reporting: Creating defect reports for those findings that require changes ☐ Fixing defects found (typically done by the author) in the work product reviewed ☐ Communicating defects to the appropriate person or team (when found in a work product related to the work product reviewed) ☐ Recording updated status of defects (in formal reviews), potentially including the agreement of the comment originator ☐ Gathering metrics (for more formal review types) ☐ Checking that exit criteria are met (for more formal review types) ☐ Accepting the work product when the exit criteria are reached

Roles and Responsibilities in Reviews

Author

-produces the document

Manager

- determines the review type
- plans the review
- allocates time
- determines if review objectives met

Review Leader

- Who, when, where

Scribe/Recorder

-documents issues,
problems, open points

Facilitator

- Ensure effective running
- mediates
- Success depends on them

Reviewer/Inspector

- checks documents
- identify defects

Roles and Responsibilities in Reviews

- In the ISTQB Certified Tester Foundation, reviews involve multiple stakeholders who assume specific roles and responsibilities. The principal roles and their responsibilities in a review process are as follows:
 1. **Manager:** *The manager oversees the review process and ensures that it aligns with project objectives and timelines. Their responsibilities include allocating resources, monitoring progress, and facilitating the review activities. The manager is responsible for setting up a conducive environment for effective reviews and promoting collaboration among team members.*
 2. **Author:** *The author is the individual or team responsible for creating the software artifact being reviewed, such as requirements documents, design specifications, or code. Their responsibility is to provide a clear and complete representation of the intended content, addressing any ambiguities and ensuring that the artifact is ready for review. The author should actively participate in the review process and be open to receiving feedback and suggestions for improvement.*

Roles and Responsibilities in Reviews

3. **Moderator:** *The moderator, also known as the facilitator or chairperson, plays a vital role in managing the review meetings. Their responsibilities include guiding the review process, maintaining focus, and ensuring that all relevant aspects of the artifact are thoroughly discussed. The moderator promotes effective communication, encourages participation from all stakeholders, and manages any conflicts or diverging opinions that may arise during the review.*
4. **Scribe:** *The scribe is responsible for documenting the discussions, observations, and identified issues during the review meetings. Their role is to capture the review findings accurately and ensure that they are properly recorded. The scribe may also assist in tracking the identified defects, maintaining a log of actions, and documenting any recommendations or decisions made during the review.*
5. **Reviewer:** *Reviewers are individuals who actively analyse and evaluate the software artifact under review. Their responsibility is to carefully examine the artifact, identify defects, and provide constructive feedback. Reviewers should possess a good understanding of the review objectives, have relevant expertise, and apply a critical mindset to identify potential issues, inconsistencies, or improvements. Reviewers contribute to the overall quality of the software by sharing their insights and suggestions.*

Review Types

- In the ISTQB Certified Tester Foundation, there are several types of reviews that can be conducted as part of the software testing and quality assurance process. These review types serve different purposes and have specific objectives.
- ***The common review types include:***

Informal Review

01

Walkthrough

02

Technical Review

03

Inspection

04

Review Types

1. Informal Review

- Informal reviews are ad-hoc and spontaneous discussions or meetings where software artifacts are informally reviewed by a small group of stakeholders. These reviews are typically less structured and aim to gather quick feedback and identify any immediate concerns or issues. Informal reviews are often conducted during the early stages of development and provide valuable insights to improve the quality of the software.

2. Walkthrough

- A walkthrough is a review technique where the author or designer leads a structured presentation of the software artifact to a group of stakeholders. The purpose of a walkthrough is to obtain feedback, clarify requirements, and gather suggestions for improvement. The participants actively engage in discussions, ask questions, and provide feedback based on their expertise and understanding. Walkthroughs promote collaboration and knowledge sharing among stakeholders.

Review Types

3. Technical Review

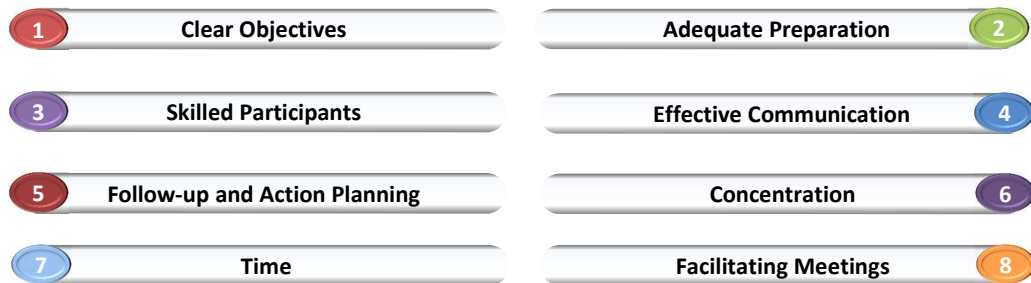
- Technical reviews, also known as peer reviews or inspections, involve a formal examination of software artifacts by a group of technical experts. The objective of technical reviews is to identify defects, validate the correctness of the artifact, and ensure compliance with technical standards and best practices. These reviews focus on identifying issues related to design, coding, and implementation, and are conducted with a structured and rigorous approach.

4. Inspection

- Inspections are formal, highly structured reviews that involve a rigorous examination of software artifacts to identify defects and improve the quality of the software. Inspections typically follow a predefined process, involving multiple stakeholders and specific roles such as moderator, author, and reviewer. The objective is to identify defects systematically and collaboratively, ensuring that the software meets the specified requirements and standards.

Success Factors for Reviews

- Success factors for reviews are critical to ensuring effective and efficient review processes. These factors significantly contribute to the overall quality of the software artifacts being reviewed and the success of the testing efforts.
- *Some important success factors to consider are:*



© The Knowledge Academy Ltd

- 1. Clear Objectives:** Clearly defined objectives are essential for a successful review. Having specific goals for each review helps focus the process and guides participants towards the desired outcomes. Whether it is to identify defects, validate requirements, or ensure compliance with standards, clear objectives provide a framework for the review and help stakeholders understand what needs to be achieved.
- 2. Adequate Preparation:** Adequate preparation is vital for productive reviews. Reviewers should thoroughly familiarise themselves with the software artifacts before the review session. This includes studying the documentation, understanding the context, and preparing relevant checklists or templates. Adequate preparation ensures that reviewers are well-equipped to analyse the artifacts effectively and provide valuable feedback.
- 3. Skilled Participants:** The expertise and skills of the participants greatly influence the success of a review. Having knowledgeable and skilled reviewers who possess domain-specific knowledge, technical expertise, and an understanding of the review objectives is crucial. Skilled participants can identify potential defects, offer valuable insights, and contribute to meaningful discussions, ultimately enhancing the quality of the review process.
- 4. Effective Communication:** Effective communication is a fundamental success factor for reviews. Participants should be encouraged to express their opinions, ask questions, and provide constructive feedback. Open and transparent communication fosters collaboration, promotes a shared understanding among stakeholders, and enables the identification and resolution of issues. Active listening and respectful communication are essential for successful reviews.
- 5. Follow-up and Action Planning:** Review activities should be followed by appropriate actions and follow-up steps. It is essential to document identified defects, recommendations, and decisions made during the review. Assigning responsibilities for addressing the identified issues and defining a clear action plan ensures that the feedback provided during the review is acted upon promptly and effectively.
- 6. Concentration:** Conducting reviews on small chunks, so that reviewers do not lose concentration during an individual review and/or the review meeting
- 7. Time:** In Success Factors for Reviews, providing adequate time for participants to prepare ensures a fair and thorough review process. It allows individuals to gather feedback, reflect on their performance, and formulate meaningful contributions, leading to more valuable and constructive reviews.
- 8. Facilitating Meetings:** Facilitating meetings in SuccessFactors for reviews involves using the platform to schedule, conduct, and manage performance review sessions. Users can access review forms and relevant employee data to guide discussions during the meetings.

Domain 4: Test Analysis and Design

Outline of Domain 4

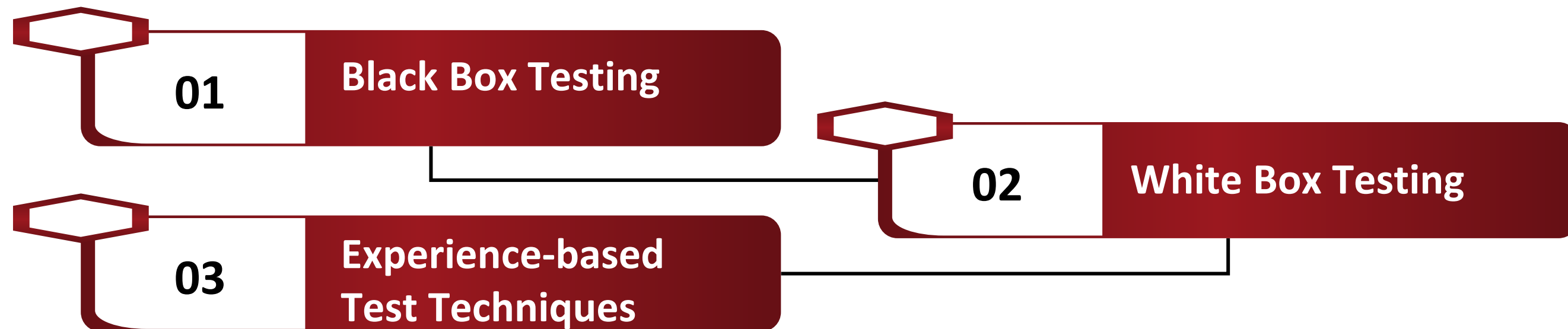
- *Module 1: Test Techniques Overview*
- *Module 2: Black-box Test Techniques*
- *Module 3: White-box Test Techniques*
- *Module 4: Experience-based Test Techniques*
- *Module 5: Collaboration-based Test Approaches*

Module 1: Test Techniques Overview



Test Techniques Overview

- Test techniques are systematic approaches used to design and analysis tests to uncover defects or verify the conformance of a system or software product to its requirements.
- There are various test techniques available, each with its strengths and areas of applicability.
- *The following is an overview of some commonly used test techniques:*



Test Techniques Overview

1. Black Box Testing

- This technique focuses on testing the functionality of a system without any knowledge of its internal structure or implementation details.
- Test cases are designed based on the system's specifications, requirements, or expected behaviour.
- Black box testing can be performed using techniques such as equivalence partitioning, boundary value analysis, decision table testing, and state transition testing.

Test Techniques Overview

2. White Box Testing

- Also known as structural testing, white box testing involves testing the internal structure, design, and implementation of a system or software product.
- Test cases are designed based on the knowledge of the system's code and internal logic.
- Techniques such as statement coverage, branch coverage, and path coverage are commonly used in white box testing to ensure that all code paths and branches are tested.

Test Techniques Overview

3. Experience-based test techniques

- Experience-based test techniques, also known as heuristic or informal techniques, rely on the tester's knowledge, skills, and experience to identify potential defects in a software application.
- These techniques are not based on formal specifications or predefined test cases but rather on the tester's intuition and past experiences.
- These experience-based techniques are highly flexible and can complement other formal testing techniques.
- They rely on the tester's expertise and intuition, making them effective in discovering defects that might be missed by formal test cases or specifications alone.

Module 2: Black-Box Test Techniques



Equivalence Partitioning

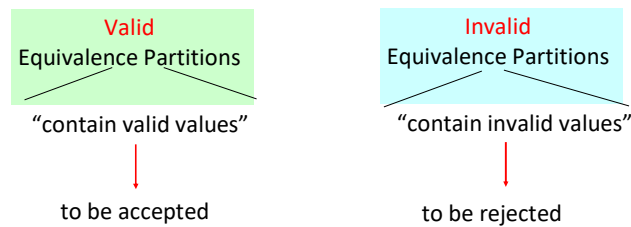
- Equivalence Partitioning helps to reduce the number of test cases while still providing adequate coverage of the system under test.
- The basic idea behind Equivalence Partitioning is to divide the input data into different partitions or classes based on the assumption that if a particular input value belongs to a specific class, the system's behaviour will be the same for any other input value within that class.
- In other words, it assumes that if one input within a partition works correctly, all other inputs within that partition will also work correctly.

Equivalence Partitioning is a software testing technique that plays a pivotal role in enhancing test efficiency and coverage. By dividing the input data into equivalent classes or partitions, this method allows testers to reduce the number of test cases while still maintaining a high level of effectiveness in identifying defects. The core idea is that all data points in a partition are considered "equivalent" in terms of the software's behavior, so testing just one or a few data points from each partition can provide confidence that the software will handle all other data points in that partition correctly. This is especially valuable in systems with large input domains, as it makes the testing process more manageable and less time-consuming.

Equivalence Partitioning

Characteristics

- Input/output values are divided in groups that are expected to exhibit the same behaviour when processed
- Such groups are referred to as Equivalence Partitions (or Classes)



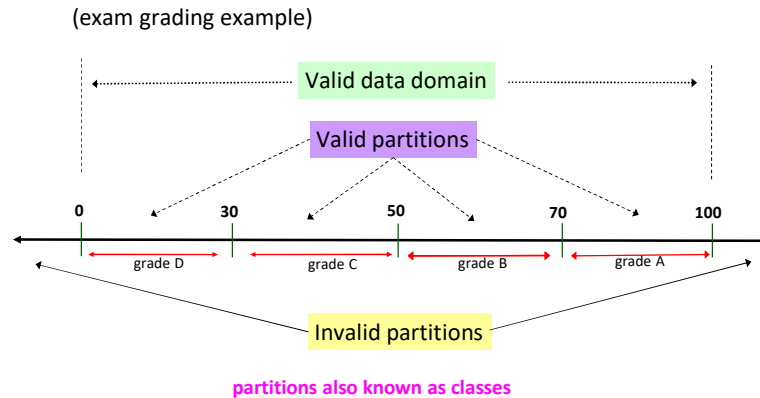
Equivalence partitions (or classes) can be found for both valid data, i.e. values that should be accepted and invalid data, i.e. values that should be rejected. Partitions can also be identified for outputs, internal values, time-related values (e.g., before or after an event) and for interface parameters (e.g., integrated components being tested during integration testing).

Tests can be designed to cover all valid and invalid partitions. **Equivalence partitioning is applicable at all levels of testing.**

Equivalence partitioning can be used to achieve input and output coverage goals. It can be applied to human input, input via interfaces to a system, or interface parameters in integration testing.

Equivalence Partitioning

Terminology



© The Knowledge Academy Ltd

The component is passed an exam mark (out of 100) from which it generates a grade as following business rules:-

- Greater than or equal to 70 - A
- Greater than or equal to 50 but less than 70 - B
- Greater than or equal to 30 but less than 50 - C
- Less than 30 - D

Where the exam mark is outside its expected range an error message should be generated:-

- less than 0 - mark negative
- greater than 100 - impossible

All inputs are passed as integers

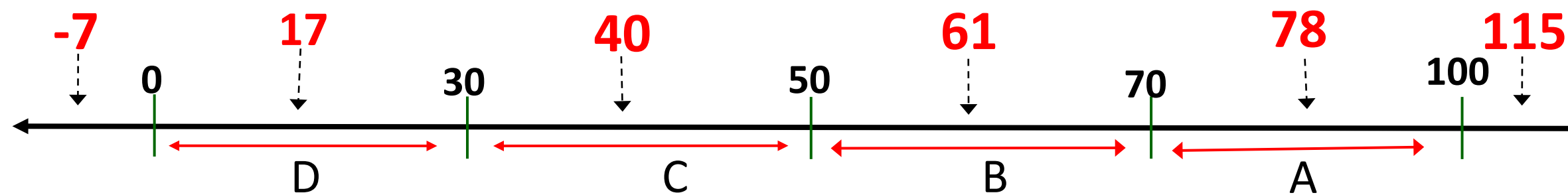
The valid range of values that a data item may take from a business perspective is known as the valid data domain. It is often the case that the business processing rule for the data item changes depending on the value it contains.

This technique is designed to test if the correct rules are applied to the right values.

Equivalence Partitioning

Principle

Only one test value needed from each partition to prove the rule



Possible documentation

	TC1	TC2	TC3	TC4	TC5	TC6
Input	-7	17	40	61	78	115
Expected result	Error message	Grade D	Grade C	Grade B	Grade A	Error message

- Less obvious invalid partitions are
- non integers input
 - alphabetic inputs
 - etc.
- tests also needed!

Equivalence Partitioning

The following are the steps involved in applying Equivalence Partitioning:

- 1. Identify the input variables or conditions:** Determine the input variables or conditions that the system takes as input. For example, if you're testing a login functionality, the input variables could be username and password.
- 2. Define the input partitions:** Divide the input variables into different partitions or classes based on the specific characteristics of the input. Each partition should have similar behaviour or requirements. For example, for the username field, you might have partitions like "valid usernames," "invalid usernames with special characters," and "invalid usernames with excessive length."
- 3. Select test cases:** From each partition, select representative test cases that cover the different scenarios within that partition. The goal is to select test cases that are likely to uncover defects if they exist. For example, in the valid usernames partition, you might choose test cases like normal alphanumeric usernames, usernames with uppercase letters, and usernames with lowercase letters.

Equivalence Partitioning

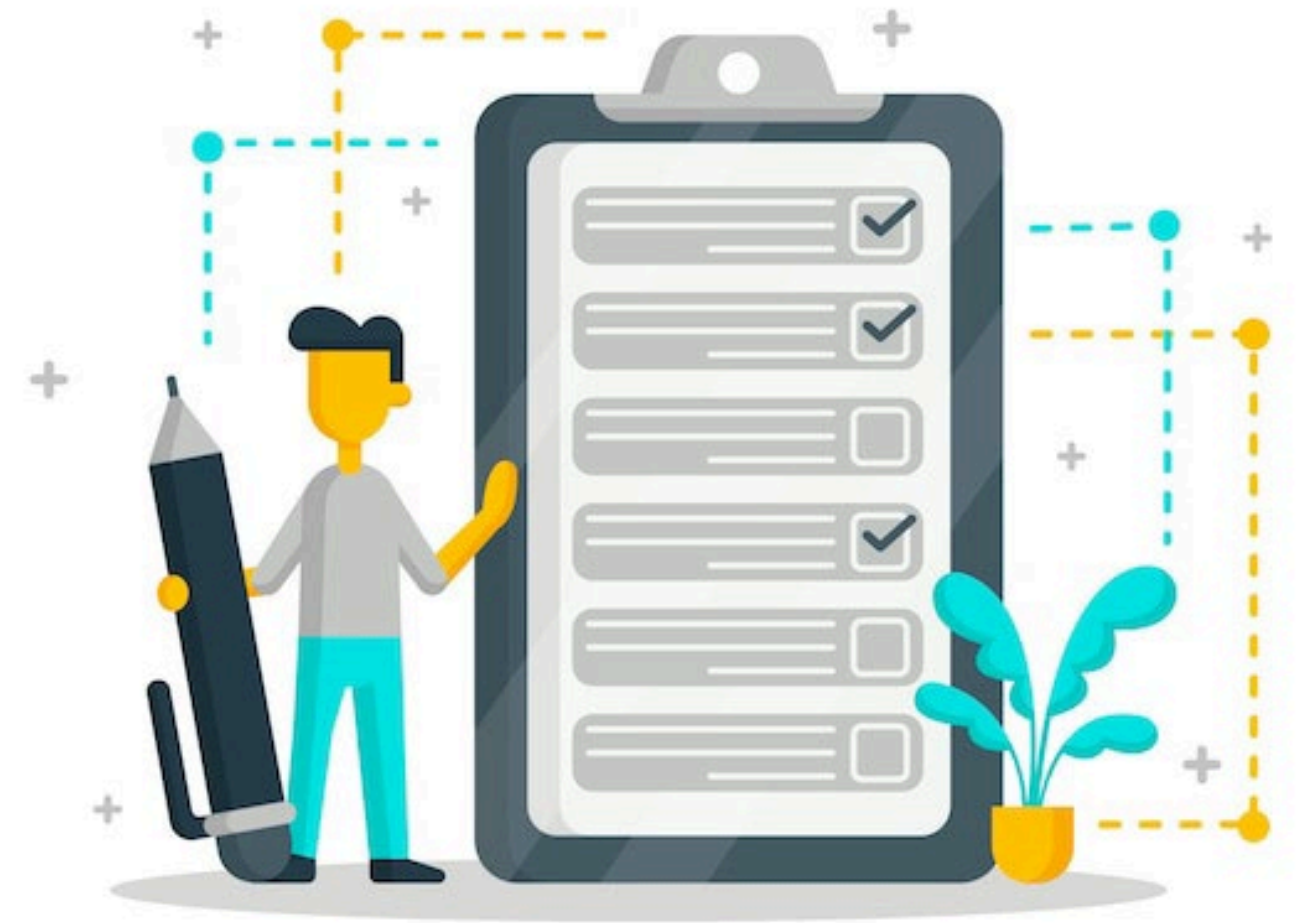
4. **Execute the test cases:** Execute the selected test cases and observe the system's behaviour. If the system behaves as expected for a specific partition, you can assume that other inputs within that partition will also produce the expected results.
5. **Repeat for other partitions:** Repeat the process for each input variable and its corresponding partitions. Identify new partitions as needed and select representative test cases from each partition. The goal of Equivalence Partitioning is to minimise the number of test cases required while still achieving reasonable coverage.

Exercise

To dive deeper into practical application, refer to the corresponding exercise in the document for hands-on exploration and analysis.

Exercise 1: Gardening App Temperature-Based Plant Suggestion

Exercise 2: Online Course Registration System

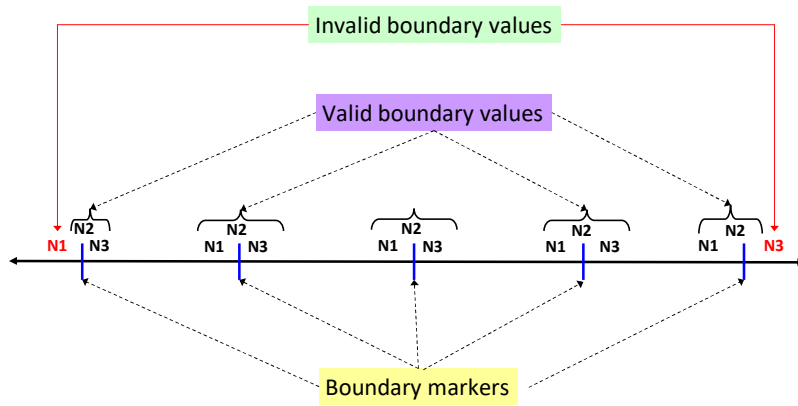


Boundary Value Analysis

- Boundary Value Analysis helps in selecting test cases by focusing on the boundary values of input variables or conditions, as these values often tend to be more error-prone and can reveal defects.
- Combining Boundary Value Analysis (BVA) with Equivalence Partitioning requires identifying critical boundary values of equivalence partitions. Testing these values ensures the system functions correctly and robustly under challenging conditions, providing comprehensive coverage and software reliability.
- The underlying concept of Boundary Value Analysis is based on the assumption that errors or defects are more likely to occur at the boundaries of input ranges rather than within the range itself.
- By testing the boundary values, you can increase the probability of detecting defects that might be missed by testing typical or middle-range values.

Boundary Value Analysis

Terminology



© The Knowledge Academy Ltd

Valid Boundary Value

- Value identified by boundary value analysis
- In a valid partition

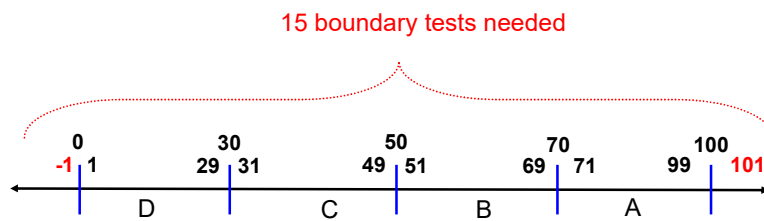
Invalid Boundary Value

- Value identified by boundary value analysis
- In an invalid partition

Boundary Value Analysis

Characteristics

- ✓ Proves that the points at which the rule changes have been encoded are correct.
- ✓ For each boundary marker the 3 boundary values might be tested.



The main purpose of this technique is to check that the points on the domain at which the processing rules should change have been encoded correctly.

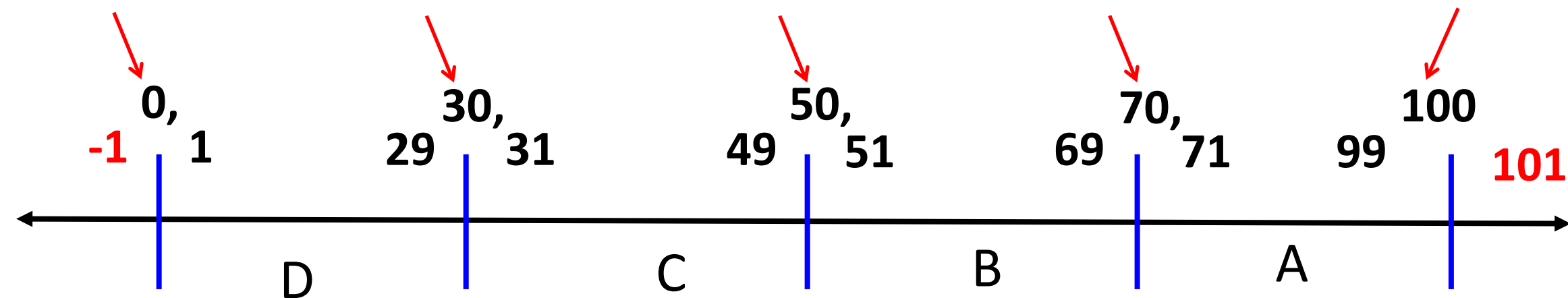
It is a well known fact developers commonly get this wrong, often due to misinterpretation of 'less than' 'equal to' and 'greater than' in specifications especially if < and > are used rather than words but also due just to the mistyping of values when coding under pressure.

When designing test cases, a test for each boundary value is chosen.

Boundary Value Analysis

Location of Boundary Values

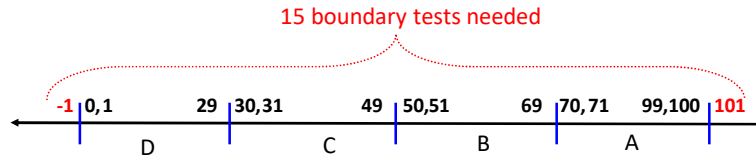
- greater than or equal to 70 - A
- greater than or equal to 50 but less than 70 - B
- greater than or equal to 30 but less than 50 - C
- less than 30 - D
- less than 0 - error negative
- greater than 100 - error impossible



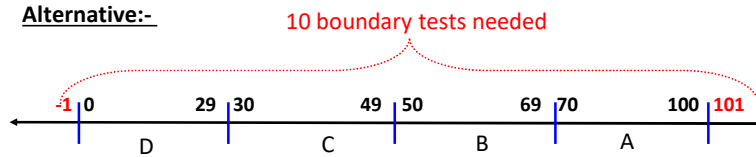
Boundary Value Analysis

How many Boundary Values?

As seen:



Alternative:-



(use only the minimum and maximum values of each partition)

There are two schools of thought on how many boundary tests are needed (3 or 2). The other one is that only the minimum and the maximum values of each partition need to be tested which significantly reduces the number of tests required overall.

Boundary Value Analysis

(Continued)

- The following are the steps involved in applying Boundary Value Analysis:

1. Identify the input variables or conditions

- Determine the input variables or conditions that the system takes as input. For example, if you're testing a form that accepts an age input, the input variable could be "age."

2. Determine the boundaries

- Identify the boundaries for each input variable or condition. The boundaries typically include the minimum and maximum values allowed by the system.
- For example, if the minimum age is 18 and the maximum age is 65, the boundaries would be 18 and 65. The values used to test the boundaries would be 17, 18, 19, 64, 65, and 66.

Boundary Value Analysis

3. Select test cases

- Select test cases that focus on the boundaries. The test cases should include values that are just below, exactly on, and just above the boundaries.
- For example, for the age input, you might choose test cases like 17 (just below the minimum), 18 (exactly on the minimum), 19 (just above the minimum), 64 (just below the maximum), 65 (exactly on the maximum), and 66 (just above the maximum).

4. Execute the test cases

- Execute the selected test cases and observe the system's behaviour. Pay close attention to how the system handles the boundary values. If the system behaves correctly for the boundary values, it is likely to handle the values within the range correctly as well.

Boundary Value Analysis

5. Consider additional test cases

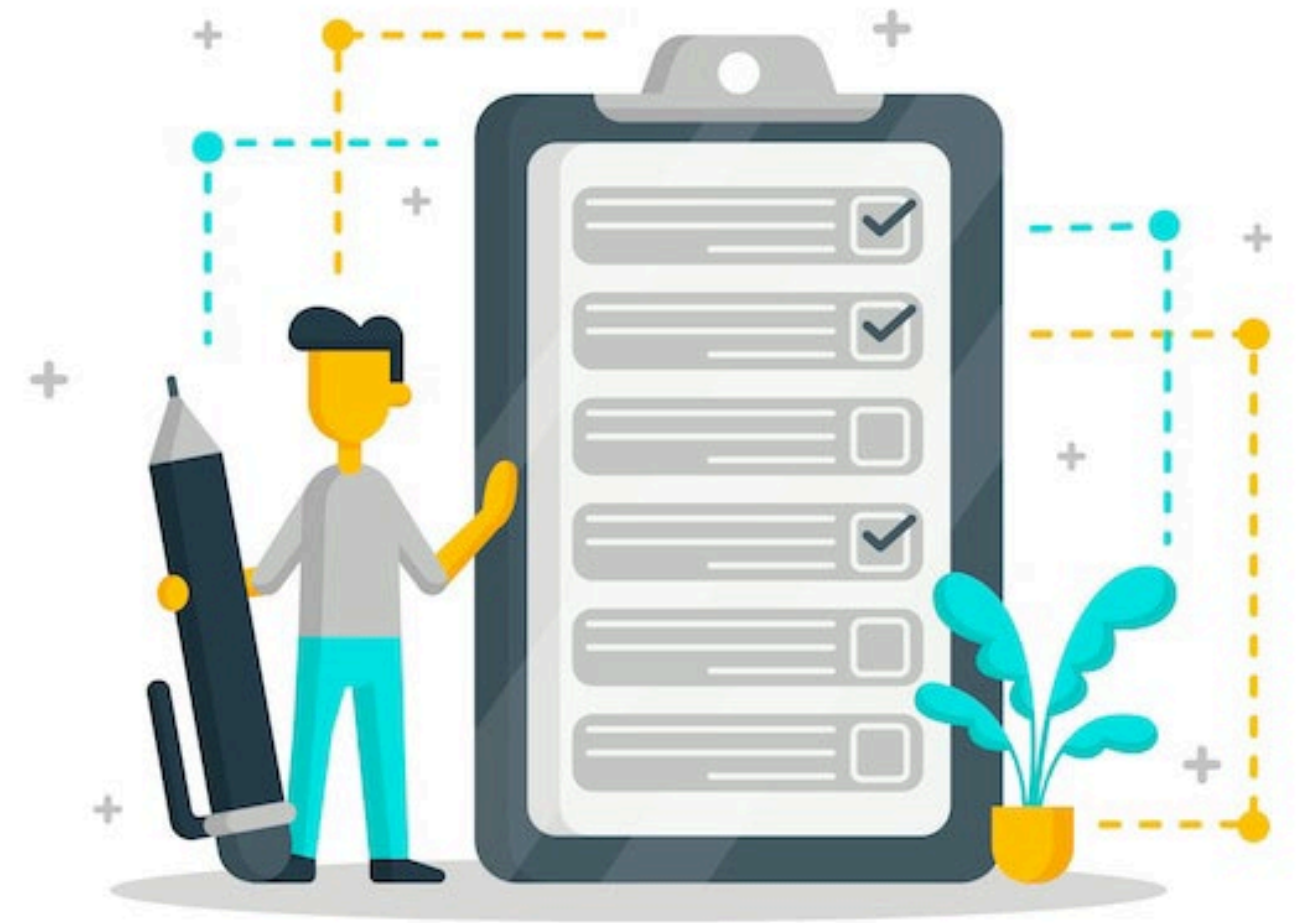
- In addition to the boundary values, it can be beneficial to include some test cases within the input range to ensure comprehensive testing.
- These additional test cases should cover typical values, extreme values, and any other specific conditions or requirements.

Exercise

To dive deeper into practical application, refer to the corresponding exercise in the document for hands-on exploration and analysis.

Exercise 3: Age Verification System

Exercise 4: Volume Control Adjustment



Decision Table Testing

- Decision Table Testing is a systematic and structured approach to testing that helps identify combinations of conditions and actions to be tested.
- The basic idea behind Decision Table Testing is to capture complex business rules or system behaviours in a tabular format, making it easier to understand and test different combinations of inputs and their corresponding outputs.
- The decision table consists of conditions, actions, and rules that define the relationships between them.
- Systematic thinking in decision tables involves a structured approach to designing test cases by systematically analysing all possible combinations of conditions and actions. Decision tables provide a visual representation of various conditions and corresponding actions, making it easier to identify different test scenarios and ensure comprehensive test coverage. Systematic thinking helps testers consider all possible combinations and outcomes, leading to more effective and efficient test case creation.

Decision Table Testing

	Rule1	Rule2	Rule3	Rule4	Rule5	Rule6	Rule7	Rule8
Condition Stub			Condition Entries					
Action Stub			Action Entries					

Decision tables are a good way to capture system requirements that contain logical conditions, and to document internal system design. They may be used to record complex business rules that a system is to implement. When creating decision tables, the specification is analysed, and conditions and actions of the system are identified. The input conditions and actions are most often stated in such a way that they must be true or false (Boolean).

The decision table contains the triggering conditions, often combinations of true and false for all input conditions, and the resulting actions for each combination of conditions. Each column of the table corresponds to a business rule that defines a unique combination of conditions and which result in the execution of the actions associated with that rule.

Decision Table Testing

(Continued)

Possible conditions at traffic lights

(rule heading omitted)

Red (on/off)									
Amber (on/off)									
Green (on/off)									

On arrival at traffic lights 3 conditions have to be tested as each light can be on or off

These go in the condition stub as above.

Decision Table Testing

(Continued)

Possible combinations of "lights on" at traffic lights

Red	x	x				x		x
Amber		x		x		x	x	
Green			x			x	x	x

Run through the possible combinations as shown above (x = on)

Explain how to calculate the number of condition combinations
(2 to the power of the number of conditions i.e. in this scenario 2 to the 3 = 8)

Decision Table Testing

(Continued)

Red	x	x				x		x
Amber		x		x		x	x	
Green			x			x	x	x
Slow down								
Prepare to stop								
Stop								
Prepare to go								
Go								
Report a fault								

Possible actions

Explain what goes in to the Action Stub

Point out that a limited selection of possible action is used here, just sufficient to make the point.

Decision Table Testing

(Continued)

Red	x	x				x		x
Amber		x		x		x	x	
Green			x			x	x	x
Slow down	x			x	x	x	x	x
Prepare to stop	x			x	x	x	x	x
Stop	x				x	x	x	x
Prepare to go		x						
Go			x					
Report a fault				x	x	x	x	

Expected results

Explain that expected results go in to the action entry quadrant

Get delegates to check that expected results are correct in for each condition combination

The coverage standard commonly used with decision table testing is to have at least one test per column in the table, which typically involves covering all combinations of triggering conditions.

Decision Table Testing

(Continued)

Red	x	x						
Amber		x		x	else			
Green			x					
Slow down	x			x	x			
Prepare to stop	x			x	x			
Stop	x				x			
Prepare to go		x						
Go			x					
Report a fault					x			

Expected results

As the expected results in the last four columns are the same, the table can be rationalised using the 'else rule' as shown above

The only problem here is the other four combinations to be tested are now not shown.

Decision Table Testing

(Continued)

How many columns do I need in Decision Table Testing?

- The number of columns required in a decision table depends on the complexity of the software's logic and the number of conditions and actions to be evaluated. Each column represents a specific condition or action, and there should be one column for each possible condition and one column for each possible action. If the software has multiple conditions and actions, the decision table should have an appropriate number of columns to capture all the combinations accurately.

How do I know I have enough columns in Decision Table Testing?

- To ensure you have enough columns in the decision table, you need to consider all the unique conditions and actions relevant to the software's behavior. Each column should represent a distinct condition or action, and there should be no overlap or redundancy. Review the software requirements and functional specifications to identify all possible conditions and actions that impact the system's behavior. If you have captured all the unique combinations of conditions and actions in separate columns, you can be confident that you have enough columns in the decision table.

© The Knowledge Academy Ltd

Benefits of Decision Table Testing:

Decision table testing, also known as cause-effect table testing, is a systematic and structured testing technique used to test complex business rules. Here are the key benefits of decision table testing:

- Comprehensive Coverage
- Simplicity
- Eliminates Redundancy
- Versatility
- Systematic Approach
- Effective for Complex Rules
- Easy Maintenance
- Enhanced Communication

Decision Table Testing

Steps for Decision Table Testing:



© The Knowledge Academy Ltd

The steps involved in Decision Table Testing are as follows:

- 1. Identify the conditions:** Determine the input variables or factors that affect the behaviour of the system. These conditions should be relevant and significant to the business rules or system requirements.
- 2. Identify the actions:** Determine the expected outcomes or responses based on the conditions. Actions should represent the desired behaviour or system response.
- 3. Create the decision table:** Construct a tabular representation where each column represents a condition or action, and each row represents a rule or combination of conditions and actions. Fill in the decision table with all possible combinations.
- 4. Define the rules:** Specify the relationships between the conditions and actions in the decision table. Rules help determine the expected action(s) for a given combination of conditions.
- 5. Identify test cases:** Generate test cases by selecting different combinations of conditions from the decision table. Each test case should cover a unique combination of conditions and verify the expected action(s).
- 6. Execute the test cases:** Run the identified test cases and compare the actual system behaviour with the expected actions defined in the decision table. By using Decision Table Testing, testers can systematically test various combinations of conditions and actions, ensuring thorough coverage of complex business rules or system behaviours. The decision table serves as a visual aid to document, understand, and test different scenarios in a structured and organised manner.

Decision Table Testing

How do I know I covered all the combinations of Decision Table Testing?

To verify that you have covered all the combinations in Decision Table Testing, you can follow these steps:

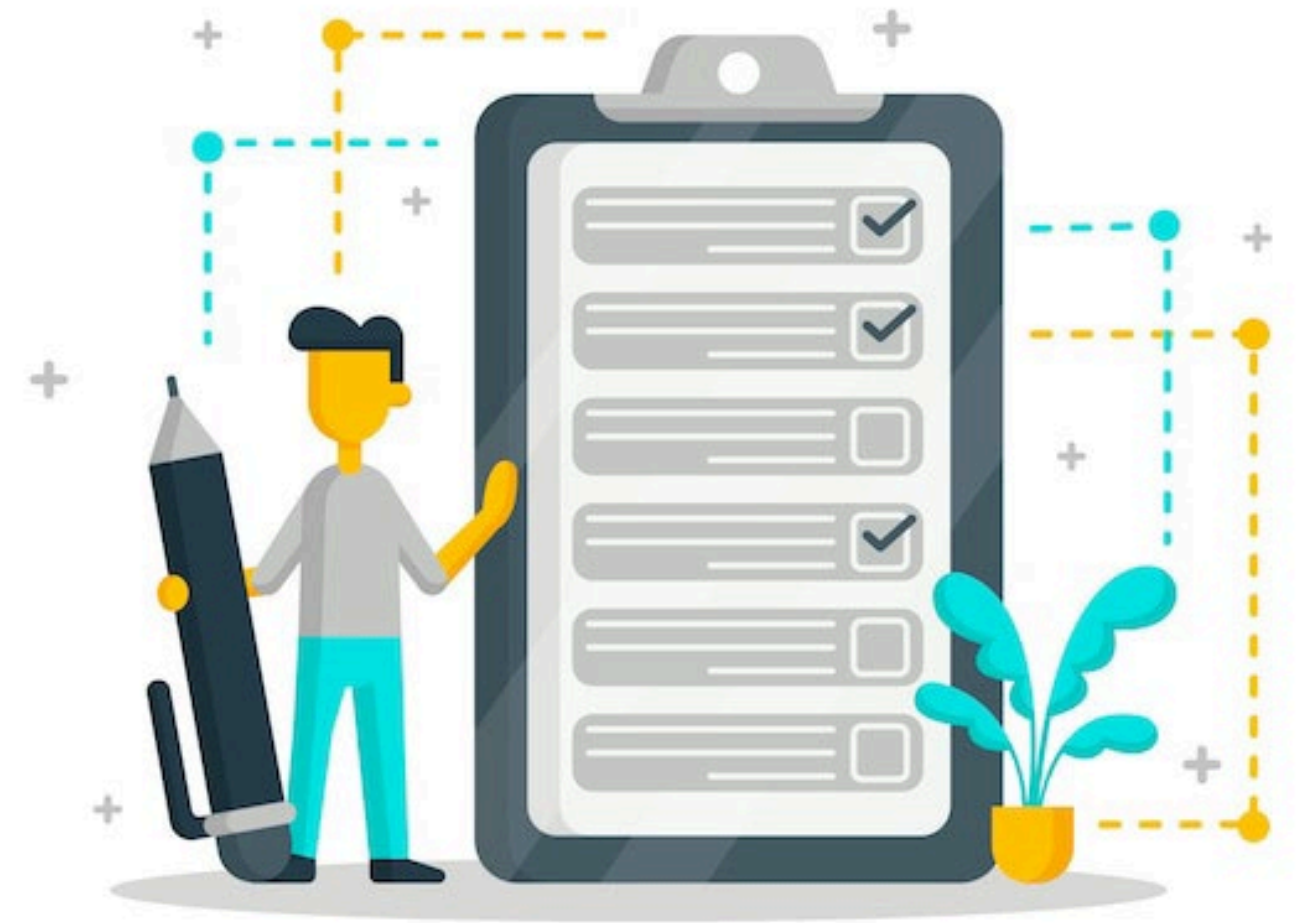
- Ensure each unique combination of conditions is represented in the decision table.
- Cross-check that each condition appears at least once in each combination.
- Confirm that all possible actions resulting from various condition combinations are included.
- Review the decision table with subject matter experts and stakeholders to validate its completeness.
- Perform test case execution using the decision table and verify if the software behaves as expected for each combination.
- By diligently following these steps, you can be confident that you have covered all the combinations in Decision Table Testing.

Exercise

To dive deeper into practical application, refer to the corresponding exercise in the document for hands-on exploration and analysis.

Exercise 5: Online Bookstore Discount Policy

Exercise 6: Employee Performance Review System



State Transition Testing

- State Transition Testing focuses on testing the behaviour of a system as it transitions between different states.
- The main objective of State Transition Testing is to verify that the system responds correctly to events or inputs and undergoes state changes as expected.
- It is particularly useful for systems with well-defined states and specific behaviours associated with those states.
- In State Transition Testing, you identify the various states that the system can be in during its execution.

Benefits

Comprehensive Coverage

State Transition Testing allows for a comprehensive examination of how a system behaves across various states and transitions. This helps in identifying potential issues that may not be evident when testing individual functions or components in isolation.

Easy to Understand

The use of state diagrams and state tables makes it easier to understand the behavior of a complex system. This visual representation aids both developers and testers in grasping how different states and transitions interact, making it simpler to design and execute tests.

Effective for Complex Systems

This testing approach is particularly effective for systems with complex business logic and multiple states. It ensures that all transitions and states are tested, thereby reducing the likelihood of defects cropping up in the later stages of development or during production.

Uncovers Edge Cases

State Transition Testing is excellent at uncovering edge cases that might otherwise go unnoticed. By focusing on the transitions between states, especially under different conditions (guard conditions), it's possible to identify scenarios that could lead to system failures.

Validates Business Rules

Many systems have specific business rules that dictate allowable state transitions. State Transition Testing ensures that these rules are correctly implemented and enforced, thereby increasing the system's reliability and adherence to specifications.

State Transition Testing

Guard conditions

- Guard conditions in State Transition Testing are conditions that must be satisfied for a state transition to occur in a software application.
- They act as prerequisites or constraints that control the flow of the system from one state to another. If the guard conditions are not met, the transition will not take place, ensuring that the system behaves correctly and consistently.

State Table

- A state table mirrors the functions of a state transition diagram. In this table, rows symbolise the states, while columns depict events, which may include any accompanying guard conditions.
- The intersections of rows and columns (or the table's cells) indicate transitions, detailing the destination state and any subsequent actions if specified. Unlike the state transition diagram, the state table clearly displays invalid transitions through unpopulated cells.

Example

Guard Conditions

In State Transition Testing, guard conditions are logical constraints that act as a gatekeeper for state transitions. For instance, consider an ATM machine with various states like "Idle," "Card Inserted," "PIN Entered," and "Transaction Complete." A guard condition for transitioning from the "Card Inserted" state to the "PIN Entered" state might be that the inserted card must be valid (`CardIsValid == True`). Similarly, moving from the "PIN Entered" state to the "Transaction Complete" state might have a guard condition that checks both the validity of the PIN (`PINIsValid == True`) and whether the account balance is sufficient for the withdrawal (`AccountBalance >= WithdrawalAmount`). These conditions must be met for the state transitions to occur, ensuring that the system behaves correctly.

State Table

A State Table is a systematic way of representing all the states, events, and transitions in a system. Taking the example of an ATM machine again, you would list states like "Idle," "Card Inserted," "PIN Entered," and "Transaction Complete." The events that trigger state transitions, such as "Insert Card" or "Enter PIN," would also be listed. You would then specify the next state resulting from a given current state and input event, often with guard conditions to add further clarity. For example, if the current state is "Idle" and the input event is "Insert Card," the next state would be "Card Inserted" if the guard condition `CardIsValid == True` is met. Likewise, the state transitions from "Card Inserted" to "PIN Entered" when the "Enter PIN" event occurs and the guard condition `PINIsValid == True` is satisfied.

State Transition Testing

Coverage Criteria

- Coverage criteria in State Transition Testing refer to the specific conditions and combinations of transitions that need to be tested to ensure thorough testing of a software application's state-based behaviour.
- It involves defining a set of test cases that cover all possible states and transitions between states, including valid and invalid transitions.
- The goal is to verify that the application behaves correctly and consistently as it moves between different states, thereby identifying potential defects and ensuring reliable software performance.

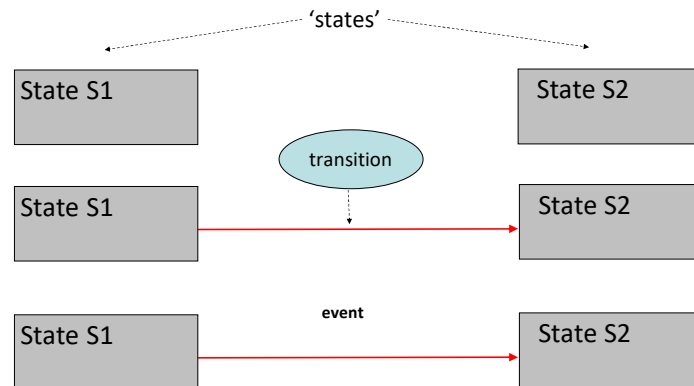
Example

Coverage Criteria

Coverage Criteria are metrics used to measure how thoroughly the state transition tests cover the software's behavior. Three common types of coverage criteria are State Coverage, Transition Coverage, and State-Transition Pair Coverage. For State Coverage, you aim to visit each state at least once during testing. So, in the case of our ATM example, you would design test scenarios that pass through each of the states: "Idle," "Card Inserted," "PIN Entered," and "Transaction Complete." For Transition Coverage, the goal is to exercise each transition between states at least once. This would involve running tests that cover all the transitions, such as from "Idle" to "Card Inserted" or from "Card Inserted" to "PIN Entered." Finally, State-Transition Pair Coverage aims to cover every possible combination of source state, input event, and target state. For example, you'd test the transition from "Idle" to "Card Inserted" via the "Insert Card" event, and similarly for all other combinations.

State Transition Testing

Diagram Notation



Lecturer Notes

Ensure the following points are made:-

State transition testing is much used within the embedded software industry and technical automation in general. However, the technique is also suitable for modelling a business object having specific states or testing screen-dialogue flows (e.g., for Internet applications or business scenarios).

Additional Lecturer Notes

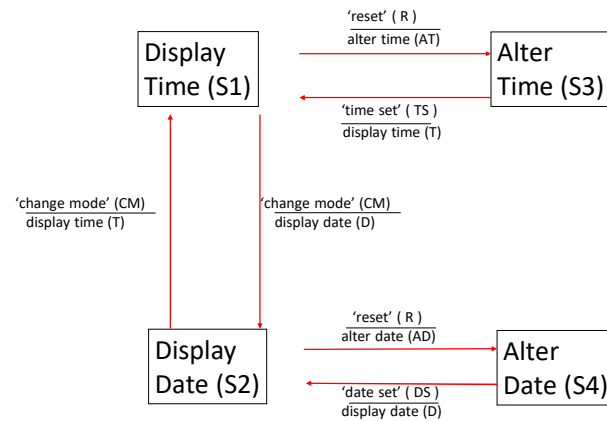
Explain the terms state, state transition, and event

Explain the idea of the labels (S1,S2).

State Transition Testing

(Continued)

▪ **Example Diagram:**



Test cases are derived from the state transition diagram to exercise each of the possible transitions. These are recorded in a State Matrix

State Transition Testing

(Continued)

- States represent the conditions or configurations of the system at different points in time. These states are often defined as specific values or combinations of variables within the system.
- Events are the triggers or inputs that cause the system to transition from one state to another. .
- Transitions, on the other hand, represent the changes in the system's state triggered by specific events.
- Each transition specifies the starting state, the event that causes the transition, and the resulting state.

© The Knowledge Academy Ltd

- To perform State Transition Testing, you need to analyse the system and identify all possible states, events, and transitions.
- Based on this analysis, you design test cases that cover different combinations of events and transitions. The aim is to ensure that the system behaves correctly and transitions smoothly between states as intended.
- By applying State Transition Testing, you can uncover defects related to state changes, boundary conditions, and unexpected system behaviour.
- It helps to provide comprehensive test coverage and validate the correct functioning of the system under different scenarios and state transitions.

Exercise

To dive deeper into practical application, refer to the corresponding exercise in the document for hands-on exploration and analysis.

Exercise 7: Online Ticket Booking System



Module 3: White-box Test Techniques



White-box Test Techniques

- White-box testing, also known as structural or glass-box testing, involves the examination of the software's internal structure, design, and implementation. A core strength of white-box techniques is their ability to facilitate defect detection even when software specifications are unclear or incomplete, by considering the entire codebase during testing.
- However, this method might not detect defects of omission if the software fails to implement certain requirements. White-box techniques are versatile and can be applied to both static and dynamic testing, including during dry runs of code, pseudocode, or other high-level logic modeled with a control flow graph. Unlike black-box testing, which does not measure actual code coverage, white-box testing provides an objective measure of coverage.
- This allows for the generation of additional tests to increase coverage, thereby enhancing confidence in the code. Despite its potential limitations, such as overlooking unimplemented requirements, white-box testing is a powerful tool in a comprehensive software testing strategy, offering in-depth insights into the inner workings of the software.

Statement Testing and Statement Coverage

- Statement testing and statement coverage are two concepts related to software testing that focus on examining and measuring the extent to which the statements in a program have been executed.
- Statement testing, also known as statement-level testing or statement coverage, is a technique used to exercise the statements in the code. By executing the individual statements during testing, statement coverage is achieved
- The goal is to ensure that every statement in the program is executed at least once.
- Statement coverage, on the other hand, is a metric used to quantify the degree to which the statements in a program have been exercised by a given set of test cases.



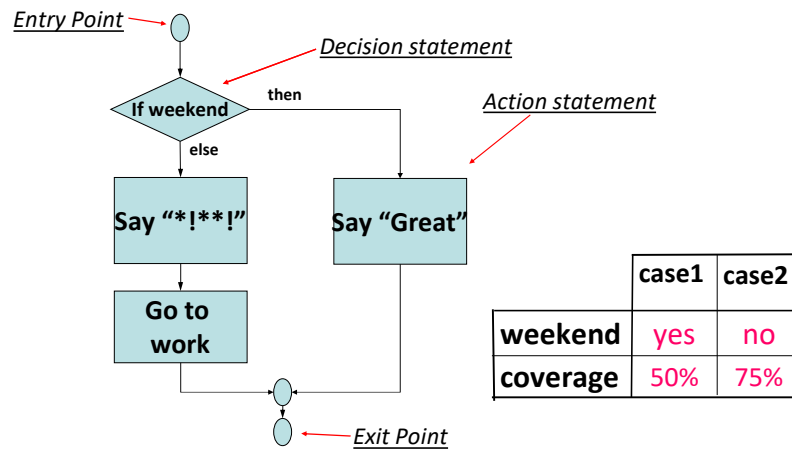
Statement Testing and Statement Coverage

(Continued)

- To achieve statement coverage, a test suite needs to include test cases that exercise each program statement at least once. This means that every line of code in the program should be executed during testing.
- By analysing the coverage achieved, testers can identify untested or uncovered parts of the code and create additional tests to improve coverage.
- Coverage is calculated by dividing the count of statements tested by the total number of executable statements in the codebase, and this ratio is then presented as a percentage.
- However, it's worth noting that achieving high statement coverage does not guarantee the absence of defects or the complete correctness of the software.
- It only ensures that the statements have been executed, but not necessarily that all possible scenarios and behaviours have been tested.

Other types of coverage metrics, such as branch coverage and path coverage, are often used in conjunction with statement coverage to provide a more comprehensive assessment of test coverage.

Statement Testing and Statement Coverage

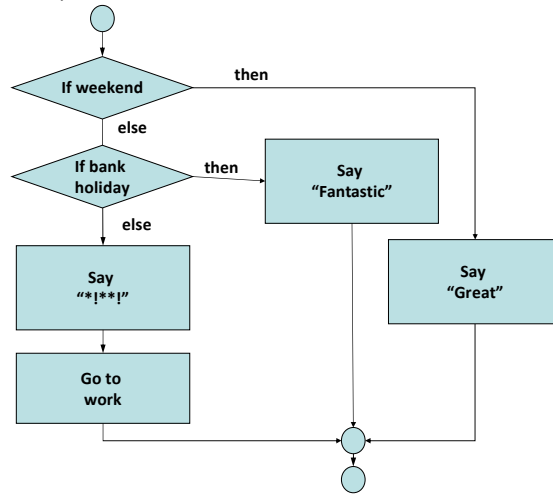


In component testing, statement coverage is the assessment of the percentage of executable statements that have been exercised by a test case suite. The statement testing technique derives test cases to execute specific statements, normally to increase statement coverage.

Statement coverage is determined by the number of executable statements covered by (designed or executed) test cases divided by the number of all executable statements in the code under test.

Statement Testing and Statement Coverage

▪ (Continued)



Case 1	Case 2	Case 3
weekend = yes	weekend = no b/holiday = yes	both = no
33.3%	50%	66.6%

Example assumes that bank holidays do not fall on weekends

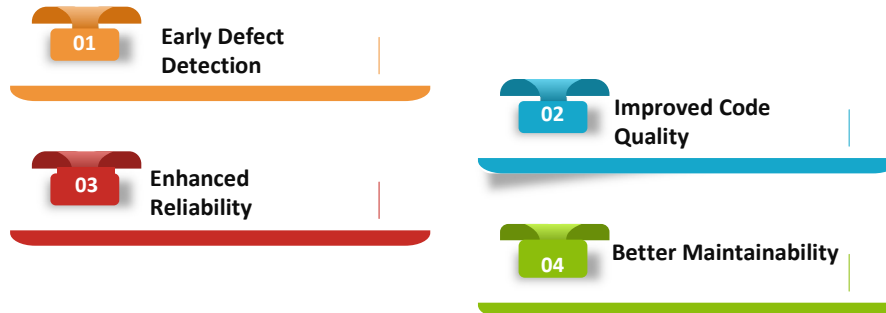
Q: How many paths there are through the above logic?

Q: Ask how many tests would be required for 100% statement coverage?

Statement Testing and Statement Coverage

(Continued)

- *Benefits of Statement Testing and Statement Coverage:*



© The Knowledge Academy Ltd

By executing each statement during testing, statement testing helps identify defects or errors early in the development cycle, reducing the cost and effort required for bug fixing in later stages.

Through statement coverage analysis, developers and testers gain insights into the portions of code that have not been tested, enabling them to focus on those areas and improve the overall code quality.

Thorough statement testing and high statement coverage increase the confidence in the software's reliability, as a larger portion of the code has been exercised and validated.

When testers execute each statement, it becomes easier to trace the flow of execution and understand the functionality of the code.

This aids in maintaining and enhancing the software in the future.

Branch Testing and Branch Coverage

Branch Testing

- In software, a branch depicts a point where control can transition between two nodes in the control flow graph. This graph visually demonstrates the interaction of different code segments, showcasing potential execution sequences. Branches can either be conditional or unconditional.
- The primary objective of branch testing is to design and execute test cases that encompass all branches in the code until a satisfactory coverage level is attained. This includes navigating both unconditional and conditional branches, which arise from "if...then" decisions, switch/case statements, or decisions within loops.



© The Knowledge Academy Ltd

Branch Testing Example

Branch Testing is a testing technique aimed at ensuring that every branch in the code is executed at least once. This testing strategy is particularly crucial for conditional statements like if, else, switch, for, and while, among others. In a business application, consider a complex decision-making algorithm that determines loan eligibility based on multiple criteria like credit score, income, and loan amount. Branch testing would be employed to ensure that every possible decision path within this algorithm is evaluated. For instance, if the algorithm has a branching condition that filters out applicants with a credit score below 600, a test case would be designed specifically to trigger this branch. Another test case might focus on the branch dealing with high-income applicants who request a small loan, ensuring that the system correctly identifies them as low-risk. By meticulously designing test cases for each branch, you ensure comprehensive test coverage and minimise the risk of undetected defects affecting the loan approval process.

Branch Testing and Branch Coverage

Branch Coverage

- Branch coverage is a metric that quantifies the proportion of tested branches to the total branches within the code, typically represented as a percentage. Achieving 100% branch coverage implies that every branch—conditional and unconditional—has been tackled by the test cases. Nonetheless, even at this comprehensive coverage level, certain defects might remain undetected, especially those requiring the traversal of a specific code path.

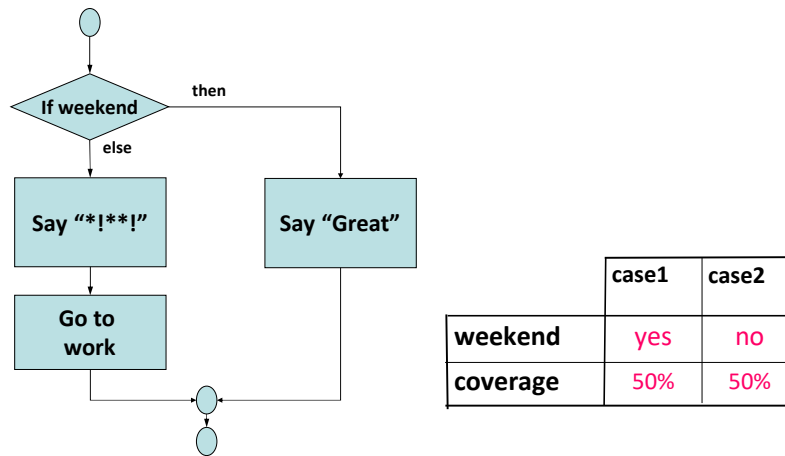
Relation to Statement Coverage

- Branch coverage inherently includes statement coverage. Consequently, test cases that secure 100% branch coverage will, by default, accomplish 100% statement coverage. This integral relationship stems from the fact that covering all branches ensures the execution of all individual code statements. However, this relationship isn't reciprocal; obtaining 100% statement coverage doesn't assure 100% branch coverage.

Branch Coverage Example

Branch Coverage is a metric that quantifies the percentage of branches that have been executed during testing. It's calculated as the number of executed branches divided by the total number of branches, often expressed as a percentage. For example, in a customer relationship management (CRM) system, there might be a function that categorises customers into different tiers ("Gold," "Silver," "Bronze") based on their purchase history and interaction with the service. This function might contain multiple branches to handle each tier. In a professional setting, Branch Coverage would not only involve ensuring that each tier categorisation is tested but would also require documentation and metrics to prove it. Tools like code coverage analysers can automate this process, providing real-time data on the percentage of branches covered. Achieving high Branch Coverage ensures that the system's decision-making logic is sound and that the application is robust enough to handle various scenarios. It serves as a key performance indicator for the overall quality assurance process.

Branch Testing and Branch Coverage



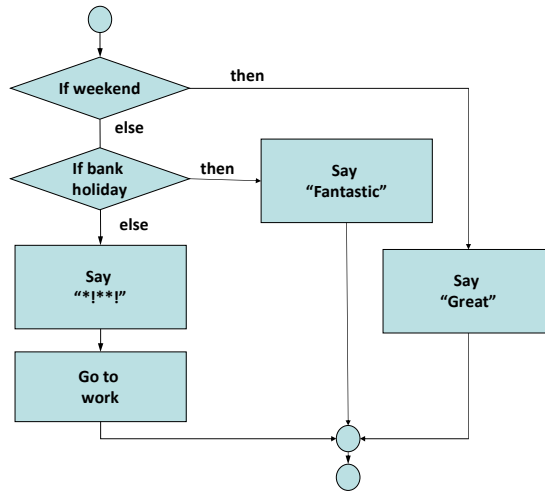
© The Knowledge Academy Ltd

Decision coverage, related to branch testing, is the assessment of the percentage of decision outcomes (e.g., the True and False options of an IF statement) that have been exercised by a test case suite. The decision testing technique derives test cases to execute specific decision outcomes.

Decision testing exercises the decisions in the code and tests the code that is executed based on the decision outcomes. To do this, the test cases follow the control flows that occur from a decision point (e.g., for an IF statement, one for the true outcome and one for the false outcome; for a CASE statement, test cases would be required for all the possible outcomes, including the default outcome). Coverage is measured as the number of decision outcomes executed by the tests divided by the total number of decision outcomes in the test object, normally expressed as a percentage

Decision coverage is stronger than statement coverage; 100% decision coverage guarantees 100% statement coverage, but not vice versa. (demonstrate this by an example on the Flip or Board).

Branch Testing and Branch Coverage



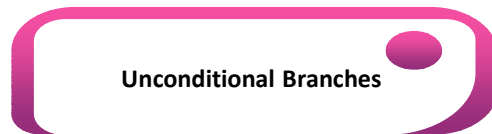
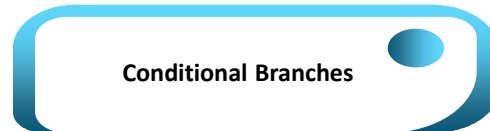
Case 1	Case 2	Case 3
weekend = yes	weekend = no b/holiday = yes	both = no
25%	50%	50%

Demonstrate how these 3 tests will give 100% decision coverage

Branch Testing and Branch Coverage

(Continued)

- *Types of Branches:*



Conditional Branches

These branches are determined by conditional statements such as if-else, switch-case, and ternary operators. They depend on the evaluation of a condition to determine the path the program takes.

Unconditional Branches

These branches are not dependent on any condition and are typically encountered in loops, function calls, and jumps.

Branch Testing and Branch Coverage

(Continued)

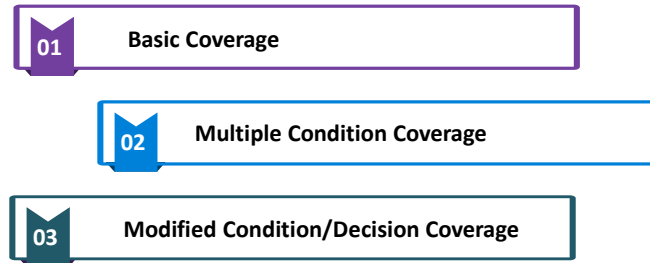
- ***Importance of Branch Testing:***

- **Understanding Control Flow:** At its core, branch testing revolves around the control flow graph, representing the multitude of code paths. This involves both unconditional and conditional control transfers.
- **Measuring Coverage:** Branch testing aspires to attain a predefined coverage level by probing every branch in the code. The coverage is gauged by comparing the branches tested to the total branches, resulting in a percentage representation.
- **Inclusive of Statement Coverage:** Branch testing's comprehensive nature ensures it subsumes statement coverage. Achieving complete branch coverage inherently leads to total statement coverage, affirming its position as a superior testing technique.

Branch Testing and Branch Coverage

(Continued)

- *Strategies for Branch Testing:*



Basic Coverage: Test cases are designed to execute both the true and false branches of each decision point. This ensures that all branches are tested at least once.

Multiple Condition Coverage: Test cases aim to cover various combinations of conditions within a decision point. This helps uncover errors that might occur due to specific combinations of conditions.

Modified Condition/Decision Coverage: Each condition within a decision point is tested independently, and the effect of changing a single condition is observed. This technique ensures that each condition has a significant impact on the decision outcome.

Branch Testing and Branch Coverage

(Continued)

- **Challenges in Branch Testing:**
 - **Combinatorial Explosion:** As the number of branches and conditions increase, the number of test cases required to achieve complete branch coverage can grow exponentially. This makes it challenging to achieve 100% coverage in complex programs.
 - **Oracle Problem:** Determining the expected outcome for each branch can be difficult, especially in large systems or when the code lacks proper documentation.
 - **Dynamic Branches:** Some branches might be determined at runtime based on user input or system states. Testing all possible dynamic branches can be challenging.

The Value of White-box Testing

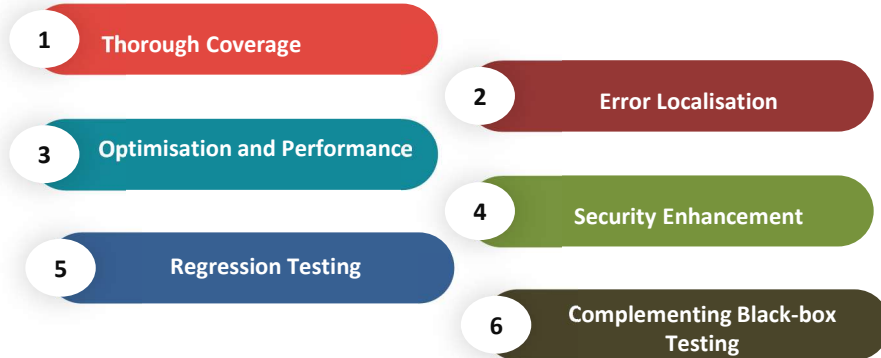
- White-box testing allows testers to design test cases with precision, targeting specific code paths, loops, and conditional statements.
- This level of granularity helps in identifying subtle programming errors that might be missed by black-box testing. Testers can craft test scenarios that exercise corner cases and edge conditions, ensuring thorough validation of the code.
- It offers several advantages that contribute to the development of reliable, secure, and high-performance software. Its ability to deep dive into the code's internals enables precise test case design, early defect detection, and optimisation of code, making it an essential component of a well-rounded testing strategy.



The Value of White-box Testing

(Continued)

- *White-box testing provides several significant benefits:*



By focusing on the internal logic and structure of the code, white-box testing enables the detection of hidden errors, helps validate data flow and conditional logic, and ensures that all code paths are thoroughly exercised, including edge cases. This level of scrutiny is particularly beneficial in complex systems where small errors can lead to significant issues.

The Value of White-box Testing

1. Thorough Coverage

- Since white-box testing requires knowledge of the internal structure, it allows for extensive coverage of the software.
- Developers can design test cases that exercise different paths and conditions within the code, aiming to test all possible scenarios.
- This helps identify hidden errors, logic flaws, and edge cases that may not be apparent through other testing methods.

The Value of White-box Testing

2. Error Localisation

- White-box testing enables pinpointing the exact location of defects within the code.
- By examining the internal structure, developers can identify the specific functions, modules, or lines of code that contain errors.
- This information is valuable for developers, as it allows them to saving time and effort in the debugging process.

The Value of White-box Testing

3. Optimisation and Performance

- White-box testing involves taking into account the entire software implementation, facilitating defect detection even if the software specification is unclear. It can effectively examine code not yet ready for execution, pseudocode, and other high-level logic, which can be modeled with a control flow graph.
- This comprehensive insight into the code structure allows for a detailed understanding of how the software performs, providing opportunities to identify inefficiencies and optimise performance.

The Value of White-box Testing

4. Security Enhancement

- In white-box testing, the underlying code's logic, structure, and implementation are fully transparent to the tester. This transparency enables the tester to evaluate the security measures embedded within the code more effectively.
- By understanding how the software functions at the granular level, vulnerabilities that may be overlooked using other testing approaches can be uncovered.
- This strengthens the software's overall security by ensuring that potential weak points are identified and addressed.

The Value of White-box Testing

5. Regression Testing

- White-box testing can play a valuable role in regression testing by offering insights into the specific areas of the code affected by changes or updates.
- Unlike black-box testing, which doesn't provide a measure of actual code coverage, white-box testing offers objective coverage measures. This information allows for the generation of additional tests to enhance coverage, providing confidence that changes to the code haven't adversely impacted existing functionality.

The Value of White-box Testing

6. Complementing Black-box Testing

- Black-Box testing focuses on validating the functionality without considering the underlying code, it does not provide a measure of actual code coverage. In contrast, white-box testing's coverage measures give an objective assessment of what parts of the code have been tested.
- This allows for the generation of additional tests to increase coverage, thereby enhancing confidence in the code. Combining white-box testing with black-box testing ensures a more comprehensive evaluation of the software, leveraging the strengths of both approaches.

Module 4: Experience-based Test Techniques



Error Guessing

- Error Guessing leverages the tester's intuition, experience, and domain knowledge to identify potential defects or errors in the system
- The concept behind Error Guessing is that experienced testers can anticipate or "guess" where errors might occur in the software based on their knowledge of common software issues, development mistakes, and possible user interactions.
- It is particularly useful when formal requirements or specifications are incomplete or unavailable. Testers using Error Guessing apply their expertise to identify possible error-prone areas or scenarios in the system.
- They make informed guesses about the types of errors that could occur and design test cases to target those areas.
- The technique relies on a combination of creative thinking and experience to uncover defects that might not be identified through other formal testing techniques.

© The Knowledge Academy Ltd

- Error Guessing is a flexible technique that does not follow a specific set of rules or procedures. Testers rely on their own judgment and experience to determine the test cases to be executed.
- The effectiveness of Error Guessing largely depends on the tester's knowledge, creativity, and ability to think from different perspectives.
- While Error Guessing can be a valuable addition to a comprehensive testing strategy, it should be used in conjunction with other formal testing techniques to ensure thorough test coverage.

Error Guessing

(Continued)

- The technique helps uncover defects that might be missed by more structured testing approaches and provides an opportunity for testers to leverage their expertise to identify potential areas of risk.

The following are some key points to further understand Error Guessing:

1. Tester's expertise

- Error Guessing heavily relies on the tester's expertise and experience. Testers use their knowledge of the system, domain, programming languages, common software issues, and potential pitfalls to anticipate areas where defects are likely to occur.

Error Guessing

2. Intuition-based approach

- Testers employ their intuition and creative thinking to "guess" possible errors based on the characteristics and complexities of the system.
- They often put themselves in the end-users' shoes and simulate various scenarios that could potentially lead to errors.

3. Lack of formal documentation

- Error Guessing is useful when formal requirements or specifications are incomplete, ambiguous, or unavailable.
- It allows testers to explore the system beyond defined boundaries and identify potential defects that may not have been explicitly documented.

Error Guessing

4. Focus on risky areas

- Testers concentrate on areas of the system that are considered more error-prone or critical. These areas might include complex algorithms, boundary conditions, input validations, error handling routines, or user interfaces where users are likely to make mistakes.

5. Test case generation

- Testers generate test cases based on their intuition and knowledge of potential errors. These test cases are designed to target specific areas where defects are likely to be found.
- Testers may also combine Error Guessing with other formal techniques to ensure comprehensive test coverage.

Exploratory Testing

- Exploratory testing is an approach to testing that is based on experience and is considered the most important method within experience-based testing. While some refer to scripted testing and free-style testing, it is preferred to categorise testing as either coverage-based or experience-based, recommending a combination of both approaches.
- Among the different experience-based testing methods, exploratory testing stands out as the most flexible and adaptable.
- The primary objective of testing is not just to create and execute tests, but rather to gather information about the quality and risks associated with an IT solution.
- The session-based approach in exploratory testing is a structured and time-boxed testing method. During each session, testers explore the application without a predefined test script, allowing for flexibility and creativity in finding defects.

Exploratory Testing is a dynamic and unscripted approach to software testing that emphasises the tester's skills and creativity. Unlike traditional methods that rely on predefined test cases, exploratory testing allows testers to concurrently design and execute tests, adapting their strategy based on the results and observations. This approach is highly effective for identifying unusual or unexpected issues, offering quick feedback, and is particularly useful in agile or fast-paced development environments where requirements may change frequently.

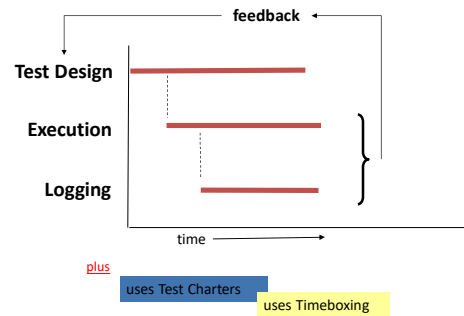
Exploratory Testing

(Continued)

- This information is crucial for establishing confidence that the solution will deliver the expected business value.
- Exploratory testing is particularly effective in supporting this confidence-building process due to its interactive nature and its ability to involve multiple stakeholders.

Test Charter

- A test charter in exploratory testing is a high-level mission or objective that guides the tester during the testing session. It outlines the key areas to be explored, the scope of testing, and any specific goals or priorities.



- In exploratory testing, informal (not pre-defined) tests are designed, executed, logged, and evaluated dynamically during test execution. The test results are used to learn more about the component or system, and to create tests for the areas that may need more testing.
- Exploratory testing is sometimes conducted using session-based testing to structure the activity. In session-based testing, exploratory testing is conducted within a defined time-box, and the tester uses a test charter containing test objectives to guide the testing. The tester may use test session sheets to document the steps followed and the discoveries made.
- Exploratory testing is most useful when there are few or inadequate specifications or significant time pressure on testing. Exploratory testing is also useful to complement other more formal testing techniques.
- Exploratory testing is strongly associated with reactive test strategies. Exploratory testing can incorporate the use of other black-box, white-box, and experience-based techniques

Exploratory Testing

(Continued)

- The key characteristics of Exploratory Testing include:

1. Learning and Adaptation

- Exploratory Testing emphasises learning and adapting throughout the testing process. Testers gain insights into the system's behaviour, identify areas of potential risk, and adjust their testing approach accordingly.

2. Test Design on-the-fly

- Testers design test cases as they explore the system, leveraging their knowledge and intuition to identify interesting scenarios, boundary conditions, and potential defects.

Exploratory Testing

3. Documentation

- Although Exploratory Testing is flexible and less reliant on formal documentation, testers may capture notes, screenshots, or recordings to document their findings and facilitate communication with stakeholders.

4. Domain Knowledge and Experience

- Testers' domain knowledge and experience play a crucial role in Exploratory Testing. Their expertise helps them navigate the system effectively and target areas that are more likely to have defects.

5. Time Constraints and Focus

- Exploratory Testing is often performed within time constraints, allowing testers to focus on specific areas, features, or risks based on the project's priorities.

Exploratory Testing

6. Collaboration and Communication

- Collaboration among testers, developers, and stakeholders is essential in Exploratory Testing.
- Testers share their findings, observations, and insights to facilitate a deeper understanding of the system's behaviour.
- Exploratory Testing is a valuable technique to complement formal test design approaches and uncover defects that may not be identified through traditional scripted testing.
- It allows testers to leverage their expertise and creativity to explore the software in a flexible and adaptive manner, promoting a deeper understanding of the system's quality.

Checklist-Based Testing

- Checklist-Based Testing involves the utilisation of predefined checklists or criteria to guide and structure the testing process, ensuring comprehensive coverage of specific aspects of the system under test.
- These checklists are typically developed based on industry best practices, relevant standards, project requirements, or domain-specific regulations.
- The checklists used in Checklist-Based Testing serve as a reference to identify and verify crucial elements or conditions that need to be assessed during testing.



Checklist-Based Testing

(Continued)

- They encompass various areas such as functional requirements, non-functional attributes (e.g., performance, security), usability guidelines, or any other pertinent criteria.
- Testers refer to the checklist items to design and execute test cases that target the listed items. By aligning their test cases with the checklist, testers can systematically examine and validate the desired behaviour, compliance, and quality attributes of the system.
- One of the advantages of Checklist-Based Testing is its ability to promote standardisation and consistency within the testing process.
- It assists testers in ensuring that important aspects are not overlooked or neglected, thus improving the overall test coverage.

© The Knowledge Academy Ltd

- Additionally, the use of checklists provides a tangible and measurable framework for evaluating the completion of testing activities, facilitating traceability and reporting.
- While Checklist-Based Testing can be a valuable technique, it is important to recognise that it should not be solely relied upon.
- It is recommended to combine this technique with other testing methods to achieve comprehensive test coverage.
- Additionally, checklists should be regularly reviewed and updated to reflect evolving project requirements and to incorporate lessons learned from previous testing efforts.

Module 5: Collaboration-based Test Approaches



Collaboration-based Test Approaches

- Collaboration-based test approaches in software testing refer to methodologies that emphasise cooperation, communication, and teamwork among various stakeholders involved in the testing process.
- These approaches recognise the value of collaboration in improving the effectiveness and efficiency of testing activities.
- In collaboration-based test approaches, testers work closely with developers, business analysts, project managers, and other relevant stakeholders throughout the testing lifecycle.
- The goal is to foster collaboration and knowledge sharing to ensure better-quality software.

The importance of collaboration-based test approaches in software development cannot be overstated. These approaches, which include methodologies like pair testing, mob testing, and participatory design, foster a culture of collective responsibility for the quality of the software. By bringing together diverse perspectives—be it developers, testers, business analysts, or end-users—collaboration-based testing enriches the test design and execution process, uncovering a broader range of potential issues and solutions.

Collaboration-based Test Approaches

Collaborative User Story Writing

- Collaborative user story writing in software testing refers to the involvement of testers in the process of creating and refining user stories.
- It emphasises the collaboration between testers, developers, business analysts, and other stakeholders to ensure that testing considerations are taken into account right from the early stages of requirements definition.
- Writing user stories in collaboration with developers and business representatives is a crucial aspect of software testing and development. The process involves effective communication and cooperation between all stakeholders to ensure that the user stories accurately capture the needs and expectations of end-users.

© The Knowledge Academy Ltd

Example: Developing a File Upload Feature for an Online Document Management System

- **Role:** Team including Business Analysts, Developers, Testers, and Stakeholders
- **Goal:** Creating a user story for the file upload feature

Process

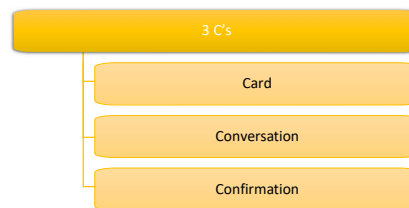
- **Card:** The team starts by jotting down the basic idea of the user story on an index card: "As a user, I want to upload files, so that I can manage my documents online."
- **Brainstorming Session:** A meeting is called where all team members collaborate, using techniques like brainstorming and mind mapping. The objective is to understand the feature from three perspectives - business, development, and testing.
- **Conversation:** Through open dialogue, the team discusses how the software will be used. Questions like "What types of files can be uploaded?" and "How will the files be organised once uploaded?" are addressed. This conversation may be documented or remain verbal.
- **Confirmation:** The team defines the acceptance criteria, guided by section 4.5.2. For this user story, the criteria may include: file size limitations, supported file types, upload speed, etc.
- **Adhering to INVEST Principles:** The user story is assessed for being Independent, Negotiable, Valuable, Estimable, Small, and Testable. For example, it must be clear, valuable to the user, and have testable outcomes.
- **Stakeholder Involvement:** If a stakeholder is unsure how to test the user story, this is addressed by clarifying the user story or providing assistance in understanding the testing process (Wake 2003).
- **Final User Story (shared vision):** "As a user, I want to upload files of supported types and within defined size limits, so that I can manage my documents online," followed by the detailed acceptance criteria.

Through collaborative authorship, the team ensures that the user story for the file upload feature is robust, clear, and aligned with the needs of the business, development, and testing perspectives. It leverages the "3 C's" framework (Card, Conversation, Confirmation) and adheres to best practices to create a user story that guides successful feature development.

Collaboration-based Test Approaches

(Continued)

- By collaborating effectively, developers and business representatives can align their understanding of user needs and technical requirements. This collaborative approach in writing user stories fosters a shared vision and leads to a more successful software testing and development process, resulting in a product that meets users' expectations and fulfills business goals.
- **The following are the three critical aspects of user stories called the 3 C's:**



© The Knowledge Academy Ltd

Card: The Card represents the physical or digital artifact that captures the user story. It typically includes a short, concise statement that describes the user's goal or need. The card serves as a placeholder for the user story and provides a high-level summary that can be easily understood. It should be simple and clear, using a user-centric language to express the desired outcome.

Conversation: The Conversation refers to the discussion and collaboration among the development team, product owner, and stakeholders. It is an opportunity for all involved parties to clarify and elaborate on the user story. During this collaborative conversation, team members can ask questions, seek clarification, and provide input on the details and acceptance criteria of the user story. This dialogue helps ensure a shared understanding and alignment among the team members.

Confirmation: The Confirmation stage focuses on defining the acceptance criteria for the user story. Acceptance criteria outline the specific conditions that must be met for the user story to be considered complete and done. They serve as the basis for testing and validating the functionality associated with the user story. The confirmation process involves reaching a shared agreement on the acceptance criteria to ensure that everyone has a clear understanding of the expected outcome and success criteria.

Collaboration-based Test Approaches

INVEST

- INVEST is an acronym which assists to remember an accepted set of criteria, or checklist, to assess the quality of a user story.
- The team may want to reword it, or even consider a rewrite, if the story fails to meet one of these criteria, the team may want to reword it, or even consider a rewrite (which often translates into physically tearing up the old story card and writing a new one).

Independent Standalone PBI with no dependencies.

Negotiable It can be changed in anytime.

Valuable Having a good value for the end user.

Estimable The team is able to estimate its size.

Small Small enough to be developed and tested.

Testable Testing is possible from AC and DOD.

Collaboration-based Test Approaches

INVEST

- **Independent:** Each user story should stand alone and be able to be developed independently of other stories. This independence ensures that the progression of one story doesn't hinder or depend on the development of another, allowing for greater flexibility and parallel development.
- **Negotiable:** A user story should be concise and not set in stone. It serves as a conversation starter between stakeholders and development teams rather than a strict contract. The details should be negotiated collaboratively, allowing for adjustments and refinements as understanding grows or requirements change.
- **Valuable:** Every user story must deliver value to the customer or end-user. If the story doesn't contribute to something useful or meaningful, it can lead to wasted effort. Ensuring that each story is valuable helps in prioritising the work and keeping the focus on what's most important to the stakeholders.

© The Knowledge Academy Ltd

INVEST is a set of criteria designed to ensure high-quality test cases. It stands for Independent (tests can run separately), Negotiable (flexible for changes), Valuable (provides clear benefit), Estimable (time to execute can be predicted), Small (concise and focused), and Testable (outcome can be clearly checked). This approach helps testers create effective, efficient, and reliable tests for software products.

Collaboration-based Test Approaches

INVEST

- **Estimable:** It's essential for a team to be able to estimate the size and complexity of a user story. Without an estimation, planning and scheduling become challenging. An estimable story provides enough clarity and detail to allow the team to gauge the effort required, without being overly prescriptive.
- **Small:** Keeping user stories small aids in faster development and testing. A small story can be completed within a single iteration, making it easier to manage and allowing for more incremental progress. If a story is too large, it might need to be broken down into smaller, more manageable parts.
- **Testable:** A well-written user story should be testable. This means that there should be clear acceptance criteria to determine whether the story has been successfully implemented.

Collaboration-based Test Approaches

Acceptance Criteria

- Acceptance criteria are the conditions that an implementation of a user story must fulfill to be considered acceptable by the stakeholders.
- They act as the standard against which the feature or functionality described in the user story is evaluated.
- From a testing perspective, acceptance criteria can be seen as the test conditions that must be met during the testing process.
- The formulation of acceptance criteria usually emerges from the "Conversation" aspect of the user story, where stakeholders, developers, and other team members discuss and clarify the expectations for the user story.

Collaboration-based Test Approaches

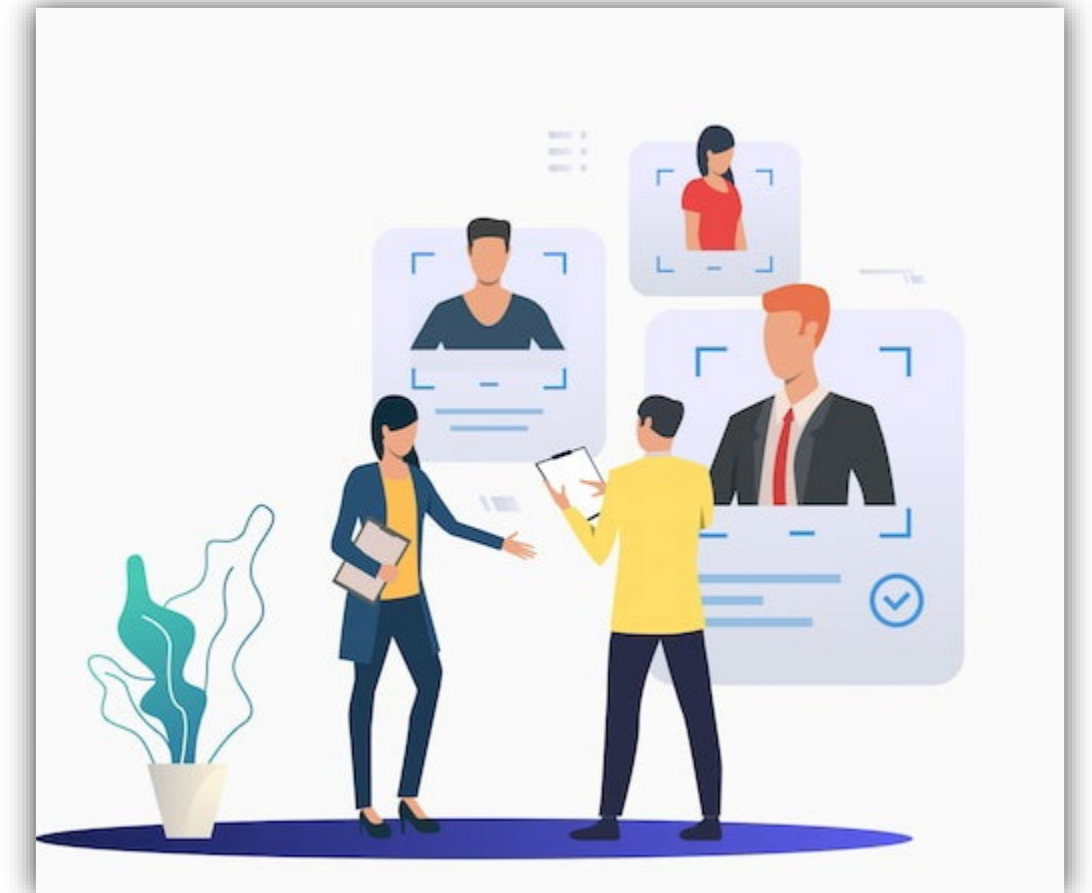
Components of Acceptance Criteria

- **Conditions of Satisfaction:** These are the key functionalities and behaviours that stakeholders expect from the feature. They are clearly defined so that everyone understands what success looks like for that particular user story.
- **Positive and Negative Scenarios:** Acceptance criteria cover both what should happen (positive scenarios) and what should not happen or how the system should respond to erroneous inputs (negative scenarios).
- **Boundaries and Constraints:** They define the limitations and constraints of the implementation, often including the technical, regulatory, or business boundaries that the solution must adhere to.

Collaboration-based Test Approaches

Acceptance Criteria serve several critical roles in the development process:

- 1. Defining the Scope:** By clearly stating the conditions that must be met, acceptance criteria outline the boundaries of the user story. They help in understanding what is to be achieved without ambiguity.
- 2. Reaching Consensus:** Through detailed and well-articulated acceptance criteria, stakeholders reach a mutual understanding of what is expected from the implementation. This consensus helps to align expectations and avoid potential conflicts or misunderstandings later in the process.



Collaboration-based Test Approaches

3. **Describing Scenarios:** Acceptance criteria detail both positive scenarios (how things should work under normal circumstances) and negative scenarios (how the system should respond to incorrect input or unexpected user behaviour). This comprehensive approach ensures that all relevant aspects of the user story are addressed.
4. **Serving as a Basis for Testing:** Acceptance criteria form the foundation for acceptance testing, a vital phase in the software development lifecycle. During acceptance testing, the software is tested against these criteria to determine if it meets the expected standards. If the software satisfies all the outlined conditions, it is considered to have passed this phase.
5. **Allow Accurate Planning and Estimation:** In software testing, accurate planning defines clear testing objectives, scope, and timelines. Estimation determines the resources and effort required. Together, they ensure efficient testing and optimised resource allocation. Within ISTQB's framework, these elements are crucial for aligning the product with user expectations and business goals, driving successful test cycles.

Collaboration-based Test Approaches

What are the several ways to write acceptance criteria in software testing?

There are several ways to write acceptance criteria in software testing, and the choice depends on the complexity and nature of the feature being tested. Some common ways include:

- **Plain Language:** Writing acceptance criteria in simple, understandable language that directly states the expected behaviour of the software.
- **Scenario-based:** Describing acceptance criteria in terms of specific user scenarios or use cases to test how the feature behaves in different situations.
- **Given-When-Then:** Following the Given-When-Then format to structure acceptance criteria, where Given sets the context, When specifies the action, and Then states the expected outcome.
- **Business Rules:** Defining acceptance criteria as explicit business rules or constraints that the software must adhere to.

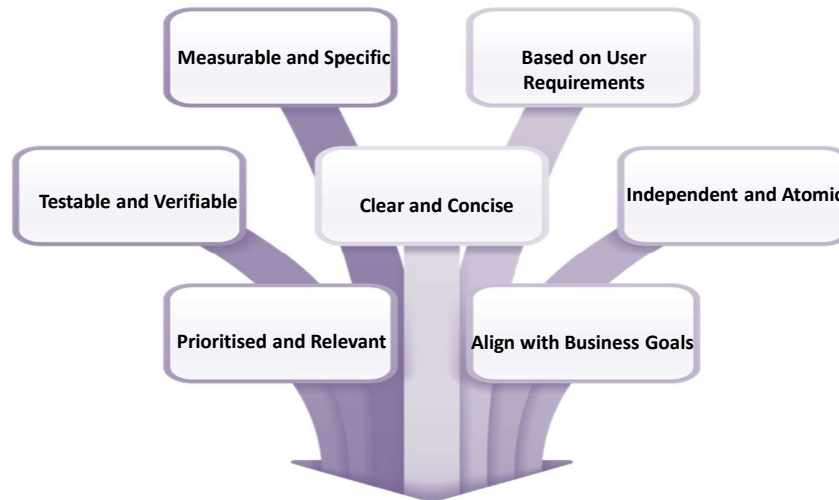
© The Knowledge Academy Ltd

Examples of the different ways to write acceptance criteria in software testing:

- **Plain Language:** "The user should be able to log in using a valid email and password."
- **Scenario-based:** "Scenario: User places an order with items in the cart. Acceptance Criteria: When the user clicks 'Place Order,' the total order cost should be calculated correctly, and the inventory levels should be updated accordingly."
- **Given-When-Then:** "Given the user is on the homepage, when they click the 'Add to Cart' button for a product, then the product should be added to their shopping cart."
- **Business Rules:** "The discount should be applied automatically when the user purchases more than three items from the 'Sale' category."

Collaboration-based Test Approaches

- *Here are some key characteristics of acceptance criteria:*



© The Knowledge Academy Ltd

1. **Measurable and Specific:** Acceptance criteria should be precise and quantifiable, providing clear indicators of whether the feature has been successfully implemented. Vague or subjective criteria should be avoided to ensure objective evaluation.
2. **Based on User Requirements:** Acceptance criteria are derived from the user's perspective and reflect their needs, goals, and expectations. They focus on how the feature should function and the value it should provide to the user.
3. **Testable and Verifiable:** Acceptance criteria should be testable, allowing testers to validate whether the feature meets the defined criteria. They should be verifiable through objective means, such as specific test cases or observations.
4. **Clear and Concise:** Acceptance criteria should be easy to understand and communicate. They should be written in a concise manner to avoid ambiguity and ensure a shared understanding among all stakeholders.
5. **Independent and Atomic:** Acceptance criteria should be independent of each other, meaning they can be evaluated individually. They should also be atomic, focusing on one specific aspect of the feature rather than combining multiple requirements into a single criterion.
6. **Prioritised and Relevant:** Acceptance criteria should be prioritised based on their importance and impact on the overall functionality. They should focus on the most critical aspects and avoid unnecessary or trivial criteria.
7. **Align with Business Goals:** Acceptance criteria should be aligned with the business goals and objectives of the software. They should reflect the desired outcomes and address the key success factors identified by the stakeholders.

Collaboration-based Test Approaches

Acceptance Test-driven Development (ATDD)

- Acceptance Test-driven Development (ATDD) is an agile development approach that emphasises collaboration among developers, testers, and business stakeholders to define and validate software requirements through the creation of acceptance tests.
- ATDD extends the principles of Test-driven Development (TDD) by incorporating the perspectives of business stakeholders and focusing on the acceptance criteria or desired outcomes of the software. It promotes shared understanding, collaboration, and early validation of requirements.
- **How to write ATDD:**
 - Collaborative Requirements Gathering
 - Formulating Acceptance Criteria
 - Acceptance Test Creation
 - Automate Acceptance Tests
 - Test Execution and Validation
 - Continuous Collaboration and Feedback
 - Continuous Improvement

© The Knowledge Academy Ltd

- **Collaborative Requirements Gathering:** Developers, testers, and stakeholders collaborate to gather detailed requirements in the form of user stories or features. These requirements should be clear, unambiguous, and represent the functionality to be implemented.
- **Formulating Acceptance Criteria:** Based on the requirements, the team defines acceptance criteria. These criteria outline the specific conditions that must be met for each requirement to be considered fulfilled. Acceptance criteria serve as a basis for creating acceptance tests.
- **Acceptance Test Creation:** In ATDD, acceptance tests are written collaboratively by the developers, testers, and stakeholders. The tests are designed to validate that the software meets the specified acceptance criteria and fulfills the intended user requirements. These tests are usually written in a business-readable, domain-specific language to ensure easy comprehension by all stakeholders.
- **Automate Acceptance Tests:** The acceptance tests created in the previous step are automated using suitable testing frameworks and tools. Automation enables the continuous execution of tests throughout the development process, ensuring quick feedback on the software's quality.
- **Test Execution and Validation:** As development progresses, the automated acceptance tests are executed regularly. When new code is integrated, the tests run to verify that the implemented features meet the acceptance criteria. Any deviations or defects are identified and addressed promptly.
- **Continuous Collaboration and Feedback:** Throughout the development cycle, developers, testers, and stakeholders continue to collaborate, discuss progress, and provide feedback. This feedback loop helps refine the requirements, tests, and implementation, fostering an iterative and incremental development process.
- **Continuous Improvement:** The collaborative approach encourages continuous improvement. The team reflects on the testing process, explores better ways to validate requirements, and identifies opportunities to enhance testing efficiency and effectiveness.

Collaboration-based Test Approaches

(Continued)

- The benefits of ATDD include improved collaboration, shared understanding of requirements, early identification of discrepancies, reduced rework, and faster feedback loops.
- It helps ensure that the developed software meets the expectations of business stakeholders and aligns with their desired outcomes.
- ATDD promotes a customer-centric approach to development, where requirements are validated through executable tests, leading to a higher degree of confidence in the software's functionality and quality.

Example 1: E-Commerce Website Checkout Feature

Situation:

- ✓ An e-commerce company wants to enhance its checkout process to improve user experience and reduce cart abandonment rates.

ATDD Approach:

- **User Story:** As an online customer, I want to receive a confirmation message once my purchase is complete so that I know my transaction was successful.
- **Acceptance Criteria:**
 - A confirmation page is displayed after the purchase is complete.
 - The confirmation page shows the order number, purchase date, and items purchased.
 - An email is sent to the customer with the order details.

Example 1: E-Commerce Website Checkout Feature

Development:

- ✓ The development team, along with business analysts and QA specialists, collaborates to define the acceptance tests based on the acceptance criteria.
- ✓ The QA team writes the acceptance tests before the feature is developed.
- ✓ Developers create the checkout feature to pass these predefined acceptance tests.

Outcome:

- ✓ The feature is only considered complete once it passes all acceptance tests, ensuring that it meets the customer's requirements.

Example 2: Hospital Management Software

Situation:

- ✓ A hospital is developing a new patient registration system to streamline the process of admitting patients and managing their records.

ATDD Approach:

- **User Story:** As hospital staff, I want to easily register new patients and access their records so that I can reduce waiting times and improve patient care.
- **Acceptance Criteria:**
 - The system allows staff to enter all necessary patient information through a simple interface.
 - Patient records are created instantly and are easily accessible.
 - Duplicate records trigger an alert to the staff.

Example 2: Hospital Management Software

Development:

- ✓ The ATDD process starts with defining clear acceptance criteria in collaboration with healthcare professionals and administrative staff.
- ✓ Acceptance tests are written prior to development.
- ✓ The development is guided by these tests to ensure all features meet the defined criteria.

Outcome:

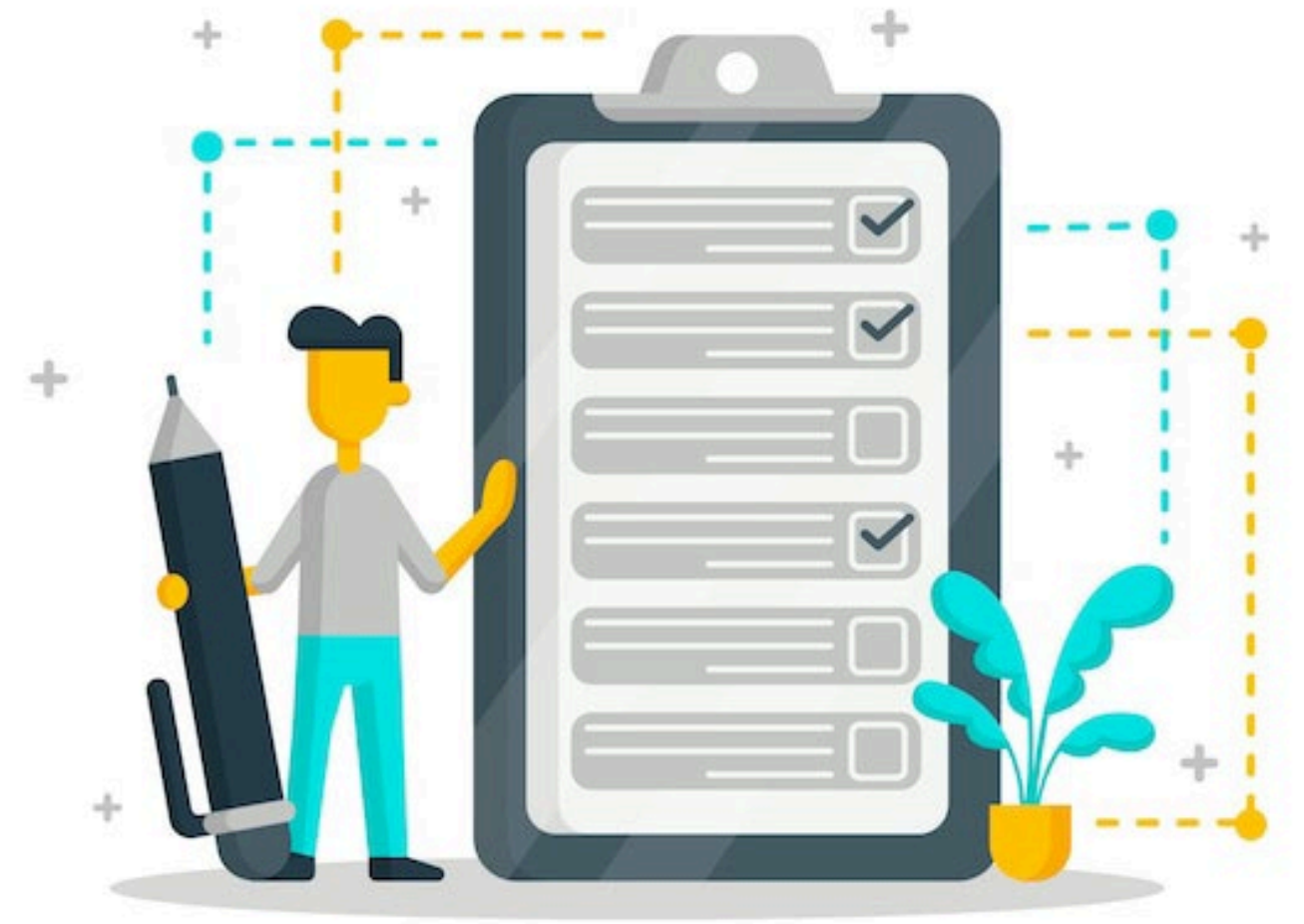
- ✓ The system is implemented successfully, having met all the acceptance tests, leading to improved efficiency and patient satisfaction.

These examples show how ATDD is used to bridge the gap between the understanding of the business requirements and the technical implementation, ensuring that the final product truly meets the needs of its users.

Exercise

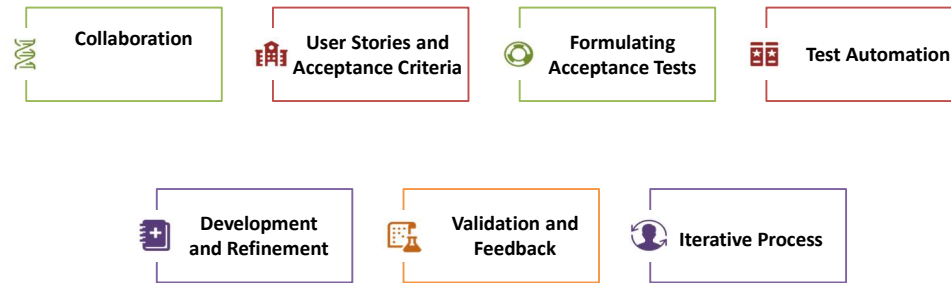
To dive deeper into practical application, refer to the corresponding exercise in the document for hands-on exploration and analysis.

Exercise 8: E-commerce Checkout Process



Collaboration-based Test Approaches

- Here are the key components and steps involved in ATDD:



© The Knowledge Academy Ltd

- 1. Collaboration:** ATDD emphasises close collaboration between developers, testers, and business stakeholders, including product owners or subject matter experts. This collaboration ensures that all perspectives are considered in defining requirements.
- 2. User Stories and Acceptance Criteria:** The development team works together to create user stories that describe specific functionality or features from the user's perspective. Acceptance criteria are defined as measurable conditions that indicate when a user story is considered complete and meets the expected outcomes.
- 3. Formulating Acceptance Tests:** Acceptance tests are created collaboratively by the development team, testers, and business stakeholders. These tests are written in a business-readable language that specifies the expected behavior of the software. They serve as a shared understanding of the requirements and form the basis for development and testing.
- 4. Test Automation:** Once the acceptance tests are defined, they are automated using appropriate testing frameworks and tools. This automation ensures that the acceptance tests can be easily executed and repeated throughout the development process.
- 5. Development and Refinement:** Developers use the acceptance tests as a guide to implement the necessary code. They focus on writing code that satisfies the acceptance tests, thereby ensuring that the software meets the defined requirements.
- 6. Validation and Feedback:** Testers execute the acceptance tests to validate the implemented functionality against the acceptance criteria. Any discrepancies or failures in meeting the acceptance criteria are identified as potential defects and communicated to the development team for resolution.
- 7. Iterative Process:** ATDD follows an iterative and incremental development approach. After validating a set of acceptance tests, the team incorporates feedback, refines the requirements if necessary, and proceeds to develop the next set of functionality.

Domain 5: Managing the Test Activities

Outline of Domain 5

- *Module 1: Test Planning*
- *Module 2: Risk Management*
- *Module 3: Test Monitoring, Test Control and Test Completion*
- *Module 4: Configuration Management*
- *Module 5: Defect Management*

Module 1: Test Planning



Purpose and Content of a Test Plan

- A test plan serves as a comprehensive document that outlines the purpose, scope, objectives, and approach of software testing activities for a specific project.
- Its purpose is to provide guidance and structure to the testing process, ensuring that all necessary testing activities are identified, planned, and executed effectively.
- ***The content of a test plan typically includes the following key elements:***

Introduction

- Provides an overview of the project, its objectives, and the role of the test plan in achieving those objectives.

Test Objectives

- Clearly defines the goals and objectives of the testing effort, outlining what is to be achieved through the testing activities.

Test planning is a critical aspect of the software development and testing lifecycle. A well-structured test plan serves as a blueprint that guides the testing team through each phase of the testing process.

Reasons why test planning is crucial:

- Clear Objectives
- Comprehensive Scope
- Resource Allocation
- Risk Identification and Mitigation
- Timeline and Milestones

Purpose and Content of a Test Plan

(Continued)

Test Scope

- Defines the boundaries of the testing effort by specifying what will be included and excluded from testing.

Test Approach

- Describes the overall strategy and approach for conducting the testing, including the types of tests to be performed (e.g., functional, performance, security), the testing techniques and methodologies to be employed, and any specific tools or resources to be utilised.

Purpose and Content of a Test Plan

(Continued)

Test Deliverables

- Lists the specific documents and artifacts that will be produced as part of the testing process, such as test plans, test cases, test scripts, and test reports.

Test Schedule

- Outlines the timeline and sequencing of the testing activities, including key milestones and dependencies on other project activities.

Test Environment

- Describes the hardware, software, and network configurations required to support the testing efforts, including any necessary test data and test environments setup.

Purpose and Content of a Test Plan

A typical test plan will comprise the following items:

- Product Analysis
- Designing Test Strategy
- Defining Objectives
- Establish Test Criteria
- Planning Resource Allocation
- Planning Setup of Test Environment
- Determine test schedule and estimation
- Establish Test Deliverables



© The Knowledge Academy Ltd

- **Product Analysis:** Evaluating the software product's features and functionalities to identify what needs to be tested.
- **Designing Test Strategy:** Outlining the overall approach and methodologies that will be used for software testing.
- **Defining Objectives:** Stating the specific goals that the testing process aims to achieve, such as finding bugs or validating performance.
- **Establish Test Criteria:** Setting the conditions or standards that must be met for the software to pass the testing phase.
- **Planning Resource Allocation:** Identifying and assigning the necessary resources like manpower, hardware, and software for the testing process.
- **Planning Setup of Test Environment:** Detailing the hardware and software configurations needed to create a test environment that mimics the production system.
- **Determine Test Schedule and Estimation:** Establishing a timeline that includes all testing activities, milestones, and estimated time for completion.
- **Establish Test Deliverables:** Defining the documents, tools, and other outputs that will be produced as part of the testing process, such as test cases, reports, and logs.

Tester's Contribution to Iteration and Release Planning

Iteration and Release Planning

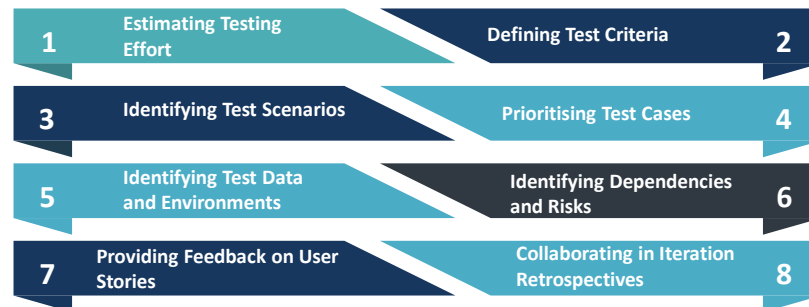
- Release planning is a longer-term view that focuses on the entire release of a product. It includes defining and continuously refining the product backlog, breaking down larger user stories, and setting the foundation for the testing approach across all iterations.
- Iteration planning zooms into a single iteration, considering a specific subset of the product backlog referred to as the iteration backlog. It involves planning the tasks and testing for a particular development cycle or "sprint."
- Release planning focuses on the broader scope, looking at the overall release of a product, whereas Iteration planning narrows down to individual development cycles. In both these phases, testers play a vital role, contributing to the creation of user stories, risk analysis, estimation, and planning, to ensure a smooth and robust testing process.

Interconnection between Iteration and Release Planning

Iteration planning is nested within the broader context of release planning. While release planning provides a macro-level view of what needs to be achieved in the long run, iteration planning focuses on the micro-level tasks that need to be completed in the short term. Each iteration contributes to the goals set during release planning, incrementally building up to the final product.

Tester's Contribution to Iteration and Release Planning

- Under a test plan, testers play a crucial role in iteration and release planning by providing valuable insights and inputs related to testing activities and requirements.
- Their contributions can include:**



© The Knowledge Academy Ltd

- Estimating Testing Effort:** Testers can provide estimates for the effort required to execute different types of tests, such as functional, performance, or security testing. By considering factors like test case complexity, test data preparation, and test environment setup, testers can help determine the time and resources needed for testing activities.
- Defining Test Criteria:** Testers can collaborate with stakeholders to define criteria for determining when a particular iteration or release is ready for testing. This includes identifying the entry criteria that must be met before testing can commence and the exit criteria that signify the completion of testing and readiness for deployment.
- Identifying Test Scenarios:** Testers can actively participate in discussions and workshops to identify test scenarios based on user stories, requirements, and acceptance criteria. Their expertise in testing techniques and domain knowledge can help ensure comprehensive test coverage for each iteration or release.
- Prioritising Test Cases:** Testers can provide insights on prioritising test cases based on risk analysis, business impact, and criticality. They can identify high-priority test cases that require immediate attention and ensure that critical functionalities or areas are thoroughly tested before the release.
- Identifying Test Data and Environments:** Testers can collaborate with developers, analysts, and other team members to identify and prepare the necessary test data and test environments for each iteration or release. They can provide input on data variations, boundary values, and different configurations to ensure realistic and effective testing.
- Identifying Dependencies and Risks:** Testers can contribute by identifying dependencies on external systems, interfaces, or components that may impact testing. They can also help identify potential risks and challenges specific to testing activities and suggest mitigation strategies or contingency plans.
- Providing Feedback on User Stories:** Testers can review user stories and acceptance criteria from a testing perspective, providing feedback on clarity, testability, and potential gaps. Their input can help refine user stories and ensure that they are well-defined and testable.
- Collaborating in Iteration Retrospectives:** Testers can actively participate in iteration retrospectives, sharing their observations, challenges faced, and lessons learned during testing. Their insights can contribute to process improvements, identify bottlenecks, and enhance the effectiveness and efficiency of future iterations.

Entry Criteria and Exit Criteria

Test Plan Outline

Master Test Plan

- Definition of test levels
- Responsibilities for each
- Entry criteria for each
- Exit criteria for each

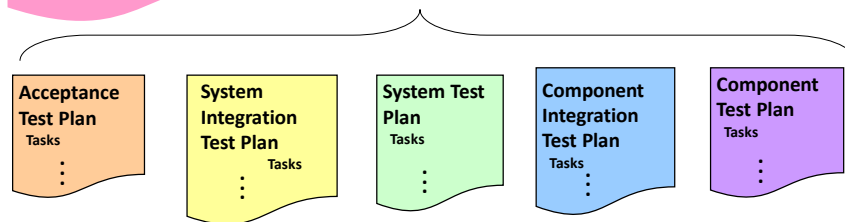
Entry Criteria

Define when to start testing

Exit Criteria

Define when to stop testing

Test Level Plans



© The Knowledge Academy Ltd

In a test plan, Entry Criteria and Exit Criteria serve as crucial guardrails that guide the software testing process from initiation to closure. These criteria contribute to the plan's effectiveness by setting clear conditions for when testing activities should start and conclude, thereby ensuring a well-organised and controlled testing phase.

Together, Entry and Exit Criteria in a test plan act as key milestones in the software testing lifecycle. They provide a structured framework for initiating and completing testing activities, thereby enhancing the efficiency, effectiveness, and reliability of the entire testing process.

Entry Criteria and Exit Criteria

Typical Entry Criteria – Ready ?

- Availability of testable requirements, user stories, and/or models (e.g., when following a model-based testing strategy)
- Availability of test items that have met the exit criteria for any prior test levels
- Availability of test environment
- Availability of necessary test tools
- Availability of test data and other necessary resource

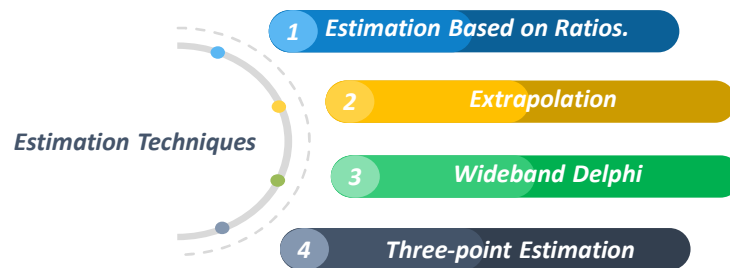
Entry Criteria and Exit Criteria

Typical Exit Criteria – Ready ?

- Planned tests have been executed
- A defined level of coverage (e.g., of requirements, user stories, acceptance criteria, risks, code) has been achieved
- The number of unresolved defects is within an agreed limit
- The number of estimated remaining defects is sufficiently low
- The evaluated levels of reliability, performance efficiency, usability, security, and other relevant quality characteristics are sufficient
- Cost/time

Estimation Techniques

- Estimation techniques involve various approaches to determine the effort, resources, and time required for testing activities.
- These techniques include expert judgment, historical data analysis, parametric estimation, three-point estimation, and analogous estimation. Expert judgment leverages the insights and experience of knowledgeable professionals, while historical data analysis examines past project data for patterns and trends.



© The Knowledge Academy Ltd

Estimation Based on Ratios

Estimation based on ratios is a technique used to predict or estimate certain values or quantities by analysing the relationship between different variables or ratios. It involves using known ratios to make inferences or projections about unknown values.

Extrapolation

Extrapolation is an estimation technique that involves extending or projecting existing data or trends beyond the observed range to estimate values outside of that range. It is based on the assumption that the observed pattern or trend will continue in the future.

Wideband Delphi

The Wideband Delphi technique is a structured approach used to obtain consensus from a group of experts or stakeholders in order to estimate project parameters or make decisions. It is commonly used in project management, software development, and risk management.

Three-point Estimation

Three-point estimation is a technique used in project management to estimate the duration, effort, or cost of an activity or task. It is based on the idea that a single-point estimate may not accurately reflect the uncertainties and risks associated with the task. Three-point estimation incorporates three estimates: the most optimistic estimate (a), the most pessimistic estimate (b), and the most likely estimate (m). The final estimate (E) is their weighted arithmetic mean.

Example

- Estimation techniques are critical for project planning in a variety of industries, from software development to construction and beyond.
- Here are real-life examples illustrating how each of these techniques can be applied:

1. Estimation Based on Ratios: Software Development Project

- ✓ In a software development project, a project manager may use ratio-based estimation to determine the effort required for testing.
- ✓ If historical data shows that testing typically takes about 30% of the total development time, and the upcoming project is estimated to require 300 hours of development work, the testing can be estimated to take approximately 90 hours.
- ✓ This estimation technique assumes that the complexity and scope of the new project are similar to those of past projects where the ratio was applicable.

Example

2. Extrapolation: Real-Life Example: Agile Software Development for a Retail Company

- ✓ A retail company is developing a new inventory management system using an agile development approach. This process involves multiple iterations, each aimed at incrementally adding features to the system.
 - **First Iteration:** Focused on developing a basic inventory tracking feature, completed in 2 weeks.
 - **Second Iteration:** Aimed to add a supplier integration feature. Based on the similar complexity to the first iteration, the team extrapolates that this will also take approximately 2 weeks.
 - **Further Iterations:** Planning involves adding features such as customer order integration and predictive stock levels. Given the team's growing familiarity with the project and incremental complexity, the team continues to use extrapolation for each new iteration. They anticipate slight variations in duration but expect to maintain a similar pace, adjusting their estimates as they gather more data from completed iterations.

Example

3. Wideband Delphi: Real-Life Example: New Product Development

- ✓ A tech company is developing a new smartphone with features that have not been implemented before.
- ✓ The project manager organises a Wideband Delphi session with a panel of experts, including engineers, designers, and product managers, to estimate the time required for product development.
- ✓ Each expert makes an initial estimate, followed by discussion and several rounds of re-estimation until the group reaches a consensus on the development timeframe.

Example

4. Three-point Estimation: Real-Life Example: Event Planning

- ✓ An event planning company is preparing for a large conference. To accurately estimate the total hours required to manage the project from inception to completion, they employ the Three-point estimation technique.
- ✓ Estimates Provided:
 - Optimistic (O): 120 hours (everything goes as planned)
 - Most Likely (M): 150 hours (the expected realistic scenario)
 - Pessimistic (P): 200 hours (encountering several unexpected issues)
- ✓ Three-point Estimation Calculation:
 - To calculate the Estimated Time (E) and understand the variability (Standard Deviation, SD), the following formulas are used:
 - Estimated Time (E) = $(O + 4M + P) / 6$
 - Standard Deviation (SD) = $(P - O) / 6$

Example

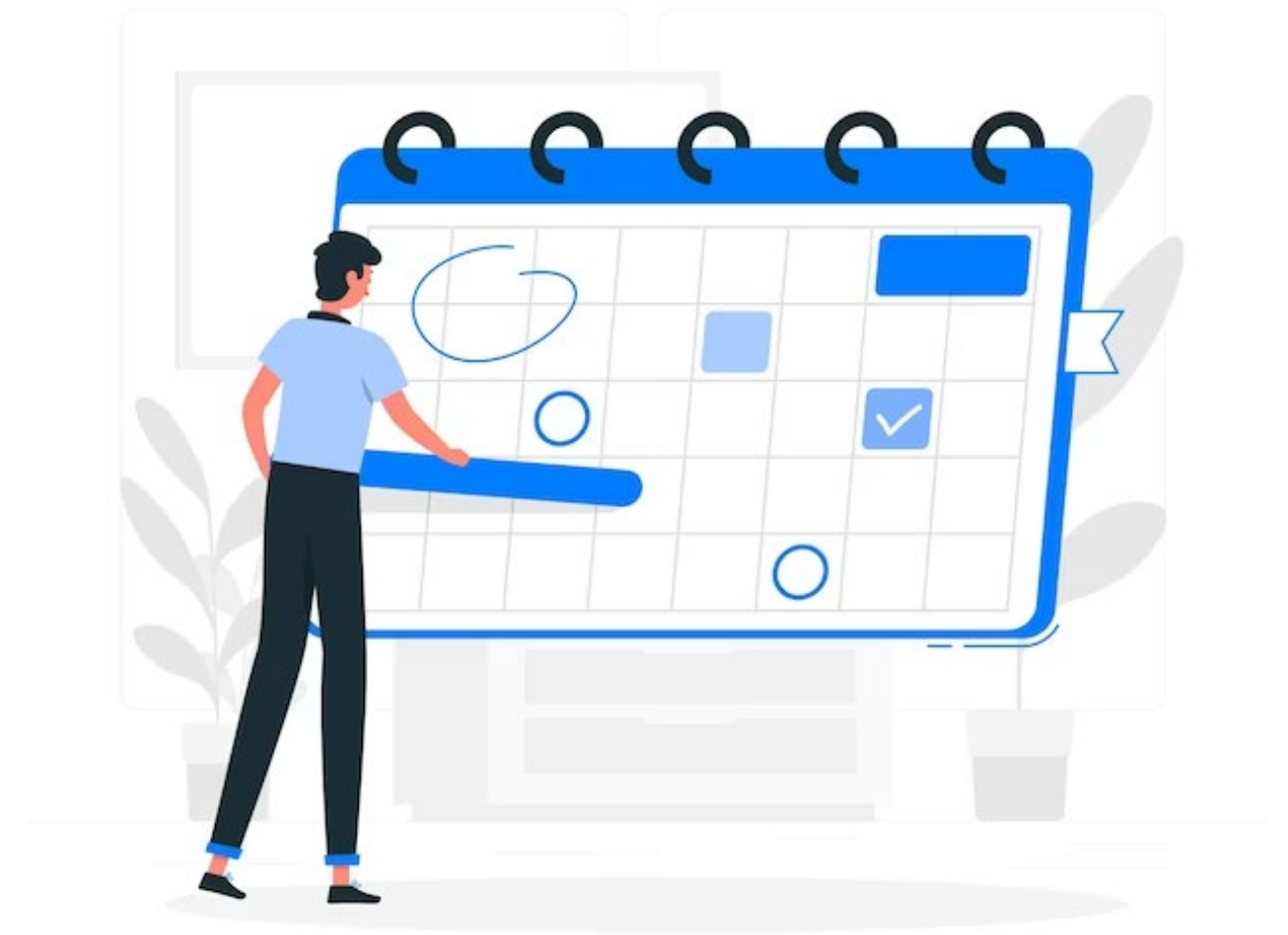
(Continued)

✓ Calculation:

Estimated Time (E) = $(120 + 4 \times 150 + 200) / 6 = 153$ hours

Standard Deviation (SD) = $(200 - 120) / 6 = 13.33$ hours

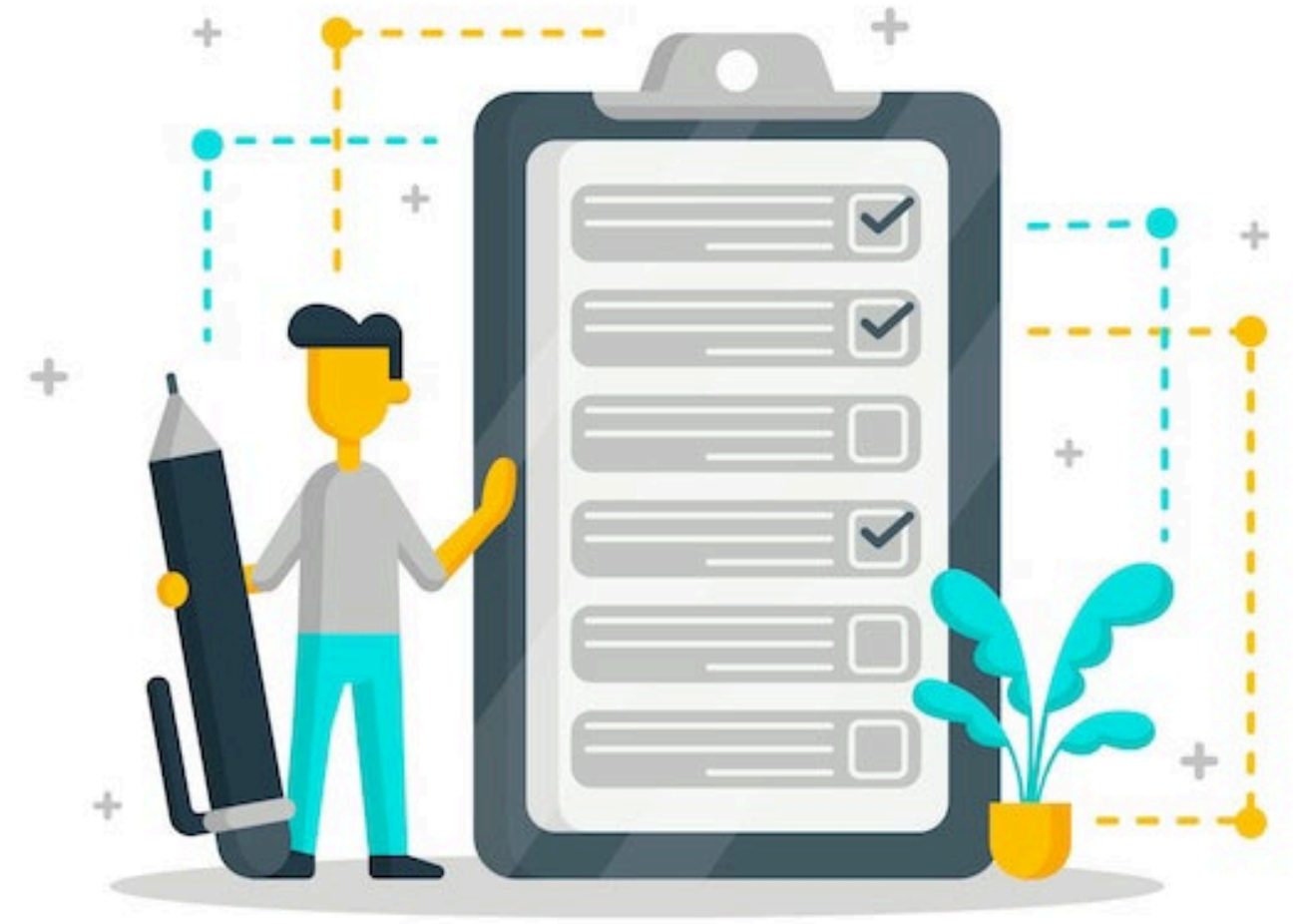
- The calculated estimated time of 153 hours, coupled with a standard deviation of 13.33 hours, equips the event planning team with a robust framework for anticipating project timelines and effectively managing potential variances, thereby enhancing overall project execution.



Exercise

To dive deeper into practical application, refer to the corresponding exercise in the document for hands-on exploration and analysis.

Exercise 9: Mobile App Release



Test Case Prioritisation

- Test case prioritisation is a vital step in test planning where test cases are ordered based on their significance and impact.
- By considering factors such as critical functionality, business value, risk, and customer impact, prioritisation ensures that high-priority test cases are executed early, maximising the chances of identifying critical defects and minimising project risks.
- This approach optimises testing efforts by focusing on the most important areas first and helps ensure that limited resources are utilised effectively, improving the overall quality of the software being tested.
- It enables testing teams to allocate time and resources efficiently by focusing on the most critical test scenarios, ensuring that testing efforts align with project goals and objectives.

Types of Test Case Prioritisation

1. **Business Criticality:** In this approach, test cases that have the most significant impact on business functionality are executed first. These could be features that are directly tied to revenue generation or customer satisfaction.
2. **Severity-Based:** Here, test cases are prioritised based on the severity of the defects they are intended to catch. Critical defects that could lead to system failure are given the highest priority.
3. **Risk-Based:** Risk-based prioritisation focuses on the likelihood and impact of potential risks. Test cases that address high-risk areas of the application are executed before those that focus on lower-risk areas.
4. **Requirement-Based:** In this type, test cases are prioritised based on the importance of the requirements they validate. Key functionalities that are crucial for the product release are given precedence.
5. **Frequency of Use:** Test cases for functionalities that are used more frequently by the end-users are prioritised higher. This ensures that the most commonly used features are robust and reliable.
6. **Complexity:** Test cases that target complex areas of the application are given higher priority. These are areas where the likelihood of defects is higher due to intricate code or complicated business logic.

Test Case Prioritisation

Prioritisation strategies are commonly used

- **Risk-based Prioritisation:** Risk-based prioritisation is a method where test execution follows the risk profile of various software components.
- Tests are arranged in a way that those covering the most significant risks are carried out first.
- This strategy is guided by an analysis of the potential risks, considering aspects like the probability of failure and the impact such failure might have.
- By addressing the most critical risks first, risk-based prioritisation helps detect major defects early in the process, reducing the potential negative impact on the project.

Example: Risk-Based Prioritisation

Situation:

- ✓ A financial services company is developing a new online banking platform.

Approach:

- ✓ High-risk areas, such as transactions, authentication, and data encryption, are identified.
- ✓ Test cases that address these areas are given the highest priority.
- ✓ Lower-risk areas, like the user interface for viewing transaction history, are assigned lower priority.

Example: Risk-Based Prioritisation

Real-Life Implementation:

- ✓ Before each release, the test team prioritises test cases based on the potential impact of a failure on the business and its customers.
- ✓ For example, a test case that validates the completion of a fund transfer would be prioritised over a test case that verifies the correct display of help text on a webpage.



Test Case Prioritisation

- **Coverage-based Prioritisation:** Coverage-based prioritisation is a technique that orders tests based on the coverage of code.
- The test cases achieving the highest statement or branch coverage are executed initially.
- This strategy has a variant called additional coverage prioritisation, where after executing the test that provides the highest coverage, subsequent tests are chosen based on their ability to offer the highest additional coverage.
- The main benefit of coverage-based prioritisation is that it ensures a wide portion of the code base is tested early, leading to broader error detection and improved code quality.

Example: Test Case Prioritisation

Situation:

- ✓ A software company is releasing an update to its graphic design tool, which includes various new features and bug fixes.

Approach:

- ✓ Test cases are ranked based on their code coverage metrics, with those covering the most code (especially newly added code) being given higher priority.
- ✓ The idea is to maximise the coverage of the test suite early in the testing cycle.

Example: Test Case Prioritisation

Real-Life Implementation:

- ✓ Test cases that validate new features such as layer manipulation and font rendering are run first.
- ✓ Test cases that exercise core functionality across the application, impacting multiple features, are also run early in the cycle.
- ✓ Test cases for less frequently used features are executed after the higher coverage test cases have passed.

Test Case Prioritisation

- **Requirements-based Prioritisation:** Requirements-based prioritisation focuses on aligning the order of test execution with the importance of corresponding requirements.
- Tests linked to the most vital requirements, as defined by stakeholders, are conducted first.
- These requirements' priorities are usually determined according to factors like business objectives, customer needs, or regulatory compliance. By targeting the most essential requirements, this approach ensures that testing is aligned with business goals and customer expectations, making the process more efficient and effective.



Example: Requirements-Based Prioritisation

Situation:

- ✓ A healthcare company is updating its patient record management system to comply with new regulations.

Approach:

- ✓ Test cases are mapped directly to specific requirements or user stories, ensuring that the most critical requirements are tested first.
- ✓ Requirements that are crucial for regulatory compliance are considered top priority.

Example: Requirements-Based Prioritisation

Real-Life Implementation:

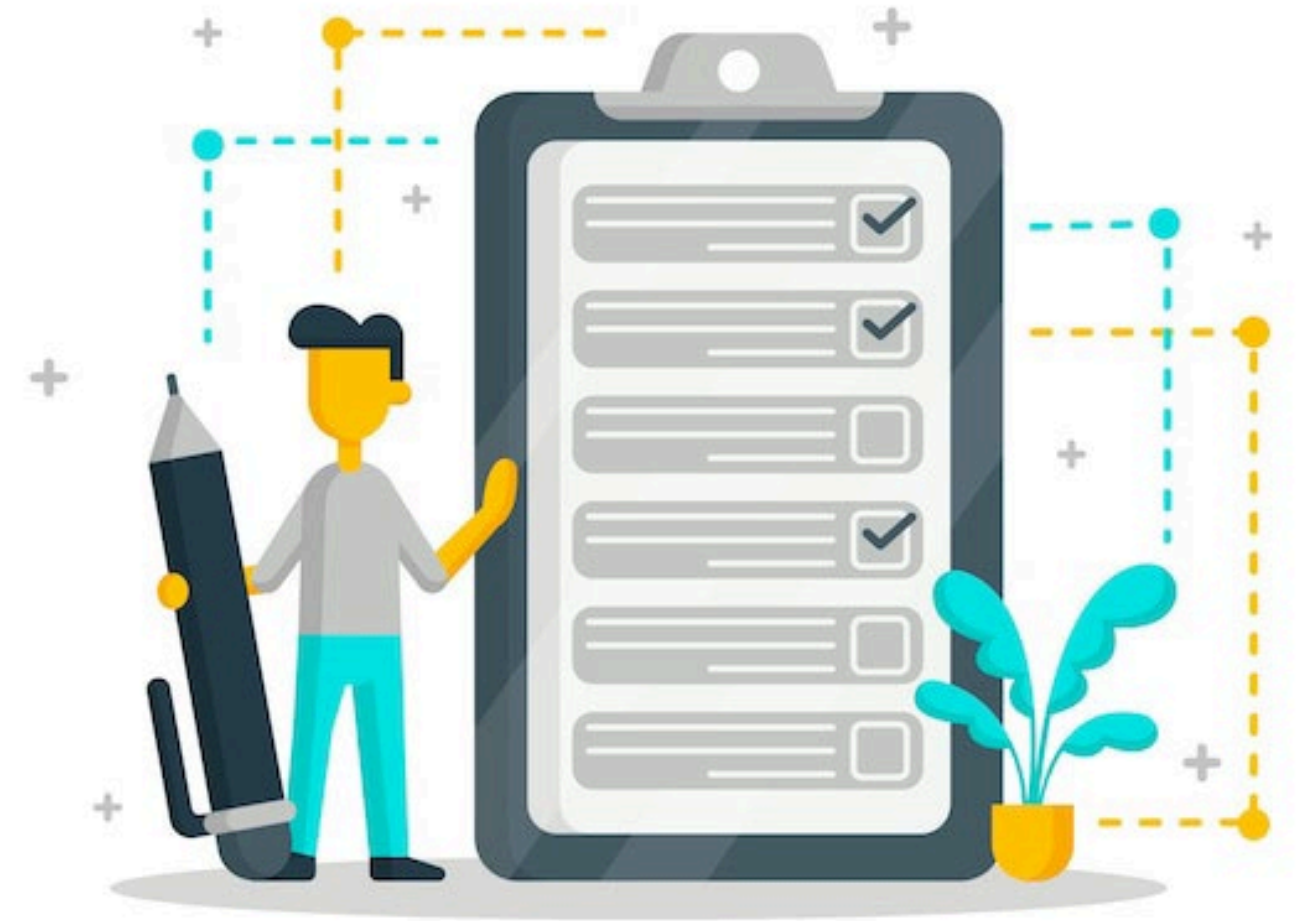
- ✓ Test cases verifying new privacy controls for patient data and reporting features for regulatory bodies are executed before other tests.
- ✓ Test cases for more routine updates, such as changes to the user interface or low-impact feature enhancements, are scheduled for later in the test cycle.

Exercise

To dive deeper into practical application, refer to the corresponding exercise in the document for hands-on exploration and analysis.

Exercise 10: Hospital Information System Upgrade

Exercise 11: Smart Home System Update



Test Pyramid

- *The test pyramid is a model that illustrates the strategy and distribution of different types of automated tests in software development. The structure of the pyramid helps in understanding how to balance the tests for efficiency and effectiveness.*
- *The test pyramid serves as a guide for creating a balanced testing strategy, emphasising a greater number of small, quick tests at the lower levels, and fewer, more comprehensive, and slower tests at the higher levels.*
- *This approach ensures wide coverage of the codebase while maintaining efficiency in the testing process. It's a fundamental concept that aids in understanding where to invest effort and automation to achieve the most effective testing across different stages of development.*

Why Use the Test Pyramid?

- **Balance and Efficiency:** The Test Pyramid helps maintain a balanced and efficient testing suite by promoting the idea that you should have more quick, small tests (unit tests) and fewer slow, large tests (end-to-end tests).
- **Quick Feedback:** With a majority of tests being unit tests, developers get quick feedback on their changes, accelerating development.
- **Reduced Cost:** Unit tests are cheaper to write and maintain compared to end-to-end tests, making the testing process more cost-effective.
- **Simplified Debugging:** Since unit tests focus on small pieces of the codebase, they make it easier to identify and fix issues.
- **Comprehensive Coverage:** The pyramid ensures that testing occurs at multiple levels, from individual components to the entire system, offering a comprehensive approach to quality assurance.
- **Scalability:** Following the Test Pyramid makes the test suite more scalable, as adding features or making changes is less likely to require changes in a large number of expensive end-to-end tests.

Testing Quadrants

- The Testing Quadrants in test planning, provide a structured framework for organising and prioritising different types of tests based on their focus and purpose.
- ***This framework helps testing teams in test planning activities by categorising tests into four quadrants:***
 1. Quadrant Q1
 2. Quadrant Q2
 3. Quadrant Q3
 4. Quadrant Q4

Testing Quadrants

- **Q1** - This quadrant includes tests that are technology-facing and typically executed by developers or testers. It encompasses unit testing, component testing, and API testing, focusing on validating individual units or components in isolation.
- **Q2** - This quadrant involves tests that are business-facing and primarily executed by testers. It includes functional testing, user interface testing, and regression testing, which ensure the software meets the desired functional requirements and behaves as expected from an end-user perspective.
- **Q3** - This quadrant comprises tests that are business-facing but rely on external dependencies, such as databases, web services, or integrations with other systems. It includes tests like performance testing, security testing, and compatibility testing, which assess the system's non-functional aspects, such as performance, security, and compatibility.
- **Q4** - This quadrant consists of tests that are typically carried out by stakeholders, such as business analysts or users. It includes tests like usability testing, acceptance testing, and exploratory testing, which evaluate the software's usability, user experience, and alignment with business needs and expectations.

Module 2: Risk Management



Risk Definition

- Risk can be defined as the potential for negative consequences or adverse events that may arise due to defects, failures, or uncertainties in software or the testing process.
- It involves identifying potential problems or vulnerabilities that could impact the quality, reliability, functionality, security, or performance of the software.

Risk Level

Likelihood (of an adverse event happening)

X

Impact (the harm resulting from that event)

Risk Attributes

A. Severity

- ✓ It refers to the potential impact or seriousness of a risk if it materialises. The severity can vary from minor issues with minimal impact to critical defects that can cause significant harm or damage.

B. Likelihood or Probability

- ✓ It indicates the chance or probability of a risk occurring. It considers factors such as the complexity of the software, the stability of the testing environment, the skills of the testing team, and the presence of similar issues in previous projects.

In the context of risk management, particularly in projects and software development, risk attributes are the characteristics or properties that help in defining, assessing, and understanding a risk. Risk attributes provide a structured way to categorise and analyse the various aspects of identified risks, aiding in more effective risk management strategies.

Project and Product Risks

- Project risks and product risks are two distinct categories of risks commonly encountered in software development and other projects. Project risks are related to the successful completion of the project itself, encompassing aspects such as budget constraints, resource availability, schedule slippage, and communication issues.
- Effective project risk management involves identifying and mitigating these risks to ensure the project stays on track and meets its objectives. On the other hand, product risks pertain to the software product being developed, including functional defects, performance issues, security vulnerabilities, and usability concerns.
- Managing product risks involves thorough testing, code reviews, and continuous monitoring to deliver a high-quality software product that meets user expectations and adheres to regulatory standards.

Project Risk Example

Budget Overrun

- **Risk Description:** The project may exceed its allocated budget due to unforeseen expenses or scope changes.
- **Likelihood:** Medium
- **Impact:** High
- **Mitigation Strategy:** Regular budget tracking and setting aside a contingency fund.

Product Risks Example

Performance Issues

- **Risk Description:** The software may not meet the performance criteria, such as response time or load handling.
- **Likelihood:** Medium
- **Impact:** High
- **Mitigation Strategy:** Conduct performance testing early and optimise code.

Project Risks

- A. Organisational Issues:** These risks revolve around challenges related to the management and coordination of project activities within the organisation. They include delays in the delivery of work products, inaccuracies in project estimates, and attempts to cut costs that might compromise the project's quality or effectiveness.
- B. People Issues:** People-related risks stem from factors involving project team members, stakeholders, and collaborators. These risks may involve insufficient skills or expertise within the team, conflicts among team members, communication breakdowns, or a shortage of qualified staff to fulfill project requirements.
- C. Technical Issues:** Technical risks pertain to the challenges that arise due to the project's technical components and processes. These can include scope creep, where the project requirements expand beyond the initially defined scope, as well as encountering poor tool support, hindering efficient project execution.
- D. Supplier Issues:** Supplier risks are associated with external parties or third-party vendors involved in the project. These risks can involve the failure of a third-party vendor to deliver necessary components or services as expected, or even the unexpected bankruptcy of a supporting company, disrupting the project's progress.

Product Risks

- **Damage to Third Parties:** Defective products can have ripple effects, causing problems for partners, vendors, or clients who rely on the product's functionality.
- **High Maintenance Costs and Helpdesk Overload:** Products with quality issues often require extensive maintenance efforts and may burden the helpdesk with a surge of support requests.
- **Criminal Penalties:** In cases where products fail to meet regulatory standards or cause harm due to negligence, there may be legal consequences, including criminal penalties for the responsible parties.
- **Extreme Consequences:** In the most severe scenarios, product defects can result in physical harm, injuries, or even loss of life. This is especially true in sectors such as medical devices or transportation systems.

Product Risk Analysis

- Product Risk Analysis is a systematic approach to identifying, evaluating, and managing risks associated with a software product's development and implementation.
- It aims to ensure that risks are understood and managed proactively, and it forms an integral part of quality assurance and risk management in software projects.



Product Risk Analysis

- Product risk analysis involves two main components: **Risk Identification** and **Risk Assessment**
- The detection phase focuses on creating an extensive inventory of potential risks, which stakeholders can identify through several methods and tools, such as brainstorming sessions, workshops, interviews, or cause-and-effect diagrams.
- In the evaluation phase, the process includes classifying the identified risks, assessing their probability of occurrence and potential consequences, ranking them according to urgency and severity, and devising strategies for their management.
- The classification is beneficial for allocating mitigative efforts because risks within the same group typically can be addressed with similar mitigation strategies.

Product Risk Analysis: Risk Identification

- Risk identification is a key step in the risk management process where the potential risks that could negatively affect a project or business are systematically identified.
- The aim is to create as comprehensive a list of risks as possible to prepare for and mitigate these risks in later stages of the risk management process.



Product Risk Analysis: Risk Identification

Here is an example of risk identification in the context of software development:

- Suppose a company is about to start a project to develop a new online shopping platform. During the risk identification phase, the project team might identify the following risks:

1. Technical Risks:

- ✓ New technology that the development team is not familiar with could lead to delays. Integrating with existing inventory and payment systems could be more complex than anticipated.

2. Operational Risks:

- ✓ High traffic volumes could overwhelm the server capacity, leading to downtime or slow performance.
- ✓ Third-party logistics providers might not integrate seamlessly with the new system.

Product Risk Analysis: Risk Identification

3. Business Risks:

- ✓ Competitors might launch similar features, affecting the unique selling proposition of the platform.
- ✓ Changes in regulatory requirements for online transactions could occur, requiring additional compliance checks.

4. Project Risks:

- ✓ Key personnel might leave the company, leading to a loss of knowledge and experience.
- ✓ The project timeline may be too aggressive, leading to rushed development and inadequate testing.

5. Financial Risks:

- ✓ The budget allocated may be insufficient due to unforeseen costs.
- ✓ Currency fluctuations could affect the costs of services and resources bought from overseas.

Product Risk Analysis: Risk Identification

6. Security Risks:

- ✓ There could be potential security vulnerabilities in the software that may lead to data breaches.
- ✓ User data might not be properly encrypted, leading to privacy concerns.

7. Legal Risks:

- ✓ There may be intellectual property disputes over the use of third-party libraries or software components.
- ✓ Failing to comply with data protection laws could result in fines and litigation.

By identifying these risks early in the project, the team can plan for risk mitigation strategies, such as providing training for new technologies, ensuring scalable infrastructure, securing intellectual property rights, and setting realistic project timelines. This proactive approach helps minimise the potential impact of these risks on the project's success.

Product Risk Analysis: Risk Assessment

- ✓ Risk assessment is the process in which you determine the qualitative or quantitative value of risk related to a concrete situation and a recognised threat.
- ✓ It involves evaluating the identified risks to understand their potential impact on project objectives and the likelihood of their occurrence.
- ✓ The purpose of risk assessment is to help prioritise risks based on their severity and the likelihood of occurrence, allowing organisations to focus their resources and risk mitigation efforts where they are most needed.



Product Risk Analysis: Risk Assessment

Example of Risk Assessment

- ✓ Let's consider a software development project as an example to illustrate risk assessment:
- 1. Risk Identification:** Imagine a project team has identified several risks to their project, such as:
 - Risk A: Lack of skilled developers in a specific programming language needed for the project.
 - Risk B: Uncertain customer requirements that might lead to frequent changes.
 - Risk C: Third-party vendor dependency for critical project components.
- 2. Risk Analysis:** For each identified risk, the team would analyse and assess:
 - Likelihood (Probability): The chance that a risk will occur.
 - Impact (Consequence): The potential negative effect on the project if the risk occurs.

Product Risk Analysis: Risk Assessment

3. Risk Evaluation:

■ Risk A (Lack of skilled developers):

- **Likelihood:** High - Currently, there is a shortage of developers with the required skills in the job market.
- **Impact:** Medium - The project could face delays, but with enough lead time, developers could be trained or hired.

■ Risk B (Uncertain customer requirements):

- **Likelihood:** Medium - The customer has a history of changing their mind, but there's an agreed-upon requirements management process.
- **Impact:** High - Changes could significantly disrupt the project timeline and budget.

■ Risk C (Third-party vendor dependency):

- **Likelihood:** Low - The vendor has a good track record, but their component is complex.
- **Impact:** High - Without the component, the project can't proceed, and no alternative vendors are available.

Product Risk Analysis: Risk Assessment

4. Risk Prioritisation:

- Based on the above assessment, risks would be ranked. For instance, although Risk C has a low likelihood, its high impact might prioritise it over Risk A which has a high likelihood but only medium impact.
- Thus, the priority for mitigation might be:
 - Risk B - Due to high impact and medium likelihood.
 - Risk C - Due to high impact, despite low likelihood.
 - Risk A - Due to medium impact, even with high likelihood.



Product Risk Analysis: Risk Assessment

5. Risk Mitigation Planning:

- For Risk A, the team might decide to invest in training for existing staff or initiate a recruiting campaign ahead of time.
- For Risk B, the team could engage more frequently with the customer to solidify requirements and include contingency time and budget for changes.
- For Risk C, they might research alternative solutions or work closely with the vendor to understand potential issues and ensure there's a support plan in place.

Risk Control

- Risk control in software testing is the practice of systematically applying procedures to identify, analyse, prioritise, and mitigate risks that may impact the testing process or the quality of the software product.
- It includes implementing strategies to reduce these risks to an acceptable level and continuously monitoring the effectiveness of these strategies.
- This ensures that the testing effort is aligned with project objectives and potential issues are addressed proactively.
- Risk Control involves Risk Mitigation and Risk Monitoring.

Risk Control – Risk Mitigation



- Risk mitigation in software testing is the process of implementing measures to reduce the potential impact of identified risks on the testing process or the software product itself.
- It aims to minimise the likelihood of risk occurrence or to lessen its impact should it occur, ensuring the software's reliability and quality.

Risk Control – Risk Mitigation (Example)

- An example of risk mitigation in software testing could be as follows:
- Suppose a software testing team identifies a risk that a critical piece of the application, such as the payment processing system, may have defects that are not detected before release. To mitigate this risk, the team decides to:
 1. Prioritise and allocate more resources to thoroughly test the payment processing system.
 2. Implement automated regression tests that run after each build to quickly identify any new defects.
 3. Conduct performance and security testing to ensure that the system can handle transactions safely and efficiently under load.
- By taking these steps, the team aims to reduce the probability of releasing a product with critical payment system issues, thus mitigating the risk to the business and its customers.

Risk Control – Risk Monitoring

- Risk monitoring in software testing is the ongoing process of tracking identified risks, reassessing their impact and likelihood over time, and evaluating the effectiveness of risk mitigation actions.
- This is to ensure that new risks are detected early and that the risk management plan is continually updated to reflect the current state of the project.



Risk Control – Risk Monitoring (Example)

- Suppose a software testing team is working on a new release of a mobile application. They've identified a risk that the application might not perform well under high user load, which could lead to slow response times or crashes. As part of risk monitoring, the team would:
 1. **Set Performance Benchmarks:** Establish performance thresholds that the application should meet.
 2. **Implement Monitoring Tools:** Use performance monitoring tools to track the application's behavior under various load conditions.
 3. **Regular Check-Ins:** Schedule regular meetings to review the performance data and discuss any deviations from the expected benchmarks.
 4. **Adjust Test Plans:** If the application begins to show poor performance at lower user loads than expected, the test plan might be adjusted to include more stress testing and performance optimisation tasks.
 5. **Stakeholder Updates:** Keep stakeholders informed about any potential impacts on the release schedule or product performance if significant risks are identified.

Module 3: Test Monitoring, Test Control and Test Completion



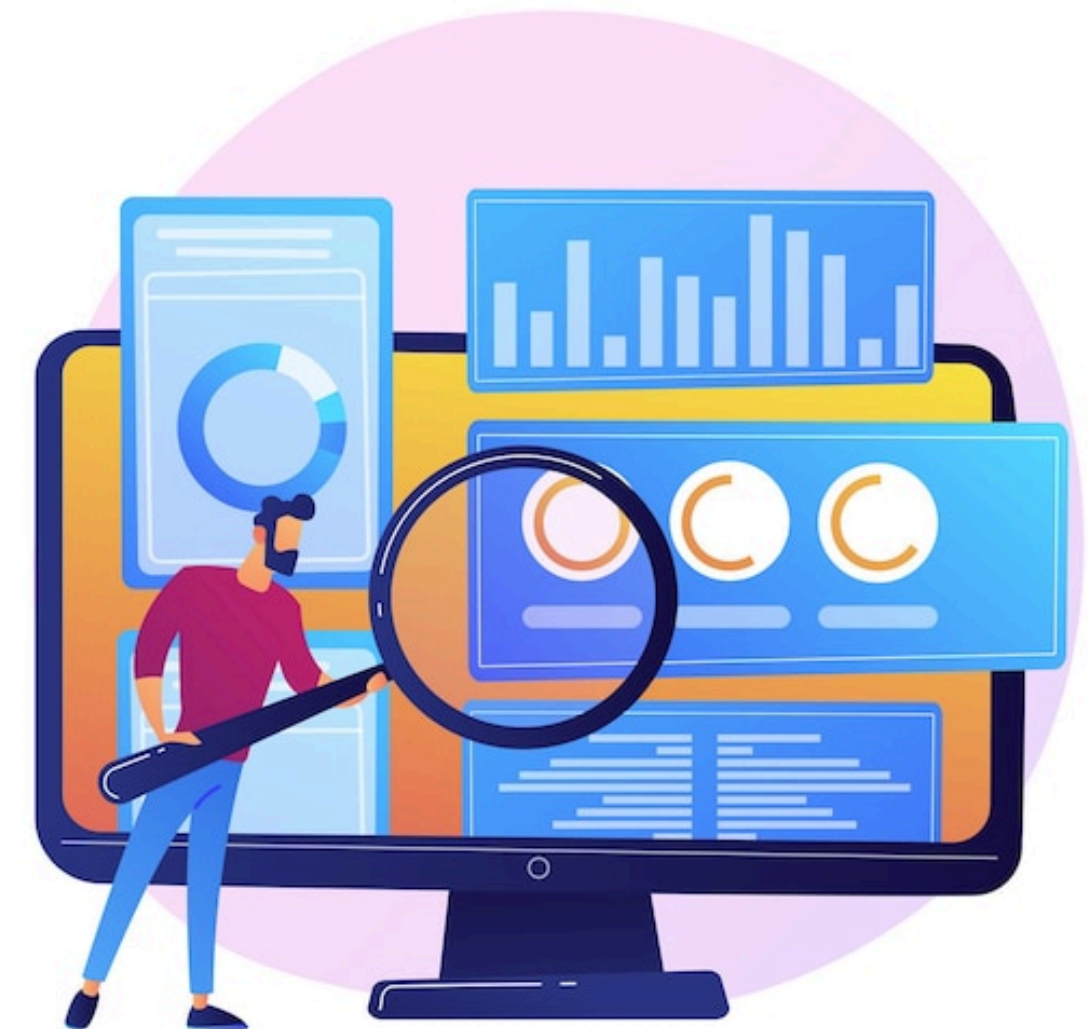
Test Monitoring

- Test monitoring is a pivotal aspect of the software testing process. It involves the systematic collection and analysis of data pertaining to testing activities. The primary goal of test monitoring is to evaluate the progress of testing efforts and determine if they align with predefined criteria for completion. These criteria could encompass coverage objectives for product risks, requirements, or acceptance criteria.
- Through continuous monitoring, testing teams can gauge how effectively their test activities are progressing and identify any deviations from the intended course. This process aids in making informed decisions about the testing strategy, allocation of resources, and necessary adjustments to meet quality objectives.
- Test monitoring acts as a navigational tool, guiding testing efforts toward successful project outcomes. It ensures that testing remains on track, validates that exit criteria are being met, and helps maintain transparency regarding testing progress. By harnessing the insights garnered from test monitoring, teams can adapt to changing circumstances, optimise their testing strategy, and ultimately contribute to the overall success of the project.

Test Monitoring

Test Control

- Test Control refers to the ongoing activities that involve monitoring the test progress, making decisions based on the information at hand, and taking appropriate actions to adjust the test process as necessary.
- It is a dynamic process that allows for the management and correction of the testing activities to align them with the project objectives.



Test Monitoring

Test Control

- Test control activities are crucial for several reasons, particularly when progress deviates from the plan:
 - **Responding to Deviations:** Test Control allows immediate response when testing activities deviate from the original plan, ensuring that objectives are still met efficiently.
 - **Managing Changes and Risks:** It enables adaptation to unforeseen changes and risks, maintaining the testing process's relevance and effectiveness.
 - **Resource Optimisation:** Test Control helps reallocate resources and adjust schedules when necessary, enhancing overall resource utilisation.
 - **Ensuring Quality and Compliance:** Adjusting testing based on actual progress and challenges, helps in maintaining the quality of the product and compliance with requirements.

Test Monitoring

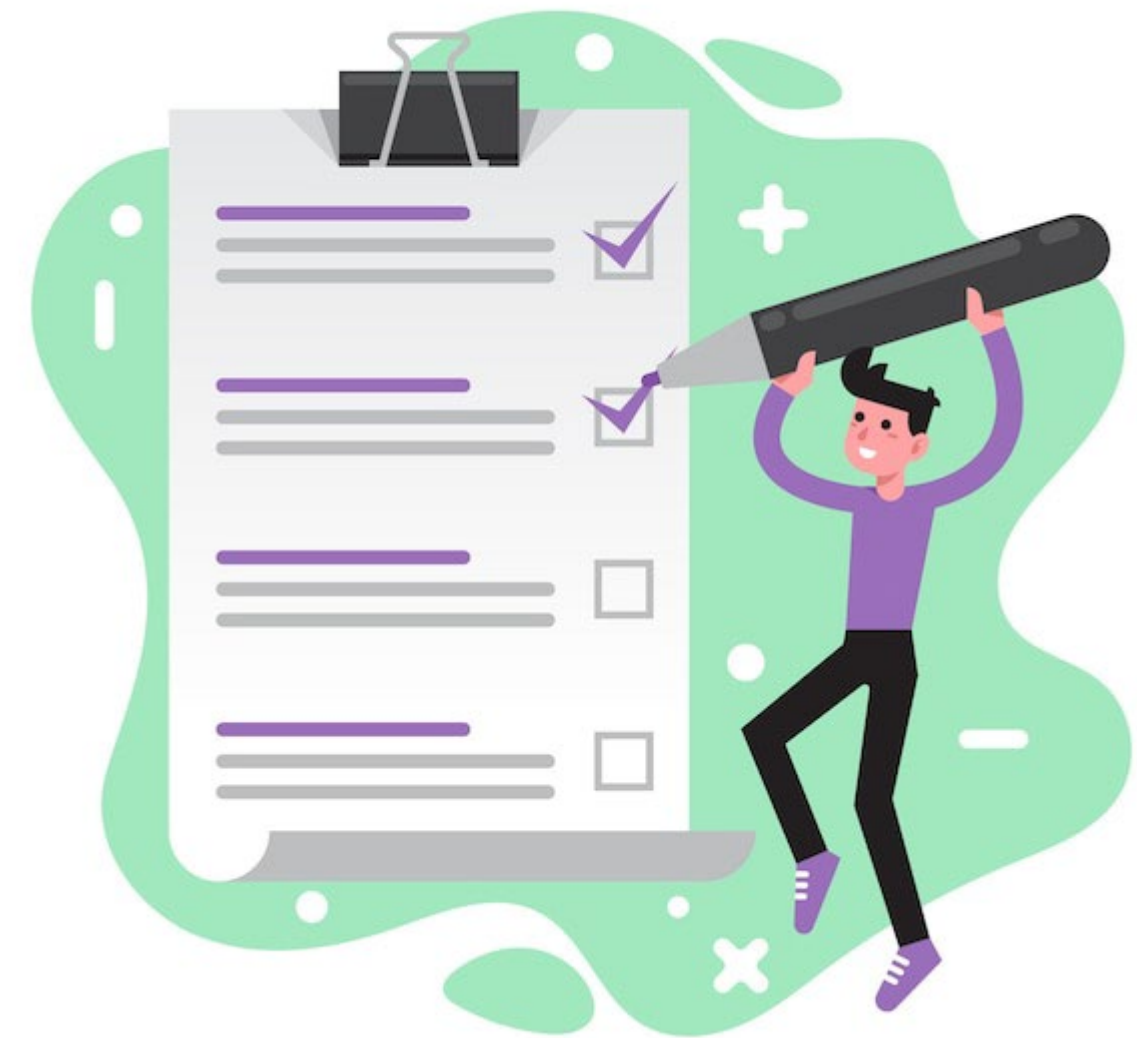
Activities involved in Test Control include:

- **Re-Prioritising Tests:** Adjusting priorities when project risks, such as late software delivery, are identified.
- **Changing the Test Schedule:** Modifying the schedule due to factors like availability or unavailability of the test environment and resources.
- **Re-Evaluating Entry and Exit Criteria:** Assessing whether a test item meets these criteria, especially after rework.
- **Review and Adjustment of Product Risks:** This might include changing risk ratings to meet targets.
- **Recommendations to Project Management:** Although not directly under the test leader's responsibility, they may recommend de-scoping functionality, delaying the release, or continuing testing post-delivery.

Test Monitoring

Test Completion

- ✓ Test Completion marks the end of the testing process for a particular phase of the software development lifecycle. It involves consolidating the results, ensuring that all test objectives have been achieved, and that the test artifacts are properly archived.
- ✓ It also includes the assessment of how the testing went and what could be improved for future projects.



Test Monitoring

Activities involved in Test Completion include:

- **Checking Test Completion Criteria:** Verifying that all planned tests have been run and that all exit criteria have been met.
- **Test Artifact Handover:** Ensuring that all test artifacts are completed and stored for future use or for audit purposes. This includes test cases, test results, defect reports, and test environments.
- **Test Summary Report:** Creating a final report that summarises the testing activities, including the quality of the product, defect statistics, and coverage of requirements.
- **Lessons Learned:** Conducting a lessons learned or post-mortem meeting to discuss what went well, what didn't, and what could be improved.
- **Release Decisions:** Providing information to stakeholders to help them decide whether the product is ready for release.

Test Completion is critical for learning from the current test cycle and improving the testing process for future projects. It also ensures accountability and provides clear documentation of the testing efforts, which can be critical for compliance, especially in regulated industries.

Test Monitoring

Test Monitoring

“The collection of information (metrics) either manually or automatically”.

- Provides feedback about test activities.
- Provides visibility of the test activities.
- Enables measurement of exit criteria
 - ✓ Decision coverage
 - ✓ Requirements coverage
 - ✓ Etc.
- Assesses progress
 - ✓ Against a planned schedule
 - ✓ Against the budget



Test Monitoring

Metrics Used in Testing

Test metrics are quantitative measures used to gauge various aspects of the software testing process, providing a data-driven foundation for assessing quality, performance, and progress.

Some common metrics:

- **Project Progress Metrics:** These metrics track the progress of the testing project as a whole. They include task completion status, resource usage, and overall test effort. Project progress metrics help evaluate if the testing activities are on schedule and within allocated resources.
- **Test Progress Metrics:** Test progress metrics focus on the progress of specific testing activities. They include test case implementation progress, the readiness of the test environment, the number of test cases executed (run/not run), test case pass/fail rates, and the time taken for test execution. These metrics provide insights into testing efficiency and coverage.

© The Knowledge Academy Ltd

Test metrics serve as indispensable tools for monitoring and controlling the testing activities, offering visibility into the effectiveness and efficiency of the testing efforts. By analysing test metrics, stakeholders, including testers, managers, and business leaders, can make well-informed decisions about resource allocation, test coverage, and overall project timelines.

Test Monitoring

(Continued)

- **Product Quality Metrics:** Product quality metrics assess the quality of the software product. They can include metrics related to availability, response time, and mean time to failure. These metrics help gauge the performance and reliability of the software.
- **Defect Metrics:** Defect metrics track the number and priorities of defects found and fixed during testing. They include defect density (defects per size of code or test cases), defect detection percentage (defects found during testing compared to total defects), and other defect-related measures. These metrics help in identifying the overall software quality and the effectiveness of the defect management process.
- **Risk Metrics:** Risk metrics measure the level of residual risk in the software. They provide insights into the potential impact and likelihood of risks identified during testing. Understanding risk metrics helps in making risk-informed decisions and prioritising testing efforts.

Test Monitoring

(Continued)

- **Coverage Metrics:** Coverage metrics assess the extent to which the software has been tested. They include requirements coverage (percentage of requirements tested), code coverage (percentage of code executed by test cases), and other coverage-related measures. Coverage metrics help ensure thorough testing of the software.
- **Cost Metrics:** Cost metrics include the cost of testing and the organisational cost of quality. These metrics help evaluate the efficiency of testing efforts and the overall cost-effectiveness of quality assurance activities.

Test Reporting

“The summarising of information about testing endeavour”

- **What happened during a period of testing**
 - such as dates when exit criteria were met.
- **Analysed information and metrics**
 - what has been tested
 - what worked
 - what did not work
 - what is fixed
 - what is not fixed
 - what has not yet been tested
 - outstanding risks

enables assessment of:

- defects outstanding
- the economic benefit of continuing
- outstanding risks
- level of confidence

Test Reporting

Purpose, Content and Audience for Test Reports

Purpose

- Test reports serve the purpose of effectively communicating the progress, findings, and outcomes of testing activities to stakeholders.
- They provide visibility into the testing process, present the results of executed test cases, highlight identified defects, and convey the overall quality of the software being tested.
- The primary goal is to inform stakeholders about the current state of the software and testing efforts.

Test Reporting

(Continued)

Content

- Test reports typically include a testing summary that outlines the objectives, scope, and timelines of the testing activities.
- The test execution status section provides details on the number of test cases executed, passed, failed, and remaining, offering insights into the progress of testing.
- The defect summary presents a summary of identified defects, including their severity, priority, and status, highlighting areas that require attention and potential risks.

Test Reporting

(Continued)

Audience

- The primary audience for test reports includes project managers, development teams, quality assurance teams, product owners, and other stakeholders involved in the software development and testing process.
- Project managers rely on reports to monitor progress and make informed decisions about resource allocation and project planning.
- Development and QA teams use the reports to understand the test results and address identified defects.

Communicating the Status of Testing

- Communicating the status of testing is a critical aspect of the software development process to ensure stakeholders are well-informed about the progress and quality of the testing effort.
- By effectively conveying the testing status, project teams can manage expectations, make informed decisions, and address any potential risks or issues.
- This communication involves providing clear and concise reports that highlight essential information such as the number of test cases executed, passed, failed, and remaining.
- Regular updates through meetings or written reports help stakeholders stay up-to-date with the latest testing progress.
- Utilising relevant metrics and key performance indicators (KPIs) allows for quantifiable data on test coverage, defect density, and test execution efficiency, providing stakeholders with an objective view of the testing progress.

Communicating the Status of Testing

Several ways to effectively communicate test status:

- Verbal Communication
- Dashboards
- Electronic Communication Channels
- Online Documentation
- Formal Test Reports



© The Knowledge Academy Ltd

Verbal Communication:

Direct verbal communication with team members and stakeholders can offer real-time updates on test progress, challenges, and achievements. This approach fosters open dialogue, enables quick feedback, and ensures that everyone is on the same page.

Dashboards:

Dashboards, including Continuous Integration/Continuous Deployment (CI/CD) dashboards, task boards, and burn-down charts, provide visual representations of testing status. These visuals offer quick insights into test progress, areas needing attention, and overall project health.

Electronic Communication Channels:

Email and chat platforms facilitate asynchronous communication, enabling the distribution of testing updates, reports, and queries. These channels help keep stakeholders informed and engaged, even when direct interaction is not possible.

Online Documentation:

Online documentation, such as wikis or shared documents, can serve as repositories of detailed testing information. This approach allows stakeholders to access comprehensive information at their convenience and encourages collaboration.

Formal Test Reports:

Formal test reports provide structured and comprehensive summaries of testing activities, results, and outcomes. These reports can be essential for stakeholders requiring in-depth insights into testing progress and quality.

Module 4: Configuration Management



Configuration Management

- Configuration Management, in the context of software testing, refers to the process of managing and controlling the software configuration items (SCIs) throughout the software development and testing lifecycle.
- It involves maintaining the integrity, consistency, and traceability of the software and its associated components, including source code, test scripts, documentation, and test environments.

Here are some key aspects of Configuration Management related to software testing:

A. Version Control

- ✓ Configuration Management ensures that all software components and artifacts are properly versioned and controlled.
- ✓ This involves using version control systems to track changes, maintain a history of modifications, and facilitate collaboration among team members.

Configuration Management helps in maintaining consistency across various elements of the software development life cycle, including code, documentation, test artifacts, and tools. By keeping track of all changes and versions, Configuration Management enables teams to understand what was tested, how it was tested, and under which configuration. This is especially crucial in environments where multiple configurations, platforms, or versions of the software exist.

Configuration Management

(Continued)

- ✓ Version control enables testers to work with specific versions of software and associated test assets, ensuring reproducibility and traceability.

B. Baseline Management

- ✓ Configuration Management establishes baselines, which are defined snapshots of the software and its associated configuration items at specific points in time.
- ✓ Baselines serve as reference points for testing and help maintain stability during the testing process.
- ✓ Testers can compare the baselined software with subsequent versions to identify any unintended changes or discrepancies.

Configuration Management

(Continued)

C. Configuration Identification

- ✓ Configuration Management involves identifying and documenting the configuration items that make up the software system.
- ✓ This includes creating a configuration management plan, defining naming conventions, and establishing a configuration item hierarchy.
- ✓ Clear identification and documentation of configuration items aid in maintaining consistency and facilitating effective change management.

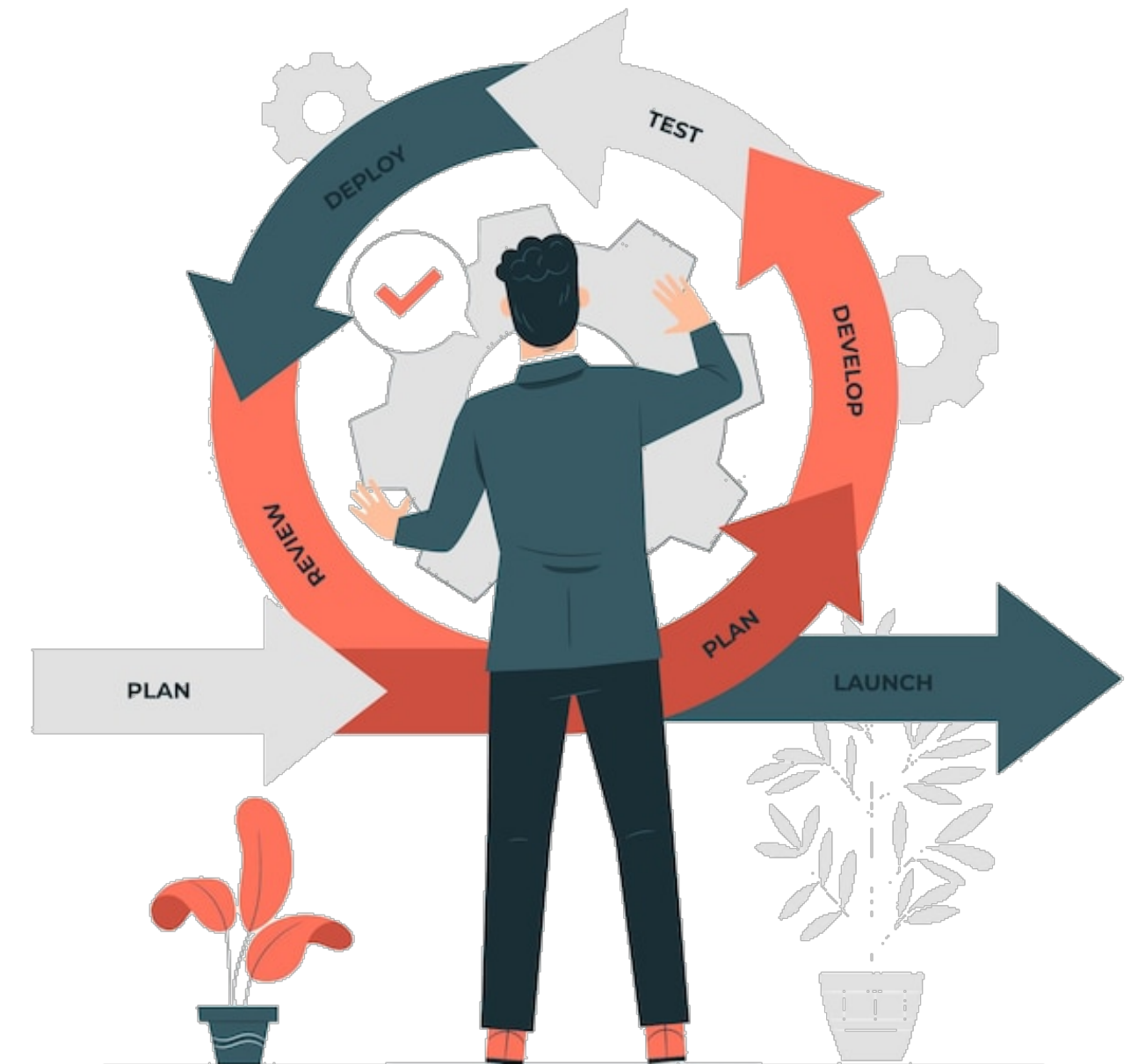


Configuration Management

(Continued)

D. Change Control

- ✓ Configuration Management includes managing and controlling changes to the software and its associated components.
- ✓ This involves a formal change control process, where proposed changes are evaluated, prioritised, and approved based on their impact and alignment with project objectives.
- ✓ Effective change control helps prevent unauthorised changes, ensures proper documentation, and facilitates accurate tracking of changes during testing.



Configuration Management

(Continued)

E. Configuration Status Accounting

- ✓ Configuration Management involves maintaining accurate and up-to-date records of the status and history of configuration items.
- ✓ This includes documenting the current versions, their relationships, and the associated changes and approvals
- ✓ Configuration status accounting provides visibility into the software's configuration, facilitates auditing, and aids in determining the components to be used for testing.



Configuration Management

(Continued)

F. Configuration Audits and Reviews

- ✓ Configuration Management includes conducting audits and reviews to assess the completeness, consistency, and compliance of the software configuration.
- ✓ These activities help ensure that the software, test environments, and related assets are properly configured, adhere to defined standards, and meet the required specifications:
 - By implementing effective Configuration Management practices, software testing teams can ensure the integrity and reliability of the software and its associated components throughout the testing process
 - It facilitates accurate and controlled testing, improves traceability, and enables efficient management of changes and versions, ultimately contributing to the overall quality and success of the software product

Configuration Management

- Configuration Management plays a crucial role in supporting testing activities throughout the software development lifecycle. It ensures proper version control of all test artefacts, including test plans, cases, data, and scripts, enabling testers to work with accurate and up-to-date assets.
- Additionally, Configuration Management helps manage and control test environments, ensuring they match the production environment, reducing the risk of environmental discrepancies affecting test results.
- By establishing baselines of software and test assets, it facilitates traceability between requirements, test cases, and defects, improving test coverage tracking and reporting. Moreover, Configuration Management allows for the documentation and control of changes, ensuring testers can plan their efforts around evolving software versions.
- It also aids in creating reproducible test scenarios, helping developers diagnose and resolve defects effectively. Through maintaining multiple test configurations, Configuration Management supports parallel testing efforts, speeding up the testing process.

Module 5: Defect Management



Defect Management

- Defect Management, as a crucial component of test planning, involves the identification, tracking, and resolution of defects or issues encountered during the testing process.
- It encompasses a systematic approach to capturing, documenting, prioritising, and managing defects to ensure they are addressed effectively.
- Under test planning, Defect Management plays a vital role in ensuring that identified issues are appropriately recorded, communicated, and resolved.
- It involves establishing processes and tools for defect tracking, assigning ownership, and defining workflows for defect triage, analysis, and resolution.
- It enables systematic identification, tracking, and resolution of defects, ensuring the delivery of high-quality software through effective defect triage and mitigation strategies.

Defect Management

Defect Report

- In the realm of software testing, the primary goal is often to identify and rectify defects. To achieve this efficiently, a comprehensive defect management process is indispensable. Although these inconsistencies are termed "defects", it's crucial to discern that not all of them might indeed be genuine defects; some might be false alarms or even change requests.
- The true nature of such anomalies is clarified during the scrutiny of the defect reports. Intriguingly, these defects can surface at any juncture of the Software Development Life Cycle (SDLC), and the manner of reporting them is contingent on the specific SDLC phase they arise in.
- A foundational defect management system embodies a workflow detailing the defect's journey, right from its identification to its eventual resolution. It also incorporates protocols for classifying the defects based on parameters like severity and impact. This workflow generally encapsulates steps like logging the defect, analysing its nature, deciding the remedial course of action, and finally closing the report post resolution.

© The Knowledge Academy Ltd

- Defect Management, as a crucial component of test planning, involves the identification, tracking, and resolution of defects or issues encountered during the testing process. It encompasses a systematic approach to capturing, documenting, prioritising, and managing defects to ensure they are addressed effectively.
- Under test planning, Defect Management plays a vital role in ensuring that identified issues are appropriately recorded, communicated, and resolved. It involves establishing processes and tools for defect tracking, assigning ownership, and defining workflows for defect triage, analysis, and resolution. It enables systematic identification, tracking, and resolution of defects, ensuring the delivery of high-quality software through effective defect triage and mitigation strategies.

Defect Management

Objectives of Typical Defect Reports:

- **Informational Utility:** The primary objective of a defect report is to arm those in charge of addressing and rectifying the reported defects with all the necessary details. This means providing comprehensive information about when, where, and under what conditions the defect was discovered. By doing so, it enables a smoother and more efficient resolution process, ensuring that developers or those tasked with resolution have a clear understanding of the problem at hand.
- **Quality Assessment:** Defect reports also function as a barometer to gauge the quality of the work product. By monitoring the number, frequency, and severity of defects logged, stakeholders can get an idea of the product's current quality status. If, for instance, a particular module continually flags a high number of critical defects, it can indicate deeper systemic issues that need urgent attention.

Defect Management

Objectives of Typical Defect Reports:

- **Process Enhancement:** Beyond just identification and resolution, defect reports also play a crucial role in the continuous improvement of the development and testing processes. By studying patterns in defects and understanding their root causes, teams can derive insights into areas of the development and test process that might be lacking. This feedback loop, fueled by data from the defect reports, paves the way for refining methodologies, improving tools, or even retraining teams, ensuring that future iterations of the product or other projects benefit from these learnings.



Defect Management

Components of a Defect Report Logged During Dynamic Testing:

- **Unique Identifier:** Every defect report is given a distinct code or number. This ensures easy referencing and tracking of the defect across the resolution lifecycle.
- **Title with Summary:** A concise header that encapsulates the essence of the anomaly, allowing readers to quickly grasp the nature of the defect.
- **Report Details:** This section covers the date the defect was discovered, the organisation logging it, and details about the author, including their specific role. This offers context about the origin and the credibility of the report.
- **Test Object and Environment Identification:** Here, the specific module or component (test object) and the testing environment in which the defect was observed are documented. This helps in recreating the conditions for defect verification and resolution.
- **Defect Context:** This elaborates on the scenario under which the defect arose, such as the test case in progress, the associated test activity, the SDLC phase, and other pertinent details like the testing technique or checklist in use.

© The Knowledge Academy Ltd

A Defect Report logged during Dynamic Testing serves as a formal record of any anomalies, issues, or bugs identified while the software application is running. This report is crucial for communicating the nature of the defect to developers, test managers, and other stakeholders involved in the project. Typically, the report will include key information such as the defect identification number, description of the issue, steps to reproduce the defect, expected and actual outcomes, severity and priority levels, and any relevant screenshots or log files.

Defect Management

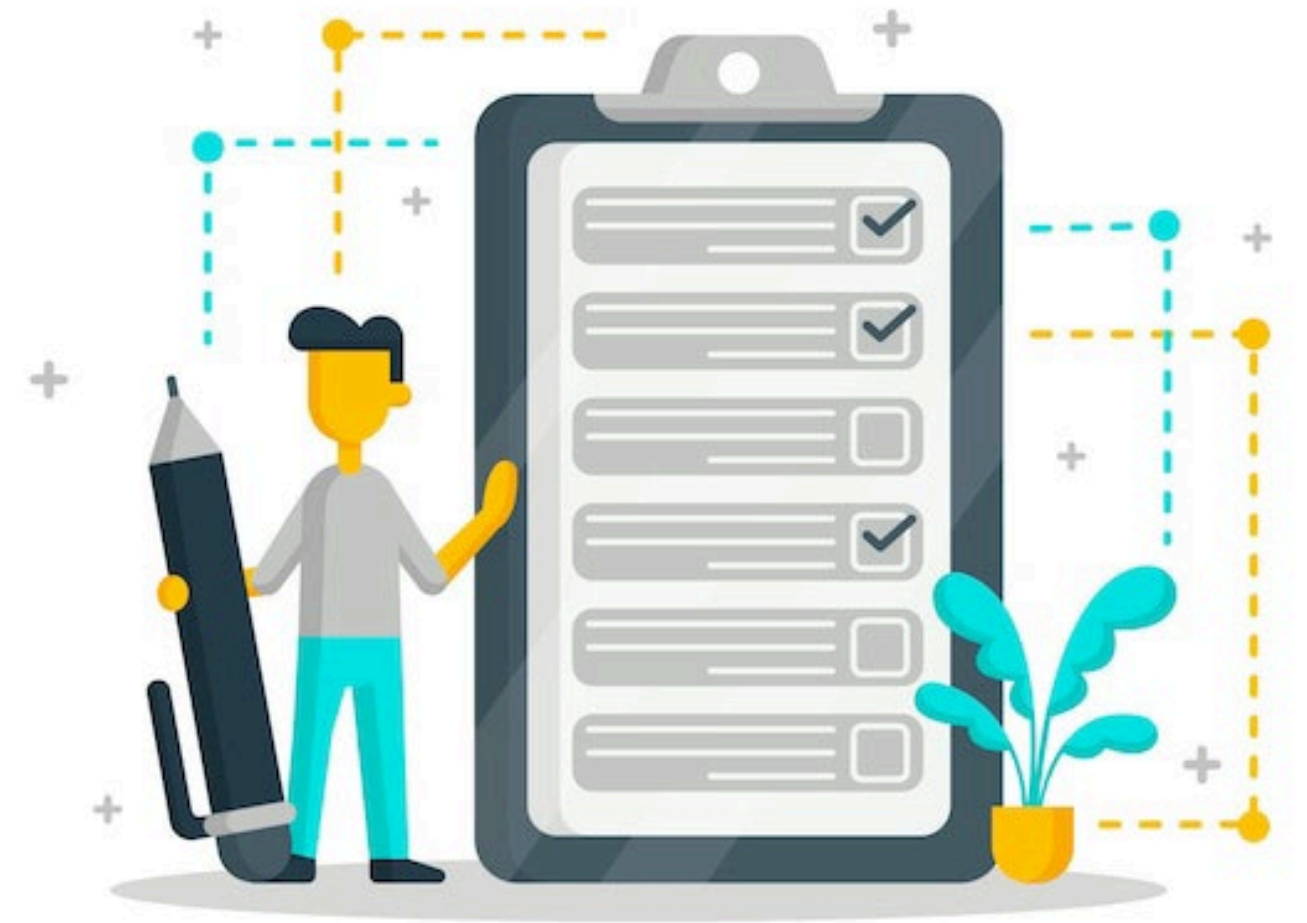
Components of a Defect Report Logged During Dynamic Testing:

- **Defect Description:** A thorough account of the anomaly, detailing the steps leading to its detection. Supplementary data, like test logs, screenshots, or recordings, can be immensely helpful for those tasked with rectification, offering them a vivid picture of the issue.
- **Expected Vs Actual Results:** A juxtaposition of what the outcome should have been against what was observed. This clarity aids in discerning the deviation and its implications.
- **Severity:** This indicates the impact level of the defect, typically in terms of how it affects stakeholders' interests or predefined requirements. It gives an idea of the defect's potential repercussions if left unaddressed.
- **Priority to Fix:** While 'severity' denotes the impact, 'priority' denotes the urgency to fix the defect. For instance, a severe defect might not be urgent to fix if it's in a less critical module.
- **Defect Status:** This keeps track of the defect's current state in its lifecycle, such as whether it's open, under review, being fixed, or closed. It's essential for managing the defect until its resolution.

Exercise

To dive deeper into practical application, refer to the corresponding exercise in the document for hands-on exploration and analysis.

Exercise 12: Creating a Defect Report for a Web-Based CRM System

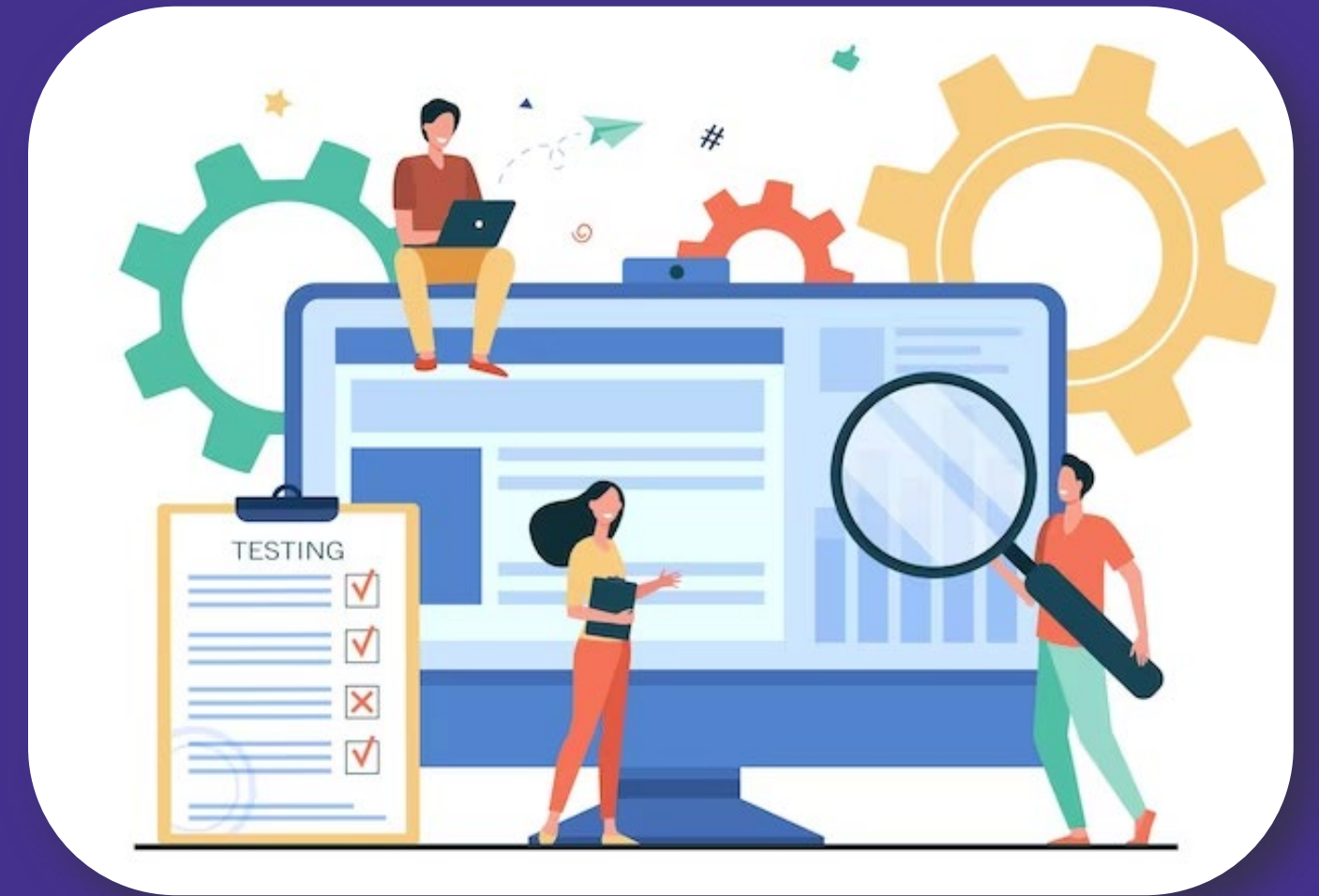


Domain 6: Test Tools

Outline of Domain 6

- *Module 1: Tool Support for Testing*
- *Module 2: Benefits and Risks of Test Automation*

Module 1: Tool Support for Testing



The Meaning and Purpose of Tool Support

- **Improve efficiency of test activities**
 - automating repetitive tasks
 - supporting manual activities
- **Automating activities that require significant resources**
 - e.g. static testing
- **Automating activities that could not be done manually**
 - e.g. large scale performance testing
- **Increasing reliability of testing**
 - e.g. by automating large data comparisons

Tool support for testing can have one or more of the following purposes depending on the context:

- Improve the efficiency of test activities by automating repetitive tasks or supporting manual test activities like test planning, test design, test reporting and monitoring
- Automate activities that require significant resources when done manually (e.g., static testing)
- Automate activities that can not be executed manually (e.g., large scale performance testing of client-server applications)
- Increase reliability of testing (e.g., by automating large data comparisons or simulating behaviour)

This is basically IT looking at itself and taking the principles of automation that it would apply to business systems and applying them to testing systems.

All the above can be done manually but are incredibly tedious and very time consuming

Tool Support for Testing

- Tool support for testing plays a significant role by enabling testers to effectively and efficiently perform their testing activities. Various types of tools are available that assist testers throughout the testing lifecycle.
- ***Test tools provide support and enable various test activities. These tools encompass a wide range of examples, with the following being just a few illustrations:***



1. Management Tools
2. Static Testing Tools
3. Test Design and Implementation Tools
4. Test Execution and Coverage Tools

© The Knowledge Academy Ltd

1. **Management Tools:** Test management tools assist in organising and tracking test cases, test plans, test schedules, and test resources. They provide features for test case management, requirement traceability, progress monitoring, and reporting.
2. **Static Testing Tools:** Static testing tools analyse the source code, documentation, or other artifacts without executing them. They help identify defects, coding errors, adherence to coding standards, and potential issues early in the development process. Examples include code review tools, syntax checkers, and static analysis tools.
3. **Test Design and Implementation Tools:** These tools support the creation and development of test cases and test scripts. They often provide features like test specification, test data generation, test script generation, and test code debugging. These tools can be specific to programming languages, automation frameworks, or testing methodologies.
4. **Non-functional Testing Tools:** Non-functional testing tools focus on evaluating aspects such as performance, security, reliability, usability, and compatibility. Examples include load testing tools, security testing tools, usability testing tools, and compatibility testing tools. These tools help simulate real-world scenarios and assess the system's behavior under different conditions.

Tool Support for Testing



5. Non-functional testing Tools
6. DevOps Tools
7. Collaboration Tools
8. Virtual Machines
9. Containerisation Tools
10. Spreadsheet Tools

5. **Non-functional Testing Tools:** These tools are essential for ensuring the robustness, performance, and security of software applications. For instance, JMeter and LoadRunner specialise in load testing, simulating high traffic or data load to assess performance under stress.
6. **DevOps Tools:** DevOps tools support the integration of testing into the continuous integration and continuous delivery pipelines. They help automate the testing process, enable test environment provisioning, facilitate deployment and configuration management, and promote collaboration between development and operations teams.
7. **Collaboration Tools:** Collaboration tools enhance communication and collaboration among testing teams and stakeholders. They can include issue tracking systems, project management tools, wikis, document sharing platforms, and instant messaging platforms. These tools promote teamwork, knowledge sharing, and efficient coordination among team members.
8. **Virtual Machines (VMs):** Virtual machines are software-based emulations of physical computers. They allow the creation of multiple isolated environments on a single physical machine, enabling the execution of different operating systems and software configurations.
9. **Containerisation Tools:** It is a lightweight virtualisation technique that packages an application and its dependencies together into a container. Containers provide a consistent and portable runtime environment, ensuring that the software runs the same across different systems. Popular containerisation tools like Docker allow testers to encapsulate their applications and dependencies into containers, making it easier to deploy, test, and scale applications across various environments.
10. **Spreadsheet tools,** Microsoft Excel, Google Sheets, or other similar applications, serve as valuable tool support for testing in various aspects of the software testing lifecycle.

Tool Support for Testing

- **Management Tools:** Test management tools assist in organising and tracking test cases, test plans, test schedules, and test resources. They provide features for test case management, requirement traceability, progress monitoring, and reporting.
- **Static Testing Tools:** Static testing tools analyse the source code, documentation, or other artifacts without executing them. They help identify defects, coding errors, adherence to coding standards, and potential issues early in the development process. Examples include code review tools, syntax checkers, and static analysis tools.
- **Test Design and Implementation Tools:** These tools support the creation and development of test cases and test scripts. They often provide features like test specification, test data generation, test script generation, and test code debugging. These tools can be specific to programming languages, automation frameworks, or testing methodologies.
- **Test Execution and Coverage Tools:** Test execution tools enable the automated execution of test cases, capturing test results, and reporting. They can integrate with test management tools to retrieve test cases and store test results. Test coverage tools track the portions of the code or system that have been exercised during testing to measure the completeness of testing.

Tool Support for Testing

- **Non-functional Testing Tools:** Non-functional testing tools focus on evaluating aspects such as performance, security, reliability, usability, and compatibility. Examples include load testing tools, security testing tools, usability testing tools, and compatibility testing tools. These tools help simulate real-world scenarios and assess the system's behavior under different conditions.
- **DevOps Tools:** DevOps tools support the integration of testing into the continuous integration and continuous delivery (CI/CD) pipelines. They help automate the testing process, enable test environment provisioning, facilitate deployment and configuration management, and promote collaboration between development and operations teams.
- **Collaboration Tools:** Collaboration tools enhance communication and collaboration among testing teams and stakeholders. They can include issue tracking systems, project management tools, wikis, document sharing platforms, and instant messaging platforms. These tools promote teamwork, knowledge sharing, and efficient coordination among team members.

Tool Support for Testing

- **Virtual Machines (VMs):** Virtual machines are software-based emulations of physical computers. They allow the creation of multiple isolated environments on a single physical machine, enabling the execution of different operating systems and software configurations. In software testing, VMs are valuable for creating standardised testing environments, enabling testers to test their software on various platforms without the need for separate physical hardware. VM snapshots can also be used to revert to a known state after testing, facilitating test repeatability.
- **Containerisation Tools:** Containerisation is a lightweight virtualisation technique that packages an application and its dependencies together into a container. Containers provide a consistent and portable runtime environment, ensuring that the software runs the same across different systems. Popular containerisation tools like Docker allow testers to encapsulate their applications and dependencies into containers, making it easier to deploy, test, and scale applications across various environments.

Tool Support for Testing

- **Spreadsheet Tools**, such as Microsoft Excel or Google Sheets, are versatile tools used as effective support in various aspects of software testing:
- Test Case Management
- Test Data Management
- Defect Tracking
- Test Execution Logs
- Test Reports and Metrics
- Data Comparison and Validation
- Traceability Matrix

Test Case Management: Testers use spreadsheets to organise and manage test cases. They can create a structured layout with columns for test case ID, description, steps, expected results, actual results, status, and other relevant details.

Test Data Management: Spreadsheets can be used to create and manage test data sets. Testers can input various test data combinations, making it easier to execute multiple test scenarios.

Defect Tracking: Testers can use spreadsheets to track defects identified during testing. Columns can include defect ID, description, priority, status, assigned to, and resolution.

Test Execution Logs: During test execution, testers may log results in spreadsheets, indicating which test cases have been executed, passed, failed, or deferred.

Test Reports and Metrics: Spreadsheets can be used to generate test summary reports, metrics, and graphs to communicate testing progress, test coverage, defect trends, and other key performance indicators.

Data Comparison and Validation: Testers can utilise spreadsheets to compare data sets or perform data validation tasks, especially when dealing with large volumes of data.

Traceability Matrix: Spreadsheets can be employed to maintain traceability between requirements, test cases, and defects, ensuring that all requirements are adequately tested.

Module 2: Benefits and Risks of Test Automation



Benefits and Risks of Test Automation

Benefits of Test Automation

- **Time Saved by Reducing Repetitive Manual Work:** Automation significantly reduces the time and effort spent on repetitive tasks, such as executing regression tests, re-entering test data, comparing expected results with actual results, and checking against coding standards. Automated tests can be run swiftly and consistently, freeing up testers' time to focus on more critical testing activities.
- **Prevention of Simple Human Errors Through Greater Consistency and Repeatability:** Automated tests ensure consistency and repeatability, reducing the likelihood of human errors caused by manual test execution. Tests are consistently derived from requirements, test data is created systematically, and tests are executed in the same order and frequency every time.
- **More Objective Assessment and Measurement of Test Coverage:** Automation enables more objective assessment of test coverage, identifying areas that need further testing. Test coverage metrics, which might be complex for humans to derive manually, can be easily generated by automation tools.

Benefits and Risks of Test Automation

Benefits of Test Automation

- **Easier Access to Information About Testing to Support Test Management and Reporting:** Test automation provides access to detailed and up-to-date testing information, such as statistics, graphs, and aggregated data about test progress, defect rates, and test execution duration. This information facilitates effective test management and reporting to stakeholders.
- **Reduced Test Execution Times for Earlier Defect Detection and Faster Feedback:** Automated tests can be executed much faster than manual tests, allowing for earlier detection of defects and providing faster feedback to developers. This leads to quicker bug resolution and more efficient development cycles.
- **More Time for Testers to Design New, Deeper, and More Effective Tests:** By automating routine and repetitive test cases, testers have more time to focus on designing new, complex, and exploratory tests. This enables them to explore deeper aspects of the software and uncover potential issues that may not be easily identified through automated scripts.

Benefits and Risks of Test Automation

Risks of Test Automation

- **Unrealistic Expectations about Tool Benefits:** There is a risk of setting unrealistic expectations about the benefits of a test automation tool, including its functionality and ease of use. Organisations may expect immediate and extensive returns, leading to disappointment if the tool does not meet their specific needs.
- **Inaccurate Estimations of Time, Costs, and Effort:** Poor estimation of the time, costs, and effort required to introduce and maintain the test automation tool can lead to budget overruns, delayed schedules, and resource allocation issues.
- **Using a Test Tool When Manual Testing is More Appropriate:** Not all testing scenarios are suitable for automation. Choosing automation when manual testing is more appropriate can result in wasted effort, time, and resources.
- **Over-Reliance on Automation:** Relying solely on automation can overlook the importance of human critical thinking in testing. Testers should exercise judgment, creativity, and exploratory testing to identify potential defects that automated scripts might miss.

Benefits and Risks of Test Automation

Risks of Test Automation

- **Dependency on the Tool Vendor:** There is a risk of depending heavily on a specific test automation tool and its vendor. If the vendor goes out of business, retires the tool, sells it to a different vendor, or provides poor support, it can lead to disruption in testing efforts.
- **Issues with Open-Source Software:** Using open-source test automation tools may pose risks if the software is abandoned or lacks regular updates. The need for frequent updates or changes to internal components can increase maintenance efforts.
- **Incompatibility with Development Platform:** If the chosen automation tool is not compatible with the development platform or technology stack, it can hinder integration and the overall testing process.
- **Choosing an Unsuitable Tool for Regulatory and Safety Compliance:** Using an automation tool that does not comply with regulatory requirements or safety standards can lead to non-compliance issues and compromise the integrity of testing results.



Congratulations



*the*knowledgeacademy

Contact Us

info@theknowledgeacademy.com

www.theknowledgeacademy.com/tickets

<https://uk.trustpilot.com/review/theknowledgeacademy.com>